MONOGRAFÍA APLICADA

Forecasting de Series Temporales en Ocho Dominios Industriales

Del Baseline Ingenuo al Forecasting Probabilístico: Una Monografía Aplicada sobre `course-timeseries-benchmarks-pro`

Autor: *[Autor pendiente de confirmación]*
Fecha: *[Fecha de entrega pendiente de confirmación]*
Repositorio de referencia: *`course-timeseries-benchmarks-pro`, versión 1.1.0*

Resumen

Esta monografía documenta y sintetiza ocho experimentos de forecasting ejecutados sobre datasets reales de tráfico, energía, calidad del aire, movilidad urbana, consumo eléctrico industrial y benchmarks de competencia (M3). La progresión metodológica avanza desde baselines ingenuos (naive) y modelos estadísticos clásicos (ARIMA, SARIMAX) hasta modelos de machine learning tabular, arquitecturas de deep learning (LSTM, N-BEATS, Transformers compactos) y forecasting probabilístico con prácticas de MLOps. Cada capítulo técnico reproduce el contrato de datos, la estrategia de validación temporal, el modelado y el diagnóstico de residuos de su notebook correspondiente, y cierra con lecciones metodológicas verificadas contra la ejecución real, no contra resúmenes secundarios. El hallazgo transversal más consistente es que la complejidad no garantiza mejora: en cuatro de los ocho experimentos, el modelo más sofisticado iguala o apenas supera a un baseline ingenuo, mientras que en los casos donde sí hay ganancias claras (feature engineering con embargo temporal, forecasting probabilístico calibrado) la mejora es sustancial y verificable. La monografía concluye con una jerarquía de complejidad fundamentada en evidencia y una reflexión sobre los requisitos de gobierno necesarios para llevar estos modelos a producción real.

Palabras clave: forecasting, series temporales, ARIMA, SARIMAX, LightGBM, LSTM, N-BEATS, Temporal Fusion Transformer, forecasting probabilístico, MLOps, validación temporal, walk-forward.

Índice general

Resumen 1
Índice de tablas 6
Índice de figuras 9
Capítulo 1 — Marco teórico-metodológico y fundamentos 10
1.1 Contexto del forecasting 10
1.2 Fundamento estadístico del pronóstico 11
1.3 Principios de diseño del repositorio 14
1.4 Arquitectura de cuatro capas 15
1.5 Sistema de reproducibilidad 16
1.6 Taxonomía de los ocho datasets 16
1.7 Mapa de ruta de modelos 18
1.8 Preguntas de investigación 20
1.9 Trazabilidad con el sílabo 20
1.10 Metodología del estudio de caso 21
1.11 Estudio de caso integrador 22
1.12 Cómo leer esta monografía para profesionales de industria 23
1.13 Declaración de posicionamiento y madurez de la evidencia 24
1.14 Ética, integridad y declaración de uso de IA 25
Capítulo 2 — Semana 1: EDA, baselines y estacionalidad 26
2.1 Fundamentos teóricos aplicados 26
2.2 Configuración y contrato de datos 29
2.3 Análisis exploratorio temporal 29
2.4 Estrategia de validación 31
2.5 Modelado 32
2.6 Análisis posterior: diagnóstico de residuos 33
2.7 Lecciones metodológicas 34
2.8 Deudas abiertas del experimento 34
Capítulo 3 — Semana 2: ARIMA, estacionariedad y diagnóstico 35

3.1 Fundamentos teóricos aplicados 35
3.2 Configuración y contrato de datos 37
3.3 Análisis previo (pre-modeling) 38
3.4 Estrategia de validación 39
3.5 Modelado 39
3.6 Análisis posterior (post-modeling) 41
3.7 Lecciones metodológicas 42
3.8 Deudas abiertas del experimento 42
Capítulo 4 — Semana 3: SARIMAX y variables exógenas 44
4.1 Fundamentos teóricos aplicados 44
4.2 Configuración y contrato de datos 48
4.3 Análisis previo (pre-modeling) 49
4.4 Estrategia de validación 50
4.5 Modelado 51
4.6 Análisis posterior (post-modeling) 52
4.7 Lecciones metodológicas 53
4.8 Deudas abiertas del experimento 54
Capítulo 5 — Semana 4: Feature engineering y Machine Learning (ML) 55
5.1 Fundamentos teóricos aplicados 55
5.2 Configuración y contrato de datos 58
5.3 Análisis previo (pre-modeling) 59
5.4 Estrategia de validación 59
5.5 Modelado 60
5.6 Análisis posterior (post-modeling) 62
5.7 Lecciones metodológicas 63
5.8 Deudas abiertas del experimento 63
Capítulo 6 — Semana 5: Modelos globales y cross-learning 64
6.1 Fundamentos teóricos aplicados 64
6.2 Configuración y contrato de datos 66
6.3 Análisis previo (pre-modeling) 67
6.4 Estrategia de validación 67
6.5 Modelado 68

6.6 Análisis posterior (post-modeling) 70
6.7 Lecciones metodológicas 70
6.8 Deudas abiertas del experimento 71
Capítulo 7 — Semana 6: Deep Learning con LSTM 72
7.1 Fundamentos teóricos aplicados 72
7.2 Configuración y contrato de datos 74
7.3 Análisis previo (pre-modeling) 75
7.4 Estrategia de validación 75
7.5 Modelado 76
7.6 Análisis posterior (post-modeling) 77
7.7 Lecciones metodológicas 78
7.8 Deudas abiertas del experimento 79
Capítulo 8 — Semana 7: Arquitecturas modernas y transferencia entre series 80
8.1 Fundamentos teóricos aplicados 80
8.2 Configuración y contrato de datos 83
8.3 Análisis previo (pre-modeling) 84
8.4 Estrategia de validación 84
8.5 Modelado 85
8.6 Análisis posterior (post-modeling) 87
8.7 Lecciones metodológicas 88
8.8 Deudas abiertas del experimento 89
Capítulo 9 — Semana 8: Forecasting probabilístico y MLOps 90
9.1 Fundamentos teóricos aplicados 90
9.2 Configuración y contrato de datos 93
9.3 Análisis previo (pre-modeling) 93
9.4 Estrategia de validación 94
9.5 Modelado 94
9.6 Análisis posterior (post-modeling) 96
9.7 Lecciones metodológicas 98
9.8 Deudas abiertas del experimento 99
Capítulo 10 — Síntesis, lecciones transversales y conclusiones 100
10.1 Tabla sinóptica de los ocho experimentos 100

10.2 Respuesta a las preguntas de investigación 102
10.3 Análisis transversal: la asimetría de palancas 103
10.4 Jerarquía de palancas, no de técnicas 104
10.5 Cuatro lecciones transversales (de longitud desigual) 104
10.6 Matriz de decisión (sustituye a las decisiones por capítulo) 105
10.7 Limitaciones con dientes 106
10.8 Deudas prioritarias 107
10.9 Traducción a costo total de propiedad (TCO) 108
10.10 Recomendación operativa + cierre 109
Apéndice A — Tabla de datasets 111
Apéndice B — Tabla de modelos 112
Apéndice C — Entorno computacional 113
Apéndice D — Methodology Cards unificadas 117
Apéndice E — Glosario 118
Apéndice F — Bibliografía 119
Referencias metodológicas y de herramientas 119
Fuentes de los datasets 120

Índice de tablas

Tabla 1 Arquitectura de cuatro capas del repositorio. 15

Tabla 2 Mecanismos de reproducibilidad del repositorio. 16

Tabla 3 Resumen de los ocho datasets utilizados, en el orden en que se introducen a lo largo del curso. 17

Tabla 4 Ruta de complejidad creciente de modelos, con veredicto empírico del curso. 19

Tabla 5 Trazabilidad entre las competencias del sílabo y los capítulos de la monografía. . . . 21

Tabla 6 Esquema y rol de cada columna del dataset de tráfico. 29

Tabla 7 Tests de estacionariedad sobre el nivel y sobre la diferencia semanal. 30

Tabla 8 Contraste entre partición aleatoria (inválida para forecasting) y partición temporal. 31

Tabla 9 Comparación de baselines sobre el bloque de test (8,115 observaciones), ordenada por MASE. 32

Tabla 10 Diagnóstico de residuos del mejor baseline. 33

Tabla 11 Contrato de disponibilidad futura de covariables candidatas. 38

Tabla 12 Tests de estacionariedad sobre el nivel y tres transformaciones candidatas. 38

Tabla 13 Los cinco candidatos con menor AIC, evaluados en validation. 40

Tabla 14 Resumen del ajuste ARIMA(1,0,2) sobre train+validation. 40

Tabla 15 Comparación final de modelos sobre el bloque de test, ordenada por MASE. 40

Tabla 16 Diagnóstico de residuos de ARIMA(1,0,2). 41

Tabla 17 Error absoluto segmentado por nivel de la serie en test (cuartiles de y). 42

Tabla 18 Matriz de disponibilidad de covariables: histórico, producción, pronóstico externo y recomendación. 49

Tabla 19 Tests de estacionariedad sobre la serie de inspección. 50

Tabla 20 Resultados por fold: SARIMA (sin exógenas) vs. SARIMAX (con exógenas observadas). 51

Tabla 21 Robustez entre folds (media y desviación estándar de MAE). 52

Tabla 22 Diagnóstico de residuos de SARIMAX, por fold. 53

Tabla 23 Error absoluto segmentado por nivel de concentración (cuartiles por fold) y sesgo por fold. 53

Tabla 24 Tests de estacionariedad sobre el nivel y la diferencia diaria (serie de inspección interpolada). 59

Tabla 25 Correlación con el target de una feature con fuga versus su versión segura. 60

Tabla 26 Resumen de métricas por modelo, promediadas sobre los tres folds (media ± desviación estándar). 61

Tabla 27 Diagnóstico de residuos de LightGBM, por fold (Ljung-Box) y agregado (Jarque-Bera, ARCH LM). 62

Tabla 28 Error absoluto segmentado por franja horaria. 62

Tabla 29 Resultado del clustering K-means ($k = 4$) sobre perfiles horarios. 67

Tabla 30 Tests de estacionariedad sobre la serie agregada del panel. 67

Tabla 31 Comparación cold-start: baseline transparente vs. LightGBM global, evaluados sobre los 74 clientes nunca vistos. 69

Tabla 32 Cross-learning interpretado: comparación de régimen de disponibilidad de historia. 69

Tabla 33 Diagnóstico de residuos del modelo global cold-start. 70

Tabla 34 Error por cluster de forma de consumo (aplicado a los 74 clientes de test). 70

Tabla 35 Tests de estacionariedad sobre el nivel y la diferencia semanal. 75

Tabla 36 Comparación de modelos, promediada sobre los tres folds (media ± desviación estándar). 77

Tabla 37 Diagnóstico de residuos de LightGBM, por fold y agregado. 77

Tabla 38 Error absoluto segmentado por nivel de carga (cuartil superior por fold vs. resto). . 77

Tabla 39 Precisión frente a costo: los cuatro modelos comparados. 78

Tabla 40 Tests de estacionariedad sobre un sensor de ejemplo (`station_400137`, primeros 120 días). 84

Tabla 41 Comparación final, ordenada por MAE, sobre las 40,176 ventanas de test de la cohorte 48/24. 86

Tabla 42 Métricas macro por sensor (media ± desviación estándar sobre los 24 sensores de test). 87

Tabla 43 Diagnóstico de residuos de `TFT-like` sobre un sensor de test ordenado temporalmente. 87

Tabla 44 Recomendación ejecutiva de arquitectura, cohorte 48/24. 88

Tabla 45 Tests de estacionariedad sobre la serie diagnóstica `M1058`. 93

Tabla 46 Comparación de los tres enfoques probabilísticos. 95

Tabla 47 Diagnóstico de residuos del punto central (P50) del baseline `EmpiricalNaive`. 96

Tabla 48 Reglas de monitoreo y reentrenamiento propuestas. 96

Tabla 49 Del concepto de gobierno a la herramienta estándar: stack MLOps de referencia. . . 97

Tabla 50 Contenido del Model Card del modelo documentado (`EmpiricalNaive`). 98

Tabla 51 Síntesis por palanca, no por métrica. Cada fila condensa el hallazgo, su número-tótem con su asterisco visible (la condición que lo califica) y la palanca que lo explicó. La métrica cambia de escala entre capítulos: MASE puntual (2, 3, 5, 7), MAE absoluto (4, 8), MASE macro mediano (6), CRPS aproximado (9). Compárense las palancas, no los números. 101

Tabla 52 Jerarquía de palancas construida sobre los resultados de los ocho capítulos. Cada nivel corresponde a una decisión metodológica cuya magnitud verificada supera la de la arquitectura elegida. Reemplaza los "Niveles 0–5" del sílabo (organizados por complejidad de técnica) por una ordenación por impacto medido. 104

Tabla 53 Especificado vs. ejecutado. La tabla más creíble que el documento puede ofrecer: las limitaciones no se descubren aquí, se administran desde el diseño. 107

Tabla 54 Traducción a costo total de propiedad. El costo de cómputo por corrida es despreciable frente al término que decide: el error evitado y el mantenimiento. Tarifa ilustrativa de USD 0.20 por vCPU-hora. 109

Tabla 55 Los ocho datasets utilizados: fuente, dimensiones, frecuencia, target, nulos y peculiaridades documentadas. 111

Tabla 56 Todos los modelos evaluados en la monografía, con hiperparámetros clave, librería y tiempo de ejecución cuando fue medido en el notebook de origen. 112

Tabla 57 Información de entorno recuperable directamente de los ocho notebooks. 113

Tabla 58 Las ocho Methodology Cards de la monografía, normalizadas a un formato comparable de cuatro campos. 117

Índice de figuras

Figura 1 El pronóstico como distribución predictiva. A la izquierda del corte t, el historial observado; a la derecha, la distribución de valores posibles en el horizonte $t + h$, representada por bandas cuantílicas anidadas (P25–P75 dentro de P10–P90) y por la densidad predictiva al borde. La esperanza condicional —y su caso particular práctico, la media— es un resumen puntual de esa distribución, no un sustituto de ella. . 13

Figura 2 Mapa de decisión ejecutivo. Tres preguntas ordenan la elección de modelo: historia del target, disponibilidad ex-ante de covariables y naturaleza de la decisión (puntual o de riesgo). La primera pregunta es siempre por la información, nunca por la arquitectura. 14

Figura 3 Diagrama del split temporal 80/20. 31

Figura 4 Diagrama del split 70/15/15 y tamaños resultantes. 39

Figura 5 Estructura de los tres folds rolling-origin. 50

Figura 6 Estructura de los tres folds walk-forward con embargo. 60

Figura 7 Partición por sensor (transferencia) frente a partición temporal. 84

CAPÍTULO 1

Marco teórico-metodológico y fundamentos

1.1 Contexto del forecasting

El **forecasting de series temporales** consiste en producir una estimación de valores futuros de una variable a partir de su historia observada y, cuando existen, de covariables asociadas. Desde el punto de vista de negocio, es la disciplina que convierte datos históricos en decisiones anticipadas: cuánta capacidad contratar, cuándo reforzar la operación, qué nivel de inventario de seguridad mantener y qué rango de resultados es razonable esperar. A diferencia de un problema de regresión transversal, el orden temporal de las observaciones es información constitutiva del problema: reordenar los datos destruye la pregunta que se intenta responder.

> **La misma pregunta con distinta información disponible.** El forecasting toma cuatro configuraciones que comparten objetivo —anticipar el futuro— y se diferencian por la información que tienen a su disposición. Esta monografía las recorre en orden de complejidad creciente de datos; la distinción emerge de los ocho experimentos, que caen en cuatro regímenes distintos según la información que cada uno tiene disponible.

Todo problema de pronóstico de negocio se define por cinco elementos que condicionan qué técnica es viable: el **horizonte** (cuánto hacia adelante se pronostica), la **granularidad** (hora, día, mes), el **lead time** o antelación con la que se emite la decisión, la **cantidad de series** que comparten estructura y la **disponibilidad futura de las covariables**. Dos problemas con los mismos datos pueden exigir modelos distintos si cambia el horizonte o si la covariable relevante no se conoce de antemano. La elección de la métrica de error es el sexto elemento: el MAE mide el error promedio típico, el RMSE penaliza los errores grandes y el MASE permite comparar series de escalas distintas porque normaliza contra un baseline simple; ninguna es universal, y la monografía declara la métrica de cada experimento en función de la decisión que habilita. En la práctica industrial, omitir estos elementos es la fuente más común de errores metodológicos: evaluar un modelo con información que no estaría disponible en producción (fuga de información) o comparar modelos con protocolos de validación distintos. Por eso esta monografía los declara explícitamente en cada capítulo, en el contrato de datos de la fase 2 de la metodología (Sección 1.10).

Esta monografía distingue cuatro configuraciones, cada una asociada a un tramo distinto del repositorio:

- **Univariado de una sola serie.** Una única secuencia de valores, sin covariables ni relación con otras series (semanas 1 y 2, con extensión a exógenas en la semana 3). Habilita decisiones como dimensionar recursos operativos de tráfico o evaluar si un modelo simple basta.
- **Multi-serie o de panel.** Muchas series relacionadas que comparten estructura y pueden entrenarse conjuntamente (semana 5, con 370 series de consumo eléctrico; semana 8, con 1,428 series del benchmark M3). Habilita decisiones sobre redes de clientes y benchmarks de competencia.
- **Multivariado con covariables.** Una serie objetivo acompañada de variables explicativas cuya disponibilidad futura debe evaluarse explícitamente, no darse por supuesta (semanas 3, 4 y 6). Habilita decisiones que dependen de contexto externo —clima, calendario, generación renovable—.
- **Probabilístico.** En lugar de un único valor puntual, la salida es una distribución completa o un conjunto de cuantiles que cuantifican la incertidumbre del pronóstico (semana 8). Habilita decisiones de riesgo con intervalos calibrados.

En la industria, estas cuatro configuraciones sostienen decisiones operativas distintas: dimensionamiento de infraestructura de tráfico y energía (semanas 1, 6 y 7), gestión de inventario y flotas (semana 4), planificación de capacidad para redes de clientes (semana 5) y gestión de riesgo bajo incertidumbre (semana 8). El repositorio que sustenta esta monografía fue diseñado para recorrer las cuatro configuraciones con datos reales, no sintéticos, preservando las imperfecciones —nulos, duplicados, gaps y covariables no disponibles en el futuro— que un curso académico convencional suele ocultar.

1.2 Fundamento estadístico del pronóstico

La Sección 1.1 definió el forecasting desde la decisión de negocio; esta sección fija su gramática matemática mínima, para que cada capítulo técnico posterior use las mismas palabras con el mismo significado. Sea y_{t+h} el valor futuro de la variable objetivo en el horizonte h y sea $\mathcal{F}_t$ el conjunto de información disponible en el instante de corte t —la historia observada del target y, cuando corresponda, las covariables legítimamente disponibles ex-ante—. Todo lo que esta monografía llama «pronóstico» es un resumen de la distribución condicional de y_{t+h} dado $\mathcal{F}_t$; lo que cambia entre capítulos es cuál resumen se entrega y para qué decisión.

El pronóstico puntual como esperanza condicional. La definición formal del pronóstico puntual es la esperanza condicional de la variable objetivo dada toda la historia observada hasta el corte:

$$\hat{y}_{t+h|t} = \mathbb{E}[y_{t+h} \mid \mathcal{F}_t]$$

En términos simples: un pronóstico a una semana responde a la pregunta «¿cuál es el valor esperado de la variable dentro de una semana, condicionado a todo lo que sé hoy?». No es una afirmación sobre el futuro sino un promedio ponderado por probabilidad de todos los futuros compatibles con lo observado; por eso dos analistas con la misma historia y el mismo modelo correcto pueden emitir el mismo punto y aun así fallar ambos en la realización concreta. La métrica que evalúa ese punto en unidades del problema es el MAE, y su versión escalada —el MASE, que normaliza contra un baseline estacional y permite comparar series de distinta

escala— sigue la definición de Hyndman y Koehler (2006); es la métrica técnica declarada en los Capítulos 2, 3, 5, 6 y 7 de esta monografía.

Del punto a la familia: los cuantiles condicionales. La esperanza condicional es un caso particular de una familia más general, los cuantiles condicionales de la distribución predictiva:

$$q_\tau(t+h) = \inf\{y : P(y_{t+h} \le y \mid \mathcal{F}_t) \ge \tau\}, \quad \tau \in (0,1)$$

Dentro de esa familia, la mediana condicional ($\tau = 0.5$) es una alternativa igualmente válida como pronóstico puntual: minimiza el error absoluto esperado, mientras que la esperanza minimiza el error cuadrático esperado (Hyndman & Athanasopoulos, 2021). La elección entre media y mediana no es estética: con colas pesadas —la regla empírica de este documento, como mostrará el diagnóstico de residuos de los ocho experimentos— la mediana es el resumen robusto. La misma lógica se extiende de un cuantil a un par de cuantiles simétricos: $q_{0.25}$ y $q_{0.75}$ delimitan un rango central del 50 % de los escenarios compatibles con la historia; el pronóstico deja de ser un número y pasa a ser un intervalo con una probabilidad declarada.

Colas de la distribución: riesgo y punto central son la misma familia. Nada impide llevar esa lógica a los cuantiles extremos. El cuantil alto $q_{1-\alpha}$ —por ejemplo, $\tau = 0.90$ o 0.99— es exactamente el valor en riesgo (VaR) de la distribución predictiva: el nivel que solo una fracción α de escenarios supera. Y la pérdida esperada en la cola (expected shortfall, ES), el promedio de los escenarios que exceden ese umbral,

$$\mathrm{ES}_\alpha = \mathbb{E}[y_{t+h} \mid y_{t+h} > q_{1-\alpha}, \mathcal{F}_t],$$

es también un funcional de la misma distribución condicional. La consecuencia conceptual es directa: la gestión de riesgo (stock de seguridad, presupuesto de contingencia, capacidad extrema) y el pronóstico puntual central no son disciplinas distintas; son la misma familia matemática de cuantiles condicionales evaluada en distintos puntos de la distribución. Que esa familia se evalúe con reglas de puntuación propias —funciones de pérdida cuya mejor estrategia es reportar la distribución verdadera, como el pinball loss y el CRPS— es el marco de evaluación probabilística de Gneiting y Katzfuss (2014), que el Capítulo 9 convierte en práctica de producción.

Cobertura empírica: la métrica que audita la promesa. Declarado un intervalo $\left[q_{\frac{\alpha}{2}}, q_{1-\frac{\alpha}{2}}\right]$ con cobertura nominal $1-\alpha$, la pregunta operativa es si la promesa se cumple. La cobertura empírica es la proporción de casos en los que el valor efectivamente observado cae dentro del intervalo pronosticado:

$$\mathrm{Cob}_{1-\alpha} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\left[q^i_{\frac{\alpha}{2}} \le y^i \le q^i_{1-\frac{\alpha}{2}}\right]$$

donde $\mathbb{1}[\cdot]$ es la función indicadora y n el número de observaciones evaluadas. La cobertura nominal es lo que el intervalo promete; la empírica, lo que cumple: un intervalo del 80 % que solo cubre el 70 % de las realizaciones subdimensiona sistemáticamente el riesgo de quien decide con él. Esta métrica se aplica más adelante en el Capítulo 4, sobre los intervalos gaussianos del modelo SARIMAX, y en el Capítulo 9, sobre los intervalos cuantílicos P10–P90 del motor probabilístico; sus resultados numéricos no se anticipan aquí.

El pronóstico es una distribución; el punto es un resumen. Esperanza condicional, mediana, pares de cuantiles, VaR y expected shortfall no son modelos rivales: son funcionales distintos de una única distribución predictiva $p(y_{t+h} \mid \mathcal{F}_t)$. Elegir cuál entregar es una decisión de negocio (qué pérdida se quiere minimizar, qué cola importa), no una decisión de algoritmo. La Figura 1 condensa esta idea: el abanico es el pronóstico; la línea central es solo su resumen más usado.

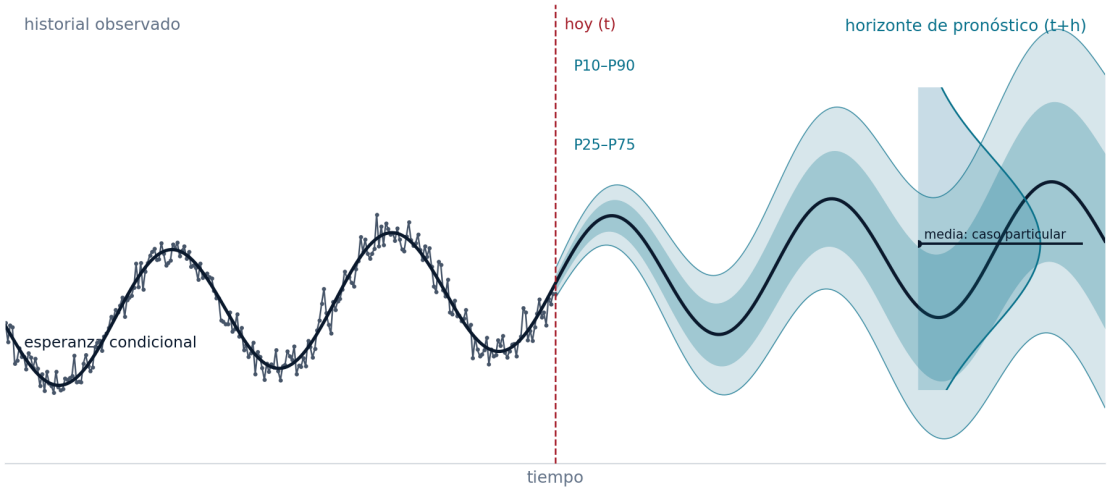


Figura 1. El pronóstico como distribución predictiva. A la izquierda del corte t, el historial observado; a la derecha, la distribución de valores posibles en el horizonte $t + h$, representada por bandas cuantílicas anidadas (P25–P75 dentro de P10–P90) y por la densidad predictiva al borde. La esperanza condicional —y su caso particular práctico, la media— es un resumen puntual de esa distribución, no un sustituto de ella.

Descripción intuitiva de la figura. La imagen se lee como un embudo que nace en «hoy». Hasta el corte, la serie es un hecho: puntos observados, sin incertidumbre. Desde el corte hacia adelante, el modelo no produce una línea sino un abanico: cada banda sombreada es un par de cuantiles condicionales que encierra una fracción declarada de los futuros plausibles, y la curva de densidad al borde es la distribución predictiva completa en el horizonte. La línea navy central —la esperanza condicional— atraviesa el embudo por su centro: es el resumen puntual más usado, y la media marcada sobre la densidad recuerda que eso punto es un caso particular de la distribución, elegido porque minimiza el error cuadrático, no porque la distribución no exista. Cuando el Capítulo 9 entregue P10/P25/P50/P75/P90 en lugar de un solo número, no estará cambiando de técnica: estará entregando la figura completa en vez de su línea central.

Mapa de decisión ejecutivo. La gramática anterior describe **qué** entrega un pronóstico; la Figura 2 traduce esa gramática —junto con la jerarquía de palancas de la Tabla 52— a un árbol de decisión operativo. Es el resumen más comprimido del documento: la primera bifurcación pregunta por la **información disponible** (historia del target y covariables ex-ante) y la segunda, por la **naturaleza de la decisión** (puntual o de riesgo). Ninguna arquitectura aparece antes de agotar las preguntas sobre datos.

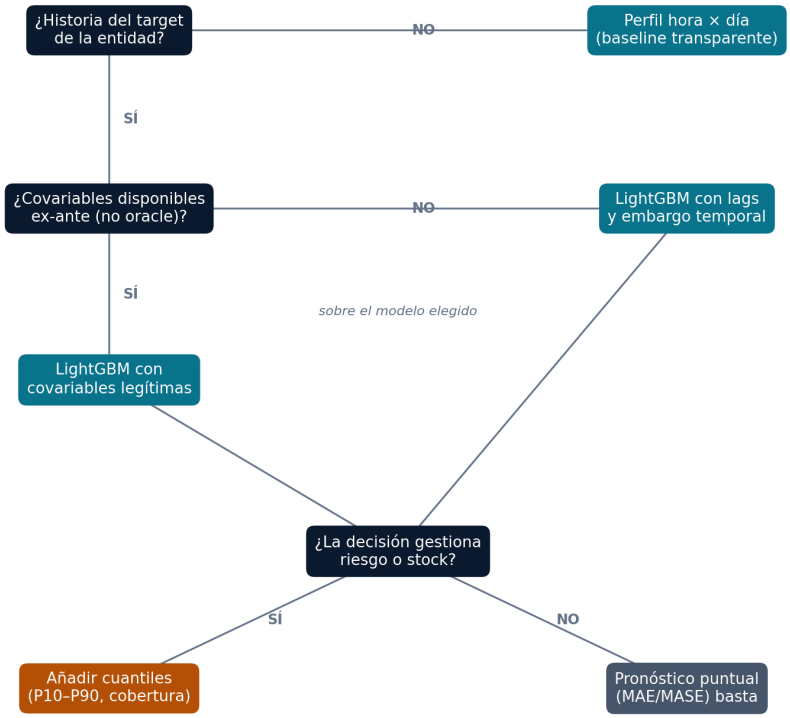


Figura 2. Mapa de decisión ejecutivo. Tres preguntas ordenan la elección de modelo: historia del target, disponibilidad ex-ante de covariables y naturaleza de la decisión (puntual o de riesgo). La primera pregunta es siempre por la información, nunca por la arquitectura.

1.3 Principios de diseño del repositorio

El repositorio `course-timeseries-benchmarks-pro` declara seis principios que gobiernan cada decisión técnica documentada en los capítulos siguientes:

1. **Reproducibilidad total.** Cada artefacto tiene un hash SHA-256 registrado; los splits son explícitos y deterministas con semilla 42; el entorno de dependencias está acotado por rangos compatibles.
2. **Autocontenido.** Los CSV limpios se incluyen en el repositorio; ninguna ejecución depende de acceso a internet ni de scripts de preparación externos.
3. **Transparencia.** Cada transformación aplicada a los datos queda registrada en un receipt JSON que documenta URL fuente, hashes de origen y destino, filas, columnas, nulos y pasos de limpieza. Nada se imputa, deduplica o agrega silenciosamente.
4. **Realismo.** Los datasets conservan imperfecciones reales —timestamps duplicados (semana 1), targets ausentes (semana 3), covariables con distinta granularidad (semana 6)— que los notebooks enseñan a detectar y gestionar explícitamente, no a esconder.
5. **Progresión pedagógica.** La complejidad crece de forma acumulativa en tres ejes simultáneos: datos (univariado → multi-serie → multivariado), modelos (naive → estadístico → ML → DL) y evaluación (pronóstico puntual → probabilístico → gobierno MLOps).

6. **Gobierno explícito.** Un catálogo machine-readable (`registry.yaml`) y una suite de tests validan contratos de forma, integridad y documentación en cada ejecución de integración continua.

Estos principios no son declaraciones retóricas: se verifican de forma recurrente en el análisis de cada capítulo técnico. Por ejemplo, el principio de realismo se materializa concretamente en los 2,043 valores ausentes del target de PM2.5 (Capítulo 4) o en los 17 registros exactamente duplicados del dataset de tráfico (Capítulo 2), y el principio de reproducibilidad se verifica en que los ocho notebooks fijan `SEED = 42` antes de cualquier operación estocástica.

1.4 Arquitectura de cuatro capas

El flujo de datos del repositorio separa procedencia, transformación, análisis y gobierno en cuatro capas:

Capa	Contenido	Regla de gobierno
1. RAW (temporal)	Fuentes públicas (UCI, OPSD, Zenodo/M3)	Excluida del release; streaming con límite 500 MiB y verificación SHA-256 en tránsito
2. CURATED (versionada)	`data/<semana>/*.csv` + `README.md` de esquema + `*.receipt.json`	Incluida en el release; cada CSV acompaña su receipt con origen, hashes, filas, columnas y pasos de limpieza
3. ANALYSIS	`notebooks/01_*.ipynb` … `notebooks/08_*.ipynb`	Ejecutados de principio a fin; semilla `SEED = 42` fijada antes de cualquier operación estocástica
4. GOVERNANCE	`registry.yaml` + `MANIFEST.sha256` + `tests/` + `validate_repository.py`	Catálogo machine-readable + tests de integración continua de forma, integridad y documentación

Tabla 1. Arquitectura de cuatro capas del repositorio.

Dos decisiones de escala en esta arquitectura son relevantes para interpretar los capítulos técnicos. Primero, el dataset de consumo eléctrico (semana 5) se distribuye en formato *long* (`ds`, `unique_id`, `y`) —9,732,480 filas pero solo tres columnas— porque ese formato es el que exige el entrenamiento de un modelo global multi-serie. Segundo, el dataset de sensores de tráfico PeMS-SF (semana 7) se mantiene en formato *ancho* (963 columnas de sensores) precisamente para evitar generar cerca de 61 millones de filas en formato long; esta decisión de ingeniería tiene consecuencias directas sobre cómo se particiona el experimento de transferencia entre sensores en el Capítulo 8.

1.5 Sistema de reproducibilidad

La reproducibilidad se sostiene en cinco mecanismos complementarios, verificables independientemente de cualquier afirmación narrativa de esta monografía:

Mecanismo	Qué garantiza
`MANIFEST.sha256`	Hash SHA-256 de cada archivo del release; verificable con `sha256sum -c MANIFEST.sha256`
Receipt JSON por CSV	Origen (URL, bytes, hash de la fuente), producto (path, bytes, hash del CSV) y transformación (timestamp UTC, filas, columnas, nulos, pasos de limpieza)
`registry.yaml`	Catálogo machine-readable: id, semana, archivos, dimensiones, frecuencia, target y covariables por dataset; usado como contrato por los tests automáticos
Entorno acotado	`requirements.txt` con rangos compatibles (p. ej. `pandas>=2.1,<3`); un lockfile (`pip freeze`) permite byte-reproducibility exacta cuando se requiere
Semilla y splits deterministas	`SEED = 42` en los ocho notebooks; ningún split de evaluación es aleatorio — todos son temporales, por serie o por rolling-origin, según el capítulo

Tabla 2. Mecanismos de reproducibilidad del repositorio.

El Apéndice C reproduce extractos verificados de estos cinco mecanismos —`requirements.txt`, un receipt JSON real, `registry.yaml` y el encabezado de `MANIFEST.sha256`— junto con las instrucciones mínimas para validar el release y reejecutar los notebooks.

1.6 Taxonomía de los ocho datasets

Los ocho datasets estructuran el curso porque cada uno materializa una decisión operativa distinta que un profesional de industria debe tomar con datos reales: cuándo reforzar la operación, qué capacidad contratar, cómo alertar sobre calidad del aire, cómo rebalancear una flota, cómo atender a clientes sin historia, cómo despachar carga con renovables, cómo estimar sensores nuevos y qué intervalo usar para planear con incertidumbre. La Tabla 3 condensa la dimensión, frecuencia, target, covariables y desafío principal de cada uno; la columna **Desafío principal** adelanta el tipo de decisión que el capítulo correspondiente habrá de tomar.

Sem.	Dataset	Dimensión	Frecuencia	Target	Covariables destacadas	Desafío principal
1	Metro Interstate Traffic	48,204×9 (40,575 h únicas)	Horaria	Volumen de tráfico (y)	Feriados, temperatura, lluvia, nieve, nubosidad	7,629 timestamps duplicados; 11,976 horas faltantes sobre la rejilla nominal
2	Appliances Energy	19,735×29 (10 min) → 3,290 h agregadas	10 min / horaria	Energía de electrodomésticos	27 sensores interiores/exteriores de temperatura y humedad	Selección de orden por AIC/BIC que no garantiza el mejor desempeño en test
3	Beijing PM2.5	43,800×14	Horaria	Concentración PM2.5	Punto de rocío, temperatura, presión, viento, lluvia/nieve	2,043 valores de target ausentes (gaps interiores); exógenas meteorológicas solo disponibles como escenario oracle
4	Bike Sharing (UCI)	17,379×17 (h) / 731×16 (día)	Horaria / diaria	Alquileres (`cnt`)	Calendario, clima, temperatura normalizada	`cnt = casual + registered`: fuga directa si esos componentes se usan como features
5	Electricity Load Diagrams	9,732,480×3 (370 series)	Horaria	Consumo por cliente	Identidad de serie únicamente	Escala (9.7M filas) y cold-start estricto sin metadatos del cliente nuevo
6	OPSD Germany	50,401×43	Horaria	Carga eléctrica nacional	Forecasts de carga, generación solar/eólica, perfiles regionales	Selección de covariables entre 43 columnas con cobertura desigual
7	PeMS-SF (Traffic)	63,360×967 (963 sensores)	10 min intradía	Vector de ocupación vial	`weekday_label`, `day_id`, `slot_10min`	Sin fechas calendario reales; transferencia a sensores nunca vistos en entrenamiento
8	M3 Monthly	167,562×3 (1,428 series)	Mensual	Serie del benchmark M3 (y)	Identidad de serie únicamente	Heterogeneidad de dominio y de longitud (66 a 144 obs./serie), sin covariables

Tabla 3. Resumen de los ocho datasets utilizados, en el orden en que se introducen a lo largo del curso.

Tres conceptos teóricos explican por qué estos datasets, y no otros, estructuran el curso. Primero, **la imperfección es información**: los nulos, duplicados y gaps no son ruido a eliminar, sino evidencia del proceso de medición real, y la forma en que se gestionan determina la validez del experimento —por eso el contrato de datos los cuantifica antes de modelar—. Segundo, **la disponibilidad futura de una covariable es una propiedad del problema, no del modelo**: una exógena solo puede usarse en producción si se conoce de antemano o si existe un pronóstico confiable de ella; la distinción entre covariable real y covariable oracle (no disponible en el futuro) recorre toda la monografía. Tercero, **la escala cambia la familia de modelos viable**: 9.7 millones de filas (semana 5) o 1,428 series heterogéneas (semana 8) implican un cambio cualitativo en el tipo de modelo que tiene sentido entrenar, no solo un problema de cómputo. La traducción operativa de cada dataset a su decisión de negocio queda condensada en la columna **Desafío principal** de la Tabla 3 y se desarrolla en la apertura del capítulo correspondiente.

1.7 Mapa de ruta de modelos

La progresión de modelado del curso no es una lista arbitraria de algoritmos: cada familia se introduce después de que la anterior demuestre sus límites concretos sobre datos reales. La Tabla 4 recorre las siete familias, su condición de justificación y el veredicto empírico verificado en el curso —cuando lo hubo. La ruta **no es una escalera de mejora monótona**: en al menos cuatro de los ocho experimentos (Capítulos 2, 3, 7 y 8) el modelo formalmente más sofisticado iguala, o supera solo marginalmente, a una alternativa mucho más simple evaluada bajo el mismo protocolo. LightGBM protagonizará un arco de cuatro capítulos que el Cap. 10 sintetiza; los ocho números-tótem (0.916, 0.681, 18.36 %, 0.457, 0.01 %, 0.234, 4 %, −43.2 %) recorren el documento y se condensan en la Tabla 51.

Familia	Condición de justificación	Veredicto en el curso	Cap.
Baselines (naive, seasonal, MA)	Piso de referencia obligatorio; se calcula siempre primero	Ganó directamente en el Cap. 2 (MASE 0.916); compitió dignamente en 7 y 8	2, 7, 8
Estadísticos (ARIMA, ETS, Theta)	Historia suficiente, frecuencia regular, estructura lineal	ETS superó a ARIMA por AIC en el Cap. 3 (0.681 vs. 0.901)	3
Estacionales con exógenas (SARIMAX)	Disponibilidad futura real de las covariables	Mejora 18.36 % en Cap. 4, pero bajo escenario oracle explícito	4
ML tabular (LightGBM)	Features ricas con embargo temporal y checklist anti-leakage	Ganó en Cap. 5 (MASE 0.457) y Cap. 7 (0.234); empate en Cap. 6	5, 6, 7
Globales / cross-learning	Cientos de series con estructura compartida	Empate técnico en Cap. 6: la disponibilidad de historia domina	6
Deep learning secuencial (LSTM/GRU)	Patrones no lineales de largo alcance + justificación de costo	No ganó ninguna comparación directa (Caps. 6 y 7); veredicto condicionado	6, 7
Arquitecturas modernas (N-BEATS, TFT-like)	Transferencia a entidades nuevas como requisito de negocio	TFT-like gana por 4 % en Cap. 8; N-BEATS es el ganador práctico al cuarto del costo	8
Forecasting probabilístico	Residuos no gaussianos o necesidad de intervalo calibrado	Único doble win del documento: calibración Y sharpness en Cap. 9 (–43.2 % CRPS)	9

Tabla 4. Ruta de complejidad creciente de modelos, con veredicto empírico del curso.

La lógica teórica de cada familia se complementa con la cita correspondiente: Box & Jenkins (2015) para la familia estadística; Shumway & Stoffer (2017) para el análisis de series temporales en general; Nielsen (2019) para las métricas de error; Hyndman y Koehler (2006) para la métrica MASE; Hyndman y Athanasopoulos (2021) para los fundamentos; Lazzeri (2020) para la selección de modelos de ML; Chen & Guestrin (2016) para LightGBM; Chollet (2021) para la familia recurrente; Oreshkin et al. (2020) para N-BEATS; Lim et al. (2021) para TFT; Gneiting & Katzfuss (2014) para el forecasting probabilístico; Treveil et al. (2020) para MLOps. La bibliografía consolidada al final del documento reúne las referencias APA completas.

1.8 Preguntas de investigación

Esta monografía organiza su evidencia alrededor de tres preguntas transversales. Cada una nació de un momento concreto del curso, no de una elección estética:

1. **Pregunta 1 — ¿Cuándo un modelo complejo aporta valor real sobre un baseline estacional simple, y bajo qué condiciones de los datos ese valor se diluye o desaparece?** Nació cuando el Capítulo 2 mostró que un SeasonalNaive168 con MASE 0.916 era prácticamente insuperable para los modelos estadísticos — y se reforzó al ver que la LSTM de Cap. 7 perdía también contra el baseline estacional. La respuesta se articula en Cap. 10 §10.2 con la condicionalidad de las cuatro palancas verificadas (features, historia, covariables, calibración).
2. **Pregunta 2 — ¿Qué papel cumplen las variables exógenas y los modelos globales cuando la disponibilidad de contexto es limitada, oracle o inexistente?** Tomó forma cuando la diferencia entre "clima observado" y "clima disponible ex-ante" estructuró todo el Capítulo 4 (la mejora 18.36 % era un techo, no un rendimiento) y se afiló con el empate 0.01 % del Capítulo 6 entre LightGBM global y perfil hora×día. La respuesta se articula en Cap. 10 §10.2 con el arco oracle (3⟶4⟶7) y la asimetría de palancas.
3. **Pregunta 3 — ¿Cómo se traduce la evidencia estadística de no normalidad y heterocedasticidad de los residuos, presente de forma casi universal en los ocho experimentos, en decisiones operativas concretas de construcción de intervalos y monitoreo en producción?** Surgió cuando el diagnóstico del Capítulo 2 ya rechazó normalidad y se acumuló capítulo tras capítulo hasta los 8/8 Jarque-Bera del Cap. 9. La respuesta se articula en Cap. 10 §10.2 con la cuenta regresiva cerrada y el doble win de calibración Y sharpness del Cap. 9.

1.9 Trazabilidad con el sílabo

La Tabla 5 muestra dónde se desarrolla y se evidencia cada una de las cinco competencias profesionales declaradas por el sílabo. La Tabla 51 (Cap. 10, §10.1) remite a esta tabla; el documento evita duplicarla.

Competencia del sílabo	Cómo se desarrolla	Dónde se evidencia
Traducir negocio a forecasting	Definición del problema y métricas de error alineadas al impacto empresarial	Capítulos 1 y 2; secciones de conexión con producción de cada capítulo
Diseñar validación temporal robusta	Splits temporales, rolling-origin, walk-forward con embargo y splits por serie, sin fuga de información	Capítulos 2, 4, 5 y 8
Comparar modelos estadísticos, ML y DL	Comparaciones contra baselines bajo el mismo protocolo, con costo computacional y mantenibilidad	Capítulos 2–8 y Capítulo 10
Generar pronósticos probabilísticos	Cuantiles, pinball loss, CRPS aproximado y cobertura empírica	Capítulo 9
Desplegar y monitorear en producción	Detección de drift, umbrales de reentrenamiento y model card	Capítulos 9 y 10

Tabla 5. Trazabilidad entre las competencias del sílabo y los capítulos de la monografía.

1.10 Metodología del estudio de caso

La metodología de diez fases no fue un plan previo; se reconstruyó a posteriori, al notar que cada notebook recorría prácticamente el mismo circuito. La ventaja de formalizarla así es que describe lo que efectivamente se hizo, no lo que debería hacerse — y los matices propios de cada dataset aparecen como desviaciones verificables del patrón común. Cada fase responde una pregunta operativa concreta, y cada capítulo técnico las aplica con la dependencia de datos y métrica que el problema exige.

1. **Definición del problema de negocio.** ¿Qué decisión habilita el pronóstico y qué métrica de error la alinea? Sin una métrica ligada a la decisión, cualquier comparación posterior es decorativa. *Se aplica en: Cap. 1 y apertura de cada capítulo técnico.*
2. **Contrato de datos.** ¿Qué contiene cada dataset y qué imperfecciones —nulos, duplicados, gaps— hay que gestionar explícitamente antes de modelar? *Se aplica en: Caps. 2–8 y Apéndice A.*
3. **Análisis exploratorio temporal.** ¿Qué componentes, estacionalidades y regímenes domina la serie? Aquí se detectan la estacionariedad y los patrones que los baselines aprovecharán. *Se aplica en: Cap. 2 y sección inicial de cada capítulo técnico.*

4. **Construcción de baselines.** ¿Qué logra un modelo simple antes de invertir en complejidad? El baseline estacional es el piso de referencia obligatorio de todo el curso. *Se aplica en: Cap. 2.*
5. **Selección de validación temporal.** ¿Cómo particionar sin filtrar información futura? La elección entre split temporal, rolling-origin, walk-forward o split por serie depende del caso y se documenta en cada capítulo. *Se aplica en: Caps. 2, 4, 5 y 8.*
6. **Modelado por niveles de complejidad.** ¿Cuánto gana cada familia —estadístico, ML tabular, global, deep learning— frente al baseline bajo el mismo protocolo? *Se aplica en: Caps. 3–8.*
7. **Diagnóstico de residuos.** ¿Qué supuestos se violan y qué implica para producción? La no normalidad y la heterocedasticidad son la regla, no la excepción. *Se aplica en: Caps. 2–8.*
8. **Pronóstico probabilístico.** ¿Cómo cuantificar la incertidumbre y verificar cobertura empírica en lugar de asumirla? *Se aplica en: Cap. 9.*
9. **Gobierno y monitoreo.** ¿Cómo detectar drift, definir umbrales y decidir cuándo reentrenar? Sin propietario y acciones concretas, el monitoreo no es gobierno. *Se aplica en: Cap. 9.*
10. **Traducción ejecutiva.** ¿Qué recomendación de negocio se desprende de la evidencia y cómo se comunica a stakeholders? *Se aplica en: Cap. 10.*

1.11 Estudio de caso integrador

Para conectar la teoría con la práctica de principio a fin, la monografía propone un caso integrador que reutiliza los experimentos ya documentados en los capítulos técnicos —sin introducir resultados nuevos—: la **cartera de energía para planificación de capacidad, gestión de clientes nuevos y control de riesgo operacional**.

> **Caso integrador: Cartera de energía.**
> **Pregunta de negocio:** ¿cómo dimensionar capacidad de red, atender a clientes nuevos sin historial y controlar el riesgo operacional con pronósticos que reportan incertidumbre?
> **Hilo conductor:** Electricity Load Diagrams (Capítulo 6) como caso central multi-serie; OPSD Germany (Capítulo 7) como soporte de covariables energéticas; M3 Monthly (Capítulo 9) como benchmark probabilístico.
> **Capacidades integradas:** cold-start, modelos globales, LightGBM, forecasting probabilístico y MLOps.

La justificación del caso se apoya en tres experimentos ya reportados:

1. **Gestión de clientes nuevos (Electricity Load Diagrams, Capítulo 6).** El experimento de cold-start sobre 370 clientes muestra que sin historia ningún modelo recupera la escala individual —MASE mediano 1.34 en warm-start frente a 38.59 en cold-start— y que el modelo global LightGBM empata con un baseline transparente de perfil hora×día. Para la cartera, esto define la política de atención de clientes nuevos: perfiles agregados cuando no hay historia y modelos con rezagos cuando la historia lo permite.
2. **Planificación de capacidad (OPSD Germany, Capítulo 7).** El experimento con carga eléctrica nacional y covariables de generación solar/eólica muestra que la LSTM no supera al baseline estacional (MASE 0.576 frente a 0.495 del Seasonal Naive), mientras que LightGBM

sí gana (0.234). Para la cartera, esto orienta la arquitectura de capacidad: el modelo tabular bien construido es el estándar y el deep learning debe justificar su costo.

3. **Control de riesgo operacional (M3 Monthly, Capítulo 9).** El benchmark probabilístico muestra que LightGBMQuantile reduce el CRPS aproximado en 43.2% frente al baseline empírico y acerca la cobertura empírica al nivel nominal (78.9% frente a 80%). Para la cartera, esto define cómo construir intervalos de decisión y cómo monitorearlos en producción.

La palanca de historia (29×) supera por un orden de magnitud cualquier diferencia verificada entre familias de arquitecturas en este documento (máximo 44 % en Cap. 5). La asimetría se formaliza en el Cap. 10 §10.4: 0.01 %–44 % (arquitectura/features) vs. 2900 % (régimen de datos).

1.12 Cómo leer esta monografía para profesionales de industria

Esta monografía no evalúa modelos por su complejidad matemática, sino por su utilidad para tomar decisiones. Cada capítulo técnico sigue el mismo patrón: el problema de negocio, el contrato de datos, la validación temporal, el modelado por niveles y la traducción ejecutiva del error.

El orden de lectura recomendado es:

1. Leer el Capítulo 1 completo: define el vocabulario, la metodología de diez fases y el caso integrador que conecta los experimentos.
2. Leer los capítulos técnicos (2 a 9) en orden, o saltar a los que correspondan al dominio de interés: energía (6 y 7), tráfico y operaciones (2 y 8), calidad del aire (4) o movilidad (5).
3. Leer el Capítulo 10 (síntesis) para la jerarquía de complejidad y las respuestas a las tres preguntas de investigación.
4. Consultar los apéndices cuando se necesite reproducir o auditar: datasets (A), modelos y tiempos (B), entorno (C) y Methodology Cards (D).

Si el lector dispone de poco tiempo, la secuencia mínima es: Sección 1.10 (metodología), Capítulo 2 (baselines), Capítulo 9 (probabilístico) y Capítulo 10 (síntesis).

1.13 Declaración de posicionamiento y madurez de la evidencia

Nivel de análisis. Este trabajo opera en un nivel organizacional/operacional: documenta sistemas socio-técnicos concretos —tráfico urbano, energía residencial, calidad del aire, movilidad en bicicleta, consumo eléctrico industrial, carga nacional, sensores viales y un benchmark de competencia— y las decisiones operativas que cada uno habilita, no un aporte metodológico nuevo a la teoría del forecasting.

Madurez de la evidencia. Los ocho experimentos están validados sobre datos reales, con protocolos de validación temporal explícitos, diagnóstico de residuos y comparación contra baselines — un nivel de madurez que corresponde a evidencia **verificada en entorno relevante** (los datos y la estructura de cada problema real), no a un piloto de laboratorio ni a un despliegue en producción viva. Los experimentos multi-fold (Capítulos 4, 5 y 7) y multi-dominio (síntesis del Capítulo 10) alcanzan el escalón inmediatamente superior, al replicar el hallazgo bajo distintas condiciones de datos en lugar de una sola corrida. Los apartados de "Deudas abiertas" de cada capítulo y la Tabla 53 documentan explícitamente qué evidencia falta para subir un nivel más.

Audiencia primaria. Profesionales de industria (data scientists, analistas de operaciones, product owners de forecasting) que deben decidir cuándo un baseline basta, cuándo migrar a un modelo estadístico o de machine learning tabular, y cuándo la complejidad de deep learning o forecasting probabilístico se justifica frente a su costo.

Límites de inferencia. Los hallazgos de un capítulo no se extrapolan automáticamente a otro dominio: cada capítulo declara su propio contrato de datos, su propio protocolo de validación y su propia métrica, y la Tabla 51 advierte explícitamente contra comparar números-tótem entre capítulos. Ninguna afirmación de esta monografía es causal; las relaciones descritas —entre features y error, entre historia disponible y desempeño, entre arquitectura y costo— son asociativas, verificadas bajo el protocolo declarado en cada capítulo, y dejan de sostenerse si ese protocolo cambia.

1.14 Ética, integridad y declaración de uso de IA

Origen y licencias de los datos. Los ocho datasets provienen de fuentes públicas o de competencia: UCI Machine Learning Repository (tráfico, energía residencial, PM2.5, bicicletas, consumo eléctrico), Open Power System Data (carga nacional alemana), y el archivo de la M3-Competition en Zenodo (serie mensual de referencia). Las licencias y términos de uso de cada fuente están documentados por dataset en la Tabla 55 y en la bibliografía (Apéndice F). Ninguno de los ocho datasets contiene datos personales sensibles ni información identificable de individuos: son series agregadas de tráfico vehicular, consumo energético, calidad del aire, alquileres de bicicletas y benchmarks de competencia.

Uso de asistencia de IA. La redacción, edición y revisión editorial de esta monografía —incluida la normalización de referencias cruzadas, la numeración automática de tablas y figuras, y la corrección de estilo— se realizó con asistencia de un modelo de lenguaje (Claude, Anthropic), bajo supervisión y revisión humana en cada etapa. Ninguna métrica, tabla de resultados o número-tótem reportado en el cuerpo del documento fue generado o inferido por el asistente de IA: todos provienen de la ejecución real de los ocho notebooks del repositorio `course-timeseries-benchmarks-pro`, verificable de forma independiente contra los receipts JSON, `registry.yaml` y `MANIFEST.sha256` descritos en el Apéndice C.

Conflictos de interés y financiamiento. No se declaran conflictos de interés. Esta monografía no recibió financiamiento externo específico; es un trabajo integrador del curso `course-timeseries-benchmarks-pro`.

Resultados no ejecutados. Las especificaciones que no llegaron a ejecutarse —`DeepAR` en el Capítulo 9, el split completo 770/193 de PeMS-SF en el Capítulo 8, el escenario few-shot intermedio del Capítulo 6— se declaran explícitamente como deudas administradas en la Tabla 53 y en las secciones “Deudas abiertas” de cada capítulo correspondiente. Ninguna de ellas aparece en el cuerpo del documento como resultado, métrica o veredicto.

CAPÍTULO 2

Semana 1: EDA, baselines y estacionalidad

Dataset: Metro Interstate Traffic Volume · **Notebook:** `01_semana_1_eda_baselines_estacionalidad.ipynb` · **Fuente:** UCI Machine Learning Repository, licencia CC BY 4.0.

Ficha de caso — Semana 1
Dominio: tráfico.
Problema de negocio: anticipar el volumen vehicular horario.
Usuario: operaciones (concesionaria vial o municipio).
Decisión que habilita: turnos, carriles, mantenimiento o contingencias.
Horizonte: bloque de test horario (8,115 horas, 11 meses).
Baseline obligatorio: Seasonal Naive 168.
Riesgo principal: random split (fuga de información).
Métrica técnica: MASE.
Métrica de negocio: error promedio en vehículos/hora.

2.1 Fundamentos teóricos aplicados

Antes de entrar al dataset conviene fijar tres piezas de vocabulario: los componentes de una serie temporal, las herramientas de diagnóstico y las métricas de error, presentadas en términos accionables para un profesional de industria.

Componentes de una serie temporal. Toda serie puede leerse como la combinación de cuatro piezas, y la primera pregunta de un análisis no es "qué modelo usar" sino "qué piezas están presentes":

1. **Tendencia.** La dirección de largo plazo: crece, decrece o se mantiene. En tráfico, refleja el crecimiento urbano o el cambio de infraestructura vial; su implicación práctica es que los promedios de hace varios años no son comparables con los de hoy sin ajuste.
2. **Estacionalidad.** El patrón que se repite con periodo fijo y conocido —diario, semanal, anual —. En este capítulo conviven dos: el ciclo de 24 horas y el ciclo de 168 horas (semanal). Es la pieza más valiosa para pronosticar porque es anticipable por calendario.
3. **Ciclicidad.** Oscilaciones de duración variable y no fija, asociadas a condiciones económicas o de contexto. A diferencia de la estacionalidad, no tienen un periodo exacto que permita anticiparlas; por eso los modelos se concentran en las dos piezas anteriores.

4. **Ruido.** La componente irregular que no sigue patrón: es lo que ningún modelo puede anticipar y lo que los diagnósticos de residuos intentan caracterizar. Si el ruido domina, ningún modelo complejo ayudará; si la estacionalidad domina, un baseline estacional será difícil de superar —las dos conclusiones centrales de este capítulo—.

STL (Seasonal and Trend decomposition using Loess) como herramienta diagnóstica. La descomposición STL separa la serie en tendencia, estacionalidad y residuo mediante regresión local, sin asumir una forma funcional global. No es un modelo predictivo: es un lente para decidir qué modelar y con qué periodo. La lectura operativa tiene tres pasos: (1) si el componente estacional es estable y dominante, el baseline estacional será competitivo; (2) si la tendencia cambia de pendiente, los promedios históricos largos serán engañosos; (3) si el residuo contiene estructura —no es ruido—, quedó señal sin modelar que un modelo más complejo podría aprovechar, pero primero hay que demostrarlo con números, no asumirlo. En este capítulo se aplica con periodo semanal (168 horas) para confirmar la doble capa diaria/semanal antes de elegir el baseline, y su residuo alimenta la detección de outliers y candidatos a cambio de régimen.

Estacionariedad. Una serie es **estacionaria** cuando su media, su varianza y su estructura de autocorrelación no cambian con el tiempo. Importa porque los modelos estadísticos de este capítulo y los dos siguientes (ARIMA, SARIMA, SARIMAX) asumen esa estabilidad: si la serie no es estacionaria, los parámetros estimados con datos pasados no son transferibles al futuro, y el modelo extrapola propiedades que ya no existen. La estacionariedad no es un requisito de los baselines —el Seasonal Naive la ignora por diseño, porque su supuesto es exactamente que el patrón se repite—, pero sí lo es de los modelos paramétricos que se introducen a partir del Capítulo 3. La consecuencia práctica: antes de ajustar un ARIMA hay que transformar (diferenciar) y verificar; antes de usar un baseline estacional, no.

ADF (Augmented Dickey-Fuller) y **KPSS (Kwiatkowski-Phillips-Schmidt-Shin)** como tests complementarios. El test ADF plantea como hipótesis nula la presencia de una raíz unitaria (serie no estacionaria); el test KPSS plantea la hipótesis inversa (serie estacionaria). Por eso se leen siempre en conjunto y nunca de forma aislada. Hay cuatro lecturas posibles y cada una tiene una interpretación distinta: (1) ADF rechaza y KPSS no rechaza —serie estacionaria, se puede modelar sin transformar—; (2) ADF no rechaza y KPSS rechaza —raíz unitaria, hay que diferenciar—; (3) ambos rechazan —la serie no es ni integrada ni estacionaria alrededor de una constante, en la práctica tiene estacionalidad fuerte u otra estructura que ninguno de los dos tests diagnostica por separado, que es exactamente el caso del nivel de tráfico—; (4) ambos no rechazan —evidencia débil, conviene más diagnóstico—. La conclusión operativa se obtiene diferenciando por el periodo estacional (168 horas) y re-testando: si los dos tests convergen en la serie diferenciada, esa transformación es la base de trabajo para el análisis de autocorrelación.

Detalle de hipótesis para tests de estacionariedad:

Test	Hipótesis Nula (H_0)	Si NO se rechaza H_0...
ADF	La serie tiene raíz unitaria (no es estacionaria).	La serie es no estacionaria; requiere diferenciación.
KPSS	La serie es estacionaria alrededor de una constante/tendencia.	La serie es estacionaria; se puede modelar tal cual.

Métricas de error y su traducción a negocio. Ninguna métrica es universal: cada una responde una pregunta de negocio distinta, y este capítulo las usa según la decisión que habilita.

1. **MAE** (error absoluto medio): el error promedio en las unidades del problema. Para operaciones de tráfico, vehículos/hora. Un MAE de 365.68 significa que, en promedio, el pronóstico se equivoca en 366 vehículos cada hora; es la métrica que un jefe de operaciones puede explicar sin fórmulas, y la adecuada cuando el costo del error es proporcional a su magnitud (turnos, mantenimiento).
2. **RMSE** (raíz del error cuadrático medio): igual que el MAE, pero otorga mayor peso a los errores grandes. Cuando RMSE es claramente mayor que MAE, hay errores puntuales severos —en tráfico, horas pico mal pronosticadas que exigen revisar los picos, no el promedio—; es la métrica a vigilar cuando un error enorme tiene consecuencias desproporcionadas (congestión crítica, contingencias).
3. **MASE** (error absoluto medio escalado): compara el error del modelo contra el error de un baseline estacional calculado sobre train. MASE menor que 1 indica que el modelo supera a ese baseline de referencia; mayor que 1, que no lo supera. Es la métrica técnica oficial de este capítulo porque es comparable entre datasets de escalas distintas y tiene un punto de corte natural: 1.
4. **WAPE** (error porcentual absoluto ponderado): el error total relativo al volumen total del periodo. Un WAPE de 11.05% significa que el error acumulado representa el 11% del tráfico total; es la métrica que se reporta a dirección cuando se pide un porcentaje, y permite comparar la calidad del pronóstico entre periodos de distinto volumen.

Por qué el random split es inválido en series temporales. Un split aleatorio mezcla observaciones pasadas y futuras entre train y test. El modelo entrena viendo valores que en producción todavía no existirían —el calendario completo aparece dentro del rango de entrenamiento—, aprende a "espiar" el futuro y obtiene un error artificialmente bajo que no se repetirá en el despliegue. En series temporales, la única partición válida respeta el orden del tiempo: entrenar con el pasado y evaluar con el futuro. La Sección 2.4 demuestra este punto con números: el mismo modelo bajo partición aleatoria obtiene un MAE menor que bajo partición temporal, y

ese error menor es exactamente la trampa que hay que saber identificar: una métrica optimista que no estima el desempeño real de producción.

2.2 Configuración y contrato de datos

El archivo `data/01_traffic_metro/traffic.csv` contiene 48,204 filas y 9 columnas, contrato verificado antes de cualquier transformación. La Tabla 6 documenta el esquema completo.

Columna	Tipo	Rol
`ds`	`datetime64`	Índice temporal, frecuencia nominal horaria
`y`	`int64`	Target: volumen de tráfico (vehículos/hora)
`holiday`	`object`	Covariable calendario, conocida a futuro; 48,143 nulos (99.87 %)
`temp`	`float64`	Covariable meteorológica, **observada**, no pronosticada
`rain_1h`	`float64`	Covariable meteorológica, observada
`snow_1h`	`float64`	Covariable meteorológica, observada
`clouds_all`	`int64`	Covariable meteorológica, observada
`weather_main`	`object`	Covariable categórica meteorológica, observada
`weather_description`	`object`	Covariable categórica, mayor granularidad que `weather_main`, observada

Tabla 6. Esquema y rol de cada columna del dataset de tráfico.

Auditoría de calidad. Ninguna columna numérica salvo `holiday` tiene nulos. Se detectaron 17 filas exactamente duplicadas y 7,629 timestamps duplicados: la fuente registra más de una fila por hora cuando cambia la descripción meteorológica dentro de la misma hora (p. ej. "light rain" y "light intensity drizzle" para el mismo `ds`). El rango temporal observado va de 2012-10-02 09:00 a 2018-09-30 23:00, con 40,575 timestamps únicos sobre una rejilla horaria completa que esperaría 52,551 —es decir, 11,976 horas ausentes (22.8 % de la rejilla nominal)—.

Decisión de consolidación. Para construir una serie univariada, las filas con el mismo `ds` se consolidan por hora: las variables numéricas continuas se promedian y las categóricas toman el primer valor observado. El resultado, `hourly_observed`, tiene 40,575 filas y 8 columnas. Una segunda versión, reindexada sobre la rejilla horaria completa e interpolada linealmente en el tiempo, se construye **exclusivamente para inspección visual y descomposición STL**; los modelos y la evaluación de la Sección 2.5 usan siempre los 40,575 puntos observados, nunca los interpolados, para no puntuar valores fabricados.

Estadísticos descriptivos del target (sobre observaciones consolidadas, vehículos/hora): media 3,290.65, desviación estándar 1,984.77, mínimo 0, percentil 1 = 259, mediana 3,427, percentil 99 = 6,713, máximo 7,280.

2.3 Análisis exploratorio temporal

Los conceptos de descomposición, estacionariedad y tests ADF/KPSS fueron presentados en la Sección 2.1; aquí se aplican directamente sobre el dataset de tráfico.

La serie exhibe dos capas de estacionalidad claramente separables en un zoom de cuatro semanas: un ciclo diario de picos laborales y un ciclo semanal con caída marcada los fines de semana. El perfil por hora del día y el perfil por día de semana confirman esta doble estructura, y el mapa de calor hora×día la resume en una sola visualización: el tráfico alcanza sus máximos en horas punta de días laborables y sus mínimos en madrugadas de fin de semana.

Una descomposición STL robusta con periodo semanal (168 horas), aplicada sobre la ventana 2016–2017 de la serie interpolada, separa tendencia, componente semanal y residuo. Los tests de estacionariedad, aplicados sobre la serie interpolada de esa misma ventana, arrojan un resultado que conviene leer con cuidado antes de continuar:

Serie	Test	Estadístico	p-valor	Decisión (α=0.05)
Nivel	ADF (H0: raíz unitaria)	−15.68	1.46e-28	rechazar H0
Nivel	KPSS (H0: estacionaria)	0.996	0.010	rechazar H0
Diferencia semanal ($y_t - y_{t-168}$)	ADF	−15.98	6.89e-29	rechazar H0
Diferencia semanal	KPSS	0.108	0.100	no rechazar H0

Tabla 7. Tests de estacionariedad sobre el nivel y sobre la diferencia semanal.

El nivel rechaza ambas hipótesis nulas simultáneamente —ADF dice que no hay raíz unitaria, pero KPSS dice que tampoco es estacionaria alrededor de una constante—. Esta aparente contradicción no es un error: refleja que la serie tiene estacionalidad fuerte y no ruido puramente estocástico ni una tendencia determinista simple; ninguno de los dos tests, por separado, está diseñado para diagnosticar estacionalidad. Al diferenciar por el periodo semanal, ambos tests coinciden en que la serie resultante es estacionaria, lo cual justifica trabajar con esa diferencia en el análisis de ACF/PACF.

El ACF y PACF de la diferencia semanal muestran picos pronunciados en múltiplos de 24 horas, consistentes con memoria diaria superpuesta a la semanal ya removida. La detección de outliers, usando un z-score robusto (MAD) sobre el residuo STL con umbral $|z| > 4.5$, marca 2,121 candidatos de 17,544 puntos analizados (12.1 %) —un umbral deliberadamente conservador que identifica desviaciones extremas, no una regla de limpieza automática—. Un análisis CUSUM descriptivo sobre el residuo estandarizado localiza el punto de mayor desviación acumulada en 2016-07-31 15:00 (CUSUM = −246.48) como candidato a cambio de régimen, sin que esto implique una prueba formal de ruptura estructural ni una interpretación causal.

Covariables: disponibilidad y correlación. Las correlaciones de Spearman entre `y` y las variables meteorológicas son descriptivas, no causales, porque clima y tráfico comparten calendario. Una correlación cruzada entre tráfico y temperatura desplazada de −48 a +48 horas no revela una ventaja de precedencia clara. Más importante para el diseño de un sistema de producción: `holiday` es una covariable **conocida a futuro** (es calendario), pero `temp`, `rain_1h`, `snow_1h`, `clouds_all`, `weather_main` y `weather_description` son **observadas**, no pronostica-

das —en producción, usarlas como predictoras exige un forecast meteorológico externo, no el valor observado que aparece en este CSV—. Esta distinción se vuelve central en el Capítulo 4 (SARIMAX), donde se demuestra el efecto de tratar una covariable observada como si fuera conocida a futuro (escenario oracle).

Vale una nota de lectura crítica: el cálculo de "festivo vs. no festivo" en el notebook compara `holiday` contra la cadena literal `"None"`, pero los valores ausentes reales son `NaN`, no ese literal; en la práctica, la comparación etiqueta como festivo a la totalidad de las 40,575 observaciones y la segmentación resultante no es informativa. Se documenta aquí para que el análisis de la Semana 4 (donde el calendario sí se usa como feature) no repita el mismo patrón.

2.4 Estrategia de validación

Split requerido: temporal 80/20. El primer 80 % de las observaciones consolidadas entrena; el último 20 % simula el futuro. No se usa random split para estimar desempeño.

```
|------------------- Train: 80% (32,460 obs.) -------------------|--- Test: 20% (8,115 obs.) ---|
2012-10-02 09:00                                  2017-10-26 17:00              2018-09-30 23:00
                                                           ^ frontera
```

Figura 3. Diagrama del split temporal 80/20.

Horizonte y ciclo operacional. El horizonte de evaluación es todo el bloque de test (8,115 horas, aproximadamente 11 meses), lo que cubre varios ciclos semanales y estacionales completos —no es un horizonte de un solo paso—. No se declara un embargo explícito entre train y test en este capítulo porque los modelos evaluados (naive, seasonal naive, medias móviles, ETS) no consumen features tabulares con riesgo de fuga por proximidad de ventana; el embargo se convierte en un requisito de diseño explícito recién en el Capítulo 5 (feature engineering).

Contraste didáctico: random split. Para hacer visible el riesgo de un split aleatorio, se construyó una tabla con variables de calendario (`hour`, `dow`, `month`, `time_ordinal`) y se entrenó el mismo `RandomForestRegressor` (`n_estimators=60`, `max_depth=12`, `min_samples_leaf=5`, `random_state=42`) bajo dos particiones:

Partición	MAE (test)
Aleatoria (`train_test_split`, `shuffle=True`) — inválida	251.66
Temporal (corte 80/20)	287.55

Tabla 8. Contraste entre partición aleatoria (inválida para forecasting) y partición temporal.

El resultado es deliberadamente contraintuitivo: la partición aleatoria produce un MAE **menor** que la temporal, no mayor. Esto no significa que el split aleatorio sea mejor; significa que filtra información del futuro hacia el entrenamiento (el `time_ordinal` de observaciones de test aparece disperso dentro del rango de entrenamiento), lo cual hace que el problema evaluado sea artificialmente más fácil y que la métrica resultante no estime el desempeño real de despliegue. La visualización de partición por color contra la línea de tiempo hace evidente que las observaciones de "train" bajo el split aleatorio cubren todo el calendario, incluyendo fechas posteriores a la frontera temporal real.

2.5 Modelado

> **Methodology Card — Baselines**
> **Pregunta:** ¿cuánto valor aporta cada nivel de complejidad sobre reglas simples?
> **Horizonte:** todo el bloque test, evaluado únicamente en timestamps observados.
> **Features:** ninguna covariable exógena; los baselines usan solo la historia del target.
> **Modelos:** Naive, Seasonal Naive (168 h), medias móviles (7/30/90), Exponential Smoothing (ETS).
> **Seed:** 42.
> **Riesgo declarado:** los gaps horarios hacen que "168 filas antes" no siempre equivalga a "168 horas antes"; Seasonal Naive se alinea por timestamp real, no por posición de fila.
> **Limitación:** no se midió coste computacional (tiempo de ajuste) en este notebook; se incorpora recién en el Capítulo 5.

Las métricas de evaluación son:

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}$$

$$\mathrm{MASE} = \frac{\frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|}{\frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|}, \quad m = 168$$

$$\mathrm{WAPE} = 100 \times \frac{\sum|y_t - \hat{y}_t|}{\sum|y_t|}$$

MASE se escala respecto al error absoluto medio de un seasonal naive semanal calculado **sobre train**, cuyo valor en este dataset es 399.13 vehículos/hora. Un MASE menor a 1 indica que el modelo supera a ese seasonal naive de referencia; ninguno de los baselines evaluados lo logra salvo el propio Seasonal Naive 168.

Modelo	MAE	RMSE	MASE	WAPE (%)
`SeasonalNaive168`	365.68	734.59	**0.916**	11.05
`ETS`	785.23	950.38	1.967	23.73
`MA90`	1,731.82	2,030.14	4.339	52.33
`MA30`	1,776.28	2,160.39	4.450	53.67
`MA7`	2,529.78	3,125.83	6.338	76.44
`Naive`	3,101.35	3,660.75	7.770	93.71

Tabla 9. Comparación de baselines sobre el bloque de test (8,115 observaciones), ordenada por MASE.

`ETS` se ajustó sobre las últimas 52 semanas de la serie regular interpolada, con tendencia amortiguada aditiva y estacionalidad aditiva de periodo 168 (`trend="add"`, `damped_trend=True`,

`seasonal="add"`, `seasonal_periods=168`). A pesar de modelar explícitamente tendencia y estacionalidad, queda muy por detrás del Seasonal Naive: en una serie donde la estacionalidad semanal es casi determinista, copiar el valor de hace una semana es más difícil de superar de lo que la intuición sugiere. Las medias móviles, al ser pronósticos constantes, ignoran por completo el patrón intradiario y pierden por un margen aún mayor.

Como ejercicio de comunicación de incertidumbre —no como forecasting probabilístico riguroso, que se aborda en el Capítulo 9— se construyó una banda heurística de ±1.96 desviaciones robustas (basadas en MAD) alrededor del Seasonal Naive, usando los errores **one-step** de train. El notebook no reporta la cobertura empírica de esa banda contra el test; esa verificación queda pendiente y se retoma con metodología adecuada en el Capítulo 9.

> **Decisión ejecutiva.** Si el baseline estacional ya captura la mayor parte de la señal, un modelo más complejo debe justificar su costo antes de ser considerado.

2.6 Análisis posterior: diagnóstico de residuos

El diagnóstico se aplica al modelo con menor MASE, `SeasonalNaive168`, sobre sus 8,115 residuos de test.

> **Guía de lectura: Hipótesis nulas (H_0) en el diagnóstico de residuos.**
> En esta etapa, buscamos que los residuos sean "ruido blanco". Por tanto, idealmente **no queremos rechazar** estas hipótesis nulas.
>
> | Test | Interpretación de H_0 (El escenario ideal) |
> |---|---|
> | Ljung-Box | Ausencia de autocorrelación serial: los residuos no tienen patrones temporales restantes. |
> | Jarque-Bera | Normalidad: los residuos siguen una distribución gaussiana (simétrica y con colas ligeras). |
> | ARCH LM | Homocedasticidad: la varianza del error es constante a lo largo del tiempo (no hay clusters de volatilidad). |

Test	H0	Estadístico	p-valor	Decisión (α=0.05)
Ljung-Box (lag 24)	sin autocorrelación	19,574.86	0.000	rechazar H0
Ljung-Box (lag 168)	sin autocorrelación	30,021.44	0.000	rechazar H0
Jarque-Bera	normalidad	148,993.14	0.000	rechazar H0
ARCH LM (24 lags)	homocedasticidad condicional	2,670.80	0.000	rechazar H0

Tabla 10. Diagnóstico de residuos del mejor baseline.

Los cuatro tests rechazan su hipótesis nula. Los residuos de `SeasonalNaive168` conservan autocorrelación tanto diaria como semanal (Ljung-Box), no son normales (asimetría −1.18,

curtosis 23.86 —colas mucho más pesadas que una normal—) y presentan heterocedasticidad condicional (ARCH LM). Esta combinación tiene una consecuencia práctica directa: cualquier intervalo de predicción construido con supuestos gaussianos sobre este modelo subestimará sistemáticamente el riesgo en las colas. El error por horizonte dentro de la semana (168 horas) no es plano —varía según en qué punto del ciclo semanal cae la hora pronosticada— y el error por hora del día muestra el mismo patrón que la propia estacionalidad intradiaria de la serie: el modelo falla más donde la señal tiene más varianza, no de forma homogénea.

Este notebook no calculó una tabla explícita de sesgo (bias) segmentado por hora o por régimen; ese análisis aparece de forma sistemática recién a partir del Capítulo 4.

2.7 Lecciones metodológicas

1. La rejilla horaria no es perfecta: consolidar duplicados y declarar gaps explícitamente es parte del modelado, no un paso de limpieza separado de él.
2. La estacionalidad diaria y semanal domina la señal; un baseline estacional bien alineado por timestamp es, en este dataset, difícil de superar incluso con métodos que modelan tendencia y estacionalidad explícitamente.
3. Un split aleatorio filtra información del futuro hacia el entrenamiento y responde una pregunta distinta de la que importa en producción; que produzca un error **menor** no lo vuelve válido.
4. La correlación entre tráfico y variables meteorológicas es descriptiva: no demuestra causalidad ni, más importante para el diseño del sistema, disponibilidad futura de esas covariables.
5. Las bandas de incertidumbre gaussianas simples son inadecuadas cuando los residuos muestran autocorrelación, colas pesadas y heterocedasticidad, como ocurre aquí de forma simultánea.

Conexión con el Capítulo 3. El Capítulo 3 (ARIMA) modela explícitamente la dependencia lineal que este baseline deja sin capturar en sus residuos, y somete esa modelización al mismo estándar de diagnóstico usado aquí.

Verificación con el sílabo. Las competencias de la Semana 1 quedan materializadas en: el contraste didáctico de splits (Sección 2.4, Tabla 8) y la evidencia de que el baseline estacional bien alineado es difícil de superar (Tabla 9, Sección 2.1). El detalle completo por competencia queda en Tabla 5.

2.8 Deudas abiertas del experimento

- **Fácil.** Comparar Seasonal Naive de 24 horas contra el de 168 horas sobre el mismo test, para aislar el componente diario del semanal.
- **Intermedio.** Construir intervalos empíricos por cuantiles de error, condicionados a la hora del día, como alternativa a la banda gaussiana heurística de la Sección 2.5.
- **Difícil.** Implementar rolling-origin mensual sobre este mismo dataset y reportar la distribución de MASE entre orígenes, en vez de un único split fijo.

CAPÍTULO 3

Semana 2: ARIMA, estacionariedad y diagnóstico

Dataset: Appliances Energy Prediction · **Notebook:** `02_semana_2_arima_estacionariedad_diagnostico.ipynb` · **Fuente:** UCI Machine Learning Repository, licencia CC BY 4.0.

Ficha de caso — Semana 2
Dominio: energía residencial.
Problema de negocio: anticipar el consumo horario.
Usuario: planificación energética o eficiencia energética.
Decisión que habilita: detectar picos, dimensionar capacidad o recursos.
Horizonte: 24 horas.
Baselines obligatorios: ETS, Theta, Seasonal Naive 24.
Riesgo principal: elegir el modelo solo por AIC/BIC.
Métrica técnica: MASE.
Métrica de negocio: error promedio en Wh/hora.

3.1 Fundamentos teóricos aplicados

El resto del capítulo se apoya en tres bloques conceptuales: la familia ARIMA, los criterios de selección y los diagnósticos de residuos, descritos aquí en términos accionables para un profesional de industria.

Estacionariedad. Una serie es **estacionaria** cuando su media, su varianza y su estructura de autocorrelación no cambian con el tiempo. Importa porque ARIMA estima coeficientes a partir del pasado y los extrapola hacia el futuro: si la serie no es estacionaria, los parámetros estimados con datos pasados no son transferibles al futuro, y el modelo extrapola propiedades que ya no existen. No es un requisito de los baselines —el Seasonal Naive de 24 horas la ignora por diseño, porque su supuesto es exactamente que el patrón de hace un día se repite—, pero sí lo es de ARIMA.

Diferenciación. Si la serie no es estacionaria, la primera herramienta es diferenciar: modelar los **cambios** entre observaciones consecutivas en lugar de los niveles. La diferencia regular $y_t - y_{t-1}$ elimina tendencias; la diferencia diaria $y_t - y_{t-24}$ elimina el ciclo de 24 horas; la log-

-diferencia estabiliza la varianza de series con cola derecha como esta. El parámetro d de ARIMA cuenta cuántas diferenciaciones se aplicaron antes de modelar la dependencia.

Procesos autorregresivos (AR). Un proceso AR(p) expresa el valor actual como una combinación lineal de sus propios valores pasados más una perturbación aleatoria:

$$y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2} + \ldots + \varphi_p y_{t-p} + \varepsilon_t$$

en consumo residencial, el uso de energía de esta hora depende del de la hora anterior (y de las $p-1$ anteriores). El coeficiente de un AR(1) se lee como cuánto persiste el nivel de una hora a la siguiente:

$$y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2} + \ldots + \varphi_p y_{t-p} + \varepsilon_t$$

Procesos de media móvil (MA). Un proceso MA(q) expresa el valor actual como una combinación de los **errores** de pronóstico pasados, no de los valores en sí:

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \ldots + \theta_q \varepsilon_{t-q}$$

captura shocks puntuales que se disipan en pocos periodos —por ejemplo, el encendido de un electrodoméstico que desplaza el consumo una hora y desaparece—.

ARIMA. ARIMA(p,d,q) combina ambos componentes sobre una serie diferenciada d veces hasta volverla estacionaria. Es una familia lineal: expresa el consumo como combinación de su propio pasado (AR), de sus errores pasados (MA) y de ruido nuevo, con la menor cantidad de parámetros posible.

El ciclo de identificación, estimación y diagnóstico. La metodología de esta semana sigue el ciclo Box-Jenkins en tres pasos: identificar la familia y los órdenes plausibles con ACF/PACF y tests de estacionariedad; estimar los parámetros sobre train; y diagnosticar los residuos para detectar estructura no capturada. La tentación profesional es saltarse el último paso cuando las métricas de error parecen aceptables; este capítulo muestra que el diagnóstico es precisamente lo que delata la estructura estacional diaria que el modelo dejó sin modelar.

Auto Covariance Function (ACF) y **Partial Auto Covariance Function (PACF)**. El ACF mide la correlación de la serie con sus propios rezagos; el PACF mide la correlación con un rezago específico, descontando la contribución de los rezagos intermedios:

$$\hat{\rho}_k = \sum_{t=k+1}^{n} (y_t - |y|) \frac{y_{t-k} - |y|}{\sum_{t=1}^{n}} (y_t - |y|)^2$$

Se usan como guía de órdenes: un PACF que se corta después de p rezagos sugiere AR(p); un ACF que decae lentamente sugiere que hace falta diferenciar. En este capítulo, el ACF de la diferencia regular muestra picos en múltiplos de 24 horas: señal de que queda estructura diaria por capturar.

Akaike Information Criterion (AIC) y **Bayesian Information Criterion (BIC)**. AIC y BIC penalizan la verosimilitud del ajuste por el número de parámetros del modelo ($2k$ o $k \log n$, respectivamente): existen porque un modelo con más parámetros casi siempre ajusta mejor el pasado que uno con menos, aunque no necesariamente generalice mejor a datos nuevos. Son

una forma de estimar, sin tocar el bloque de validación ni de test, cuánto de esa mejora aparente es señal real y cuánto es sobreajuste. Se usan para **preseleccionar** candidatos; la decisión final pertenece a la validación.

Diagnóstico de residuos. Después de ajustar, los residuos —lo que el modelo no explicó— deben comportarse como ruido: sin autocorrelación (Ljung-Box), sin heterocedasticidad condicional (ARCH LM) y, si se usarán intervalos gaussianos, aproximadamente normales (Jarque-Bera). Un residuo con estructura es señal de que quedó señal sin modelar, incluso cuando las métricas de error agregadas parecen aceptables. La lectura operativa de cada test es directa: autocorrelación residual significa que el modelo dejó información aprovechable sobre la mesa; heterocedasticidad condicional, que la incertidumbre cambia con el nivel de consumo y los intervalos gaussianos pierden calibración; no normalidad, que las colas reales son más pesadas que las supuestas.

Por qué AIC/BIC no sustituyen la validación temporal. AIC y BIC se calculan sobre la misma muestra que estima los parámetros: miden ajuste dentro de muestra, penalizado. La validación temporal mide lo que importa en producción: el error sobre datos que el modelo no vio. La secuencia correcta —grid en train, selección por MAE en validation, test consultado una sola vez— existe precisamente porque el criterio de información no garantiza generalización. Este capítulo es la demostración empírica: el orden ganador por AIC en train pierde en test contra modelos más simples.

3.2 Configuración y contrato de datos

El archivo `data/02_energy/energy.csv` contiene 19,735 filas y 29 columnas a frecuencia de 10 minutos. El target `y` (energía de electrodomésticos, Wh por intervalo) se agrega por hora sumando los intervalos, mientras que las 27 variables restantes —sensores continuos de temperatura y humedad interior/exterior, presión, viento, visibilidad y dos variables de control `rv1`/`rv2`— se promedian. El resultado es una serie horaria de 3,290 observaciones, del 2016-01-11 17:00 al 2016-05-27 18:00.

Auditoría de calidad. A diferencia del Capítulo 2, este dataset no tiene nulos, no tiene filas ni timestamps duplicados, y la rejilla de 10 minutos es perfectamente regular: 19,735 timestamps esperados, 19,735 observados, 0 gaps. Es el dataset más limpio de los ocho en términos de integridad estructural.

Estadísticos descriptivos del target (horario, Wh/h): media 97.69, desviación estándar 102.52 —una dispersión mayor que la media, señal de asimetría—, mediana 60, percentil 99 = 576.6, máximo 1,080. La cola derecha pronunciada motiva comparar, más adelante, el nivel contra su log-diferencia.

Contrato de disponibilidad de covariables. Antes de modelar, el notebook clasifica explícitamente qué variables podrían usarse como exógenas en un ARIMAX y bajo qué condición:

Variable	Disponibilidad futura	Recomendación
Calendario	Conocida	Usar
Pronóstico meteorológico	Depende del escenario	Específica del escenario
Sensores interiores contemporáneos	No conocida salvo pronóstico separado	Solo como rezago
`lights` contemporáneo	Adyacente al target; normalmente desconocida	Excluir del horizonte futuro
`rv1` / `rv2`	Ruido conocido pero inútil	Control negativo

Tabla 11. Contrato de disponibilidad futura de covariables candidatas.

`rv1` y `rv2` son variables sintéticas incluidas en el dataset original de UCI precisamente para verificar que ningún modelo les asigne importancia espuria; se documentan aquí porque cumplen esa función de control negativo a lo largo del capítulo.

3.3 Análisis previo (pre-modeling)

El perfil horario e intradiario confirma un ciclo diario claro, con menor amplitud relativa que el tráfico del Capítulo 2. Una descomposición STL robusta (periodo 24) sobre la serie horaria completa separa tendencia, componente diaria y residuo. La detección de outliers con z-score robusto MAD ($|z| > 4.5$) sobre ese residuo marca 769 candidatos —una tasa considerablemente más alta que en el Capítulo 2 (23.4 % de las 3,290 observaciones frente a 12.1 % en Metro Traffic)—, consistente con un consumo doméstico más errático que el tráfico vehicular. El análisis CUSUM ubica la mayor desviación acumulada en 2016-02-01 15:00, sin implicar causalidad.

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Nivel	−9.13 (3.10e-15)	0.042 (0.10)	Estacionaria por ambos tests
Diferencia regular ($y_t - y_{t-1}$)	−16.91 (1.02e-29)	0.018 (0.10)	Estacionaria por ambos tests
Diferencia diaria ($y_t - y_{t-24}$)	−13.65 (1.62e-25)	0.007 (0.10)	Estacionaria por ambos tests
Log-diferencia ($\log(1+y_t) - \log(1+y_{t-1})$)	−16.62 (1.71e-29)	0.013 (0.10)	Estacionaria por ambos tests

Tabla 12. Tests de estacionariedad sobre el nivel y tres transformaciones candidatas.

A diferencia del Capítulo 2, donde ADF y KPSS discrepaban sobre el nivel, aquí ambos tests coinciden en que el **nivel mismo** ya es estacionario. Esto no elimina la necesidad de explorar $d = 1$ en el grid de la Sección 3.5 —la estacionariedad formal no implica ausencia de estructura autocorrelacionada aprovechable—, pero sí anticipa que un orden con $d = 0$ es un candidato

razonable. El ACF de la diferencia regular muestra picos en múltiplos de 24 horas, señal de que queda estructura diaria por capturar incluso después de diferenciar una vez.

3.4 Estrategia de validación

Split requerido: temporal 70/15/15. Train estima parámetros, validation elige el orden `(p,d,q)` y compara familias de modelos, test permanece sellado hasta la selección final.

```
|---------- Train: 70% (2,303 obs.) ----------|-- Val: 15% (493) --|-- Test: 15% (494) --|
2016-01-11 17:00                   2016-04-16 15:00   2016-05-07 04:00      2016-05-27 18:00
```

Figura 4. Diagrama del split 70/15/15 y tamaños resultantes.

El horizonte operacional elegido es de 24 horas, lo que permite hasta 20 orígenes de evaluación dentro del bloque de test (494 observaciones ÷ 24 ≈ 20). MASE se escala respecto al error absoluto medio de un salto diario ($m = 24$) calculado sobre train.

3.5 Modelado

Nota. Los conceptos de procesos AR, MA, ARIMA, ACF/PACF y los criterios AIC/BIC se presentan en la Sección 3.1; aquí se aplican directamente al caso.

> **Methodology Card — Cuándo usar ARIMA y cuándo no**
> **Use ARIMA:** una serie, frecuencia regular, dependencia lineal, historia suficiente y residuos diagnosticables.
> **Evite ARIMA solo:** múltiples estacionalidades largas, miles de series, relaciones no lineales fuertes, covariables futuras desconocidas o cambios de régimen frecuentes.
> **Selección:** AIC/BIC en train, MAE en validation, test consultado una sola vez.
> **Seed:** 42.

$$\text{AIC} = -2\log L + 2k, \quad \text{BIC} = -2\log L + k\log n$$

$$\text{MASE} = \frac{\text{MAE}}{\frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|}, \quad m = 24$$

Grid search en train. Se evaluaron 18 órdenes candidatos ($p, q \in \{0,1,2\}$, $d \in \{0,1\}$), ajustados únicamente sobre train (7.4 segundos en total; ningún ajuste falló). El orden con menor AIC fue $(1,0,2)$, con AIC=34,211.83 y BIC=34,240.54—separado por menos de 1 punto de AIC de los siguientes tres candidatos, una diferencia estadísticamente poco informativa por sí sola.

Selección por validación. Los cinco mejores órdenes por AIC se reevaluaron en el bloque de validation completo:

Orden (p,d,q)	MAE (val.)	MASE (val.)	WAPE (val., %)
(1, 0, 2)	**301.97**	**0.887**	53.90
(2, 0, 2)	302.01	0.888	53.91
(2, 0, 1)	302.03	0.888	53.92
(2, 1, 1)	302.03	0.888	53.92
(2, 1, 2)	302.04	0.888	53.92

Tabla 13. Los cinco candidatos con menor AIC, evaluados en validation.

Los cinco candidatos son prácticamente indistinguibles en validation; se selecciona $(1, 0, 2)$ por tener el MAE marginalmente menor. El modelo se reajusta sobre train+validation (2,796 observaciones) antes de tocar el test:

Parámetro	Valor (error estándar)
const	586.96 (36.86)
ar.L1	0.639 (0.024)
ma.L1	−0.175 (0.026)
ma.L2	0.152 (0.017)
σ^2	157,900 (2,858.70)
AIC (train+val)	41,416.49
Ljung-Box in-sample (L1), p-valor	0.93 (no rechazar H0)
Jarque-Bera in-sample, p-valor	0.00 (rechazar H0; asimetría 2.37, curtosis 11.76)

Tabla 14. Resumen del ajuste ARIMA(1,0,2) sobre train+validation.

El modelo ajustado, en forma explícita, es:

$$\hat{y}_t = 586.96 + 0.639 y_{t-1} - 0.175 \varepsilon_{t-1} + 0.152 \varepsilon_{t-2}$$

El coeficiente `ar.L1=0.639` significa, en términos llanos, que alrededor del 64 % del nivel de una hora persiste en la siguiente, antes de sumar ruido nuevo —la memoria de corto plazo que la parte AR del modelo aprovecha—.

Comparación en test (494 observaciones, consultadas una sola vez):

Modelo	MAE	RMSE	MASE	WAPE (%)
`ETS`	221.71	372.52	**0.681**	38.18
`Theta`	243.26	383.62	0.748	41.89
`SeasonalNaive24`	271.48	489.47	0.834	46.75
`ARIMA(1,0,2)`	293.28	424.40	0.901	50.50

Tabla 15. Comparación final de modelos sobre el bloque de test, ordenada por MASE.

Este es el resultado central del capítulo: el modelo ARIMA seleccionado con el procedimiento formalmente correcto —grid en train, selección en validation, test consultado una única vez—

pierde en el test contra ETS, contra Theta y contra el propio Seasonal Naive de 24 horas. `ETS` se ajustó con tendencia amortiguada aditiva y estacionalidad aditiva de periodo 24 (`trend="add"`, `damped_trend=True`, `seasonal="add"`, `seasonal_periods=24`); `Theta` usa la implementación de `statsforecast` con `season_length=24`. Ninguno de los dos exige el ciclo de identificación de órdenes que sí exige ARIMA, y ambos generalizan mejor en este dataset.

> **Decisión ejecutiva.** No se recomienda elegir un modelo solo por AIC/BIC. La validación temporal y el impacto en picos de consumo deben decidir la adopción.

3.6 Análisis posterior (post-modeling)

El diagnóstico de residuos de esta sección se aplica deliberadamente a `ARIMA(1,0,2)`, no al modelo con menor MASE en test (`ETS`): el objeto de estudio metodológico de esta semana es el ciclo de diagnóstico Box-Jenkins, y ese ciclo se ilustra sobre la familia que lo requiere. Los residuos in-sample (2,766 observaciones, tras descartar el burn-in inicial) tienen media −0.015 y desviación estándar 397.22.

El test de Ljung-Box responde una pregunta concreta: ¿los residuos del modelo son ruido blanco, o todavía conservan autocorrelación aprovechable? Su lógica es directa: si los residuos son ruido, la autocorrelación muestral de los residuos $\hat{\rho}_k$ debería ser cercana a cero en todos los rezagos; el estadístico $Q(h)$ suma esas autocorrelaciones al cuadrado hasta el rezago h, de modo que cualquier autocorrelación distinta de cero lo hace crecer:

$$Q(h) = n(n+2)\sum_{k=1}^{h}\frac{\hat{\rho}_k^2}{n-k}$$

La lectura tiene dos pasos. Primero, $Q(h)$ se compara con una distribución χ^2: si Q es grande, el p-valor cae y se rechaza H_0 (ausencia de autocorrelación) —quedó estructura temporal sin modelar—. Segundo, el rezago h elegido determina qué parte de la estructura se puede detectar: una dependencia que vive en el rezago 24 es invisible con $h = 10$. Ese es exactamente el fenómeno de la Tabla 16: a $h = 10$ los residuos parecen limpios; a $h = 20$ y $h = 30$ el test rechaza, porque el ciclo diario de periodo 24 solo se manifiesta en rezagos largos.

Test	H0	Estadístico	p-valor	Decisión (α=0.05)
Ljung-Box (lag 10)	sin autocorrelación	6.95	0.730	no rechazar H0
Ljung-Box (lag 20)	sin autocorrelación	44.99	0.0011	rechazar H0
Ljung-Box (lag 30)	sin autocorrelación	221.91	3.65e-31	rechazar H0
Jarque-Bera	normalidad	11,750.74	0.000	rechazar H0
ARCH LM (24 lags)	homocedasticidad condicional	140.40	1.97e-18	rechazar H0

Tabla 16. Diagnóstico de residuos de ARIMA(1,0,2).

El patrón es revelador: a lag 10 los residuos parecen limpios, pero la estructura reaparece a partir del lag 20 —consistente con que ARIMA(1,0,2), sin componente estacional explícito, no captura

por completo el ciclo diario (periodo 24) que el ACF de la Sección 3.3 ya insinuaba—. La cobertura empírica de los intervalos gaussianos del `get_forecast` de statsmodels resultó 91.09 % para el nominal 80 % y 94.94 % para el nominal 95 %: los intervalos son, si acaso, más anchos de lo necesario en el nivel 80 %, no más angostos, un patrón distinto al de sobreconfianza que suele asumirse por defecto.

Segmento	MAE medio	Mediana	n
Valle (Q1)	274.89	276.96	126
Medio	162.75	196.96	245
Pico (Q4)	572.13	378.32	123

Tabla 17. Error absoluto segmentado por nivel de la serie en test (cuartiles de y).

El error se concentra fuertemente en los picos de consumo: el MAE medio en el segmento de mayor consumo (572.13) es más de tres veces el del segmento medio (162.75). Un modelo lineal de orden bajo, ajustado para minimizar error promedio, sacrifica precisión en los eventos de mayor magnitud —que son, operacionalmente, los más relevantes para dimensionar capacidad—.

3.7 Lecciones metodológicas

1. ADF y KPSS responden hipótesis distintas; deben usarse ambos junto con inspección gráfica, no uno solo como criterio de decisión.
2. AIC y BIC seleccionan dentro de una familia de modelos ajustada a train, pero es la validación —no el criterio de información— la que estima generalización real.
3. El test no participa en el grid search ni en la selección de orden; se consulta una única vez, al final.
4. Residuos autocorrelacionados en lags largos, incluso cuando los lags cortos parecen limpios, indican estructura estacional no capturada por la especificación elegida.
5. Picos, colas pesadas y heterocedasticidad degradan la validez de intervalos gaussianos, incluso cuando la cobertura empírica agregada parece razonable.
6. Un ARIMAX exige covariables futuras disponibles o pronosticadas; no basta con que existan en el histórico.

Conexión con el Capítulo 4. El Capítulo 4 incorpora estacionalidad explícita (SARIMA) y variables exógenas reales (SARIMAX), evaluadas con rolling-origin en vez de un único split fijo.

Verificación con el sílabo. Las competencias de la Semana 2 quedan materializadas en: el ciclo de identificación, estimación y diagnóstico (Sección 3.5), la validación temporal con test sellado (Sección 3.4) y la demostración empírica de que AIC no garantiza ganar en test. Ver Tabla 5 para la trazabilidad completa por competencia.

3.8 Deudas abiertas del experimento

- **Fácil.** Ampliar el grid a $p, q \leq 3$ y comparar AIC, BIC y MAE de validation para verificar si la selección cambia.

- **Intermedio.** Incorporar un componente estacional SARIMA diario y contrastar sus residuos contra los de la Tabla 16.
- **Difícil.** Construir intervalos de predicción por bootstrap que respeten bloques diarios completos, en vez de asumir normalidad.

CAPÍTULO 4

Semana 3: SARIMAX y variables exógenas

Dataset: Beijing PM2.5 · **Notebook:** `03_semana_3_sarimax_variables_exogenas.ipynb` · **Fuente:** UCI Machine Learning Repository, licencia CC BY 4.0.

> **Ficha de caso — Semana 3**
> **Dominio:** calidad del aire.
> **Problema de negocio:** anticipar episodios de PM2.5.
> **Usuario:** autoridad ambiental, operaciones o salud.
> **Decisión que habilita:** alertas, restricciones operativas o protección de actividades sensibles.
> **Horizonte:** 168 horas.
> **Baseline obligatorio:** SARIMA sin exógenas.
> **Riesgo principal:** usar clima observado como si fuera futuro.
> **Métrica técnica:** MAE.
> **Métrica de negocio:** reducción de error en episodios de contaminación.

4.1 Fundamentos teóricos aplicados

Conviene precisar cuatro nociones antes de avanzar: la extensión estacional de ARIMA, el papel de las variables exógenas y las dos disciplinas que este capítulo convierte en decisión,la validación temporal y la imputación causal, en términos accionables para un profesional de industria.

Estacionalidad explícita. Una serie con estacionalidad explícita repite su patrón con un periodo conocido —aquí, 24 horas—. El Capítulo 3 mostró el costo de ignorarla: el ARIMA(1,0,2) dejó estructura diaria sin capturar en sus residuos, visible recién a partir del lag 20. La lección es que la estacionalidad no es un detalle del dataset: es una propiedad del problema que el modelo debe declarar, no descubrir por accidente. En calidad del aire, el ciclo diario existe pero es más débil que en tráfico: la concentración de PM2.5 está gobernada más por eventos meteorológicos y de emisión que por el calendario.

SARIMA. SARIMA extiende ARIMA agregando un segundo bloque de términos AR/MA y una diferenciación adicional, pero aplicados no al rezago inmediato sino al periodo estacional s (aquí, 24 horas): en vez de solo "¿qué pasó hace una hora?", el modelo también pregunta "¿qué pasó a esta misma hora, el ciclo anterior?". La notación completa del orden es:

$$\text{SARIMA}(p,d,q) \times (P,D,Q)_s, \quad s = 24$$

donde (p, d, q) son los órdenes no estacionales —los mismos de ARIMA— y (P, D, Q) sus análogos estacionales aplicados al rezago s. La diferenciación estacional, cuando $D \geq 1$, opera sobre el periodo completo:

$$y_t - y_{t-s}, \quad s = 24$$

Es una forma de decirle explícitamente al modelo dónde buscar la dependencia, en lugar de esperar que la encuentre en rezagos cortos. El orden se fija en este capítulo de forma parsimoniosa —("ORDER") = (1,0,1), ("SEASONAL_ORDER") = (1,0,0,24)— según el ACF/PACF y la estacionalidad diaria del análisis previo, no mediante un grid exhaustivo.

SARIMAX. SARIMAX agrega a SARIMA un bloque de regresión sobre variables exógenas: el modelo es conceptualmente una regresión lineal sobre las covariables, temperatura, presión, viento, lluvia, cuyos **residuos** se modelan con la estructura SARIMA. La parte exógena captura variación sistemática asociada al clima; la parte SARIMA captura lo que queda de dependencia temporal después de descontar esa variación:

En forma explícita:

$$y_t = \beta_0 + \sum_{i=1}^{m} \beta_i x_{i,t} + \varepsilon_t$$

donde $x_{i,t}$ son las covariables exógenas y es el residuo ε_t —no el target— el que sigue la estructura SARIMA. Esta separación es la que permite leer los coeficientes β_i como efectos de las covariables, y el bloque SARIMA como la memoria temporal que queda después de descontarlas.

Por eso una covariable que no reduce el residuo autocorrelacionado no está aportando nada que el propio SARIMA no capturara ya.

Variables exógenas. Una variable exógena es información adicional al histórico del target que puede explicar parte de su variación: en este capítulo, las seis covariables meteorológicas (`DEWP`, `TEMP`, `PRES`, `Iws`, `Is`, `Ir`) más la dirección de viento categórica (`cbwd`). La primera pregunta sobre cualquier exógena no es "¿correlaciona con el target?" —casi todo correlaciona con algo— sino "¿estará disponible en el momento de pronosticar?".

Variable observada frente a variable disponible a futuro. Esta distinción organiza el capítulo. Una variable **observada** existe en el histórico porque ya ocurrió: el CSV registra la temperatura que efectivamente hizo. Una variable **disponible a futuro** es la que existirá en el momento de pronosticar: el pronóstico meteorológico que la agencia comprará o generará, no el valor que se conocerá después. En el experimento, las seis covariables son observadas; en producción, solo estarán disponibles como pronóstico. La confusión entre ambas es la fuente del escenario oracle.

Escenario oracle. Cuando un modelo usa covariables **observadas** del propio bloque de test, el clima que efectivamente ocurrió durante el horizonte pronosticado, la evaluación se llama escenario oracle: estima el valor superior de la covariable si el pronóstico meteorológico fuera perfecto, no el desempeño alcanzable en producción con un forecast real e imperfecto. Es un techo optimista: útil para saber cuánto podría aportar la covariable, pero engañoso como expectativa de despliegue.

Rolling-origin validation. La validación por rolling-origin reemplaza el split único por varios orígenes de pronóstico: cada fold entrena con una ventana fija (180 días, 4,320 horas) y pronostica el horizonte operacional (7 días, 168 horas), y los orígenes se desplazan hacia adelante (30 días entre folds). Responde una pregunta que el split único no puede: ¿la ganancia del modelo se mantiene cuando cambia el punto de corte, o depende de una sola frontera temporal? La lectura operativa: una mejora media debe acompañarse de evidencia de estabilidad entre folds; si un modelo gana en promedio pero es muy variable entre orígenes, la adopción es más riesgosa.

Imputación causal. Los 2,043 nulos del target (4.66 %) obligan a decidir cómo llenarlos antes de modelar. La imputación causal rellena cada hueco usando solo información del pasado de ese punto —nunca del futuro—, porque cualquier valor imputado con observaciones posteriores filtra información que no existiría al pronosticar. Por eso se descarta explícitamente la interpolación bilateral global: usa observaciones futuras respecto de cada punto imputado.

Forward fill limitado. La estrategia adoptada es `ffill` causal con límite de 24 horas dentro de cada fold, seguida por la mediana de train para gaps más largos que ese límite: copia el último valor conocido hacia adelante mientras el hueco sea corto, y no inventa estructura para los largos:

En notación: para un hueco en el instante t, se copia el último valor observado hacia adelante,

$$\hat{y}_t = y_{t-k}, \quad 1 \le k \le L, \quad L = 24$$

donde k es la distancia al último valor observado. Si el hueco supera L, el punto se imputa con la mediana de train en lugar de seguir copiando.

La regla de oro es que la imputación —como el escalado de las exógenas— se re-ejecuta **dentro de cada fold**, ajustando sus estadísticos solo con train, para que ni la mediana ni la escala del bloque de test se filtren hacia el ajuste del modelo.

Por qué el horizonte es de 168 horas. El horizonte operacional de este capítulo es de 7 días (168 horas), no de 1 hora. La decisión que habilita el pronóstico —alertas y restricciones ante episodios de contaminación— se toma con días de anticipación, no con una hora: una autoridad ambiental necesita saber el viernes si el fin de semana tendrá concentraciones altas para activar medidas. Un horizonte de 168 horas cubre siete ciclos diarios completos, lo que obliga a pronosticar no solo el nivel medio sino también la forma del ciclo en cada día. Esto explica dos decisiones del capítulo: la estacionalidad diaria se modela explícitamente (no se deja que la capturen rezagos cortos) y la validación se hace con ventana deslizante en vez de un único split.

Cómo leer un coeficiente SARIMAX. El bloque exógeno de SARIMAX se lee como en una regresión lineal, con una salvedad: los coeficientes están estandarizados, porque las covariables se escalan con media y desviación de train:

Concretamente, cada covariable se transforma como:

$$z_t = \frac{x_t - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

donde μ_{train} y σ_{train} se calculan solo con el bloque de entrenamiento de cada fold. Por eso un coeficiente se interpreta en desviaciones estándar de la covariable, y por eso el escalado debe re-

-ejecutarse dentro de cada fold: usar la media o la escala del fold completo filtraría información de test.

Un coeficiente estandarizado de la presión indica cuántos microgramos por metro cúbico cambia el pronóstico por cada desviación estándar de presión, manteniendo el resto constante. La inestabilidad entre folds de un coeficiente —como la del bloque meteorológico de este capítulo — es una señal operativa: si el efecto estimado cambia de magnitud o signo según el origen de pronóstico, la covariable no es un ancla confiable para la decisión, aunque mejore el error promedio.

La disponibilidad futura como restricción de diseño. La disponibilidad de una covariable no es un problema del notebook: es una restricción de arquitectura. El sistema de producción debe conectarse a un proveedor de pronóstico meteorológico, negociar su latencia y su resolución horaria, y definir qué hacer si el pronóstico llega tarde o falta. Por eso el capítulo separa dos preguntas que los análisis suelen mezclar: ¿la covariable ayuda en el experimento? y ¿la covariable estará disponible al pronosticar? La primera se responde con la validación; la segunda, con la matriz de disponibilidad de la Sección 4.2. Un sistema que responde solo la primera pregunta está condenado a desplegar un modelo que no puede ejecutar en producción.

Métrica técnica y métrica de negocio. La métrica técnica de este capítulo es el MAE, el error promedio en $\mu g/m^3$: la unidad que una autoridad ambiental puede explicar sin fórmulas y la adecuada cuando el costo del error es proporcional a su magnitud. La métrica de negocio, en cambio, es la reducción de error en episodios de contaminación —los eventos de alta concentración que activan alertas—. Las dos no siempre coinciden: un modelo puede mejorar el promedio global pero fallar justamente en los episodios extremos, que es lo que la segmentación por cuartiles de la Sección 4.6 verifica. Por eso el capítulo reporta ambas: la primera dice si SARIMAX es mejor en promedio; la segunda dice si es mejor donde la decisión importa.

Regla de decisión para incluir una exógena. La regla que este capítulo aplica —y que la decisión ejecutiva resume— tiene tres condiciones: la variable debe mejorar la validación (no solo el ajuste dentro de muestra), ser estable entre folds (no depender de un solo origen de pronóstico) y estar disponible en producción (no requerir información futura no pronosticada). Si falla cualquiera de las tres, la exógena se descarta o se reclasifica. Esta regla es deliberadamente más exigente que mirar significancia estadística o AIC, porque penaliza exactamente los dos errores que un sistema de alerta no puede pagar: sobreestimar la capacidad del modelo y depender de datos que no existirán.

Imputación en producción. La imputación no es solo un paso de limpieza: es la política que el sistema usará cada vez que un sensor falle en producción. La estrategia de este capítulo —`ffill` con límite de 24 horas y mediana de train como respaldo— define explícitamente cuánto tiempo se puede operar con el último valor conocido antes de asumir el nivel promedio. La regla de oro de re-ejecutar la imputación dentro de cada fold tiene su análogo operativo: en producción, los parámetros de imputación se recalibran en cada reentrenamiento, nunca se congelan sobre el histórico completo.

4.2 Configuración y contrato de datos

El archivo `data/03_beijing/beijing_pm25.csv` contiene 43,800 filas y 14 columnas horarias. El target es `pm2.5` (concentración de partículas finas); las covariables numéricas son `DEWP` (punto de rocío), `TEMP` (temperatura), `PRES` (presión), `Iws` (velocidad acumulada de viento), `Is` e `Ir` (horas acumuladas de nieve y lluvia), más una covariable categórica `cbwd` (dirección combinada de viento: SE, NW, cv, NE).

Auditoría de calidad. El target tiene 2,043 valores ausentes (4.66 %), concentrados en gaps interiores, no en los bordes de la serie. A diferencia del Capítulo 2, aquí **no** hay gaps en la rejilla horaria de timestamps —las 43,800 horas del rango 2010-01-02 a 2014-12-31 están todas presentes—; lo que falta es el valor del target dentro de esa rejilla completa, una distinción operacionalmente relevante: no es un problema de reconstrucción del índice temporal, es un problema de imputación del target. Los 2,043 nulos se agrupan en 213 gaps discretos: duración media 9.6 horas, mediana 1 hora, pero con un máximo de 155 horas (casi 6.5 días)—la distribución está fuertemente sesgada por unos pocos gaps muy largos.

Decisión de imputación (documentada, no oculta). Se comparan tres estrategias —eliminación, forward-fill causal e interpolación bilateral— y se descarta explícitamente la interpolación bilateral global porque usa observaciones futuras respecto de cada punto imputado, lo cual constituye fuga de información si se calcula antes del split. La estrategia adoptada es `ffill` causal con límite de 24 horas dentro de cada fold, seguida por la mediana de train para gaps más largos que ese límite. Es importante notar que esta decisión se re-ejecuta **dentro de cada fold** de validación (Sección 4.4), no una sola vez sobre la serie completa.

Estadísticos descriptivos del target (sobre 41,757 observaciones no nulas): media 98.61, desviación estándar 92.05, mediana 72, percentil 95 = 284, máximo 994 —una distribución con cola derecha pronunciada, consistente con episodios de contaminación extrema—.

Matriz de disponibilidad de covariables. La pregunta que separa un escenario oracle de un sistema de producción es la disponibilidad futura de cada covariable. La matriz clasifica las siete columnas antes de modelar:

Variable	¿En el histórico?	¿Disponible en producción?	¿Requiere pronóstico externo?	Recomendación de uso
`DEWP` (punto de rocío)	Sí (observada)	No directamente	Sí — meteorológico	Usar solo con pronóstico externo; con observado en test es oracle
`TEMP` (temperatura)	Sí (observada)	No directamente	Sí — meteorológico	Usar solo con pronóstico externo; con observado en test es oracle
`PRES` (presión)	Sí (observada)	No directamente	Sí — meteorológico	Usar solo con pronóstico externo; coeficiente inestable entre folds
`Iws` (viento acumulado)	Sí (observada)	No directamente	Sí — meteorológico	Usar solo con pronóstico externo
`Is` (nieve acumulada)	Sí (observada)	No directamente	Sí — meteorológico	Usar solo con pronóstico externo
`Ir` (lluvia acumulada)	Sí (observada)	No directamente	Sí — meteorológico	Usar solo con pronóstico externo; coeficiente más estable
`cbwd` (dirección de viento)	Sí (categórica, observada)	No directamente	Sí — meteorológico	Codificar one-hot; usar solo con pronóstico externo

Tabla 18. Matriz de disponibilidad de covariables: histórico, producción, pronóstico externo y recomendación.

4.3 Análisis previo (pre-modeling)

Sobre una serie auxiliar construida solo para inspección (`ffill` con límite 24 y mediana como respaldo global, nunca usada para modelar ni evaluar), la descomposición STL diaria (periodo 24) y semanal (periodo 168) permite calcular una fuerza estacional descriptiva basada en varianzas: 0.049 para el ciclo diario y 0.149 para el semanal. Ambos valores son notablemente menores que la estacionalidad de Metro Traffic (Capítulo 2): la concentración de PM2.5 está mucho menos gobernada por un patrón calendario determinista y mucho más por eventos meteorológicos y de emisión.

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Nivel	−21.55 (0.000)	0.072 (0.10)	Estacionaria por ambos tests
Diferencia regular	−36.87 (0.000)	0.002 (0.10)	Estacionaria por ambos tests
Diferencia diaria (24h)	−31.04 (0.000)	0.002 (0.10)	Estacionaria por ambos tests

Tabla 19. Tests de estacionariedad sobre la serie de inspección.

La detección de outliers sobre el residuo STL diario (MAD, $|z| > 4.5$) marca 2,250 candidatos —la tasa más alta de los tres capítulos estadísticos hasta ahora—, coherente con una serie dominada por picos de contaminación más que por un patrón suave. El CUSUM descriptivo ubica su mayor desviación en 2013-04-01 08:00.

Disponibilidad de covariables: el punto central del capítulo. Las seis covariables numéricas (`DEWP`, `TEMP`, `PRES`, `Iws`, `Is`, `Ir`) usadas en la Sección 4.5 son mediciones **observadas** del pasado, no pronósticos. Evaluar un modelo que las usa como si estuvieran disponibles en el momento de pronosticar constituye, por definición, un **escenario oracle**: estima el valor superior de la covariable si el pronóstico meteorológico fuera perfecto, no el desempeño alcanzable en producción con un forecast meteorológico real e imperfecto.

4.4 Estrategia de validación

Rolling-origin con 3 folds, en vez del split único usado en capítulos anteriores. Cada fold entrena con 180 días (4,320 horas) y pronostica 7 días (168 horas); los orígenes de test están separados 30 días entre sí, sobre los últimos meses de la serie.

```
Fold 1: train [37872:42192] (4320 h) -> test [42192:42360] (168 h)
Fold 2: train [38592:42912] (4320 h) -> test [42912:43080] (168 h)
Fold 3: train [39312:43632] (4320 h) -> test [43632:43800] (168 h)
```

Figura 5. Estructura de los tres folds rolling-origin.

La imputación y el escalado de exógenas se recalculan **dentro de cada fold**: la función `prepare_fold` aplica `ffill(limit=24)` con mediana de train como respaldo, y las covariables numéricas se estandarizan con media y desviación de train, nunca del fold completo. Esto evita que información del bloque de test —incluida su propia mediana o su propia escala— se filtre hacia el ajuste del modelo.

Discusión de disponibilidad en producción. Como se anticipó en 4.3, esta estrategia de validación evalúa SARIMAX en modo oracle: usa el clima efectivamente observado durante el horizonte de test. Un sistema en producción necesitaría sustituir esas seis columnas por un forecast meteorológico externo, cuyo propio error se propagaría al pronóstico de PM2.5. El capítulo no cuantifica esa degradación adicional —queda fuera de su alcance— pero la señala explícitamente como brecha entre este experimento y un despliegue real.

4.5 Modelado

Nota. Los conceptos de SARIMA, estacionalidad explícita y SARIMAX se presentan en la Sección 4.1; aquí se aplican directamente al caso.

> **Methodology Card — Exógenas**
> Use una variable exógena si mejora la validación, es estable entre folds y existe en el horizonte de pronóstico. Meteorología observada en test representa un escenario oracle; producción necesita pronóstico meteorológico, no observación. `cbwd` se codifica one-hot. No se debe incorporar una exógena solo porque correlaciona con el target.
> **Seed:** 42.

$$\text{MAE} = \frac{1}{n}\sum|e_t|, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum e_t^2}, \quad \text{WAPE} = 100 \times \frac{\sum|e_t|}{\sum|y_t|}$$

Este capítulo **no reporta MASE** —a diferencia de los Capítulos 2 y 3—; se documenta la omisión en vez de calcular una cifra no presente en el notebook original.

El orden SARIMA/SARIMAX se fija de forma parsimoniosa según el ACF/PACF y la estacionalidad diaria de la Sección 4.3, no mediante grid search exhaustivo: $(\text{ORDER}) = (1, 0, 1)$, $(\text{SEASONAL_ORDER}) = (1, 0, 0, 24)$, con `trend="c"` y `simple_differencing=True` para acotar el costo computacional del backtesting con tres folds (107.8 segundos en total).

Fold	Modelo	MAE	RMSE	WAPE (%)	AIC
1	SARIMA	105.73	129.68	99.44	35,974.62
1	SARIMAX	**84.65**	**102.68**	**79.61**	36,155.67
2	SARIMA	113.05	143.68	74.72	35,854.93
2	SARIMAX	**99.28**	138.24	**65.61**	36,597.79
3	SARIMA	92.20	120.64	76.07	36,840.26
3	SARIMAX	**71.03**	**102.44**	**58.60**	37,827.71

Tabla 20. Resultados por fold: SARIMA (sin exógenas) vs. SARIMAX (con exógenas observadas).

SARIMAX reduce el MAE en los tres folds: 19.9 %, 12.2 % y 23.0 % respectivamente, para una **mejora media de 18.36 %**—esta es la cifra que se obtiene al recalcular directamente sobre los resultados del notebook. El README del repositorio reporta una mejora de " 20.7 %"; esta monografía usa 18.36 % por ser la cifra verificable a partir de la ejecución real documentada en el notebook, y dado el criterio de esta monografía de no reportar resultados sin respaldo directo en la ejecución citada.

La mejora relativa de cada fold se calcula como:

$$\text{mejora}_{\text{fold}} = \frac{\text{MAE}_{\text{SARIMA}} - \text{MAE}_{\text{SARIMAX}}}{\text{MAE}_{\text{SARIMA}}} \times 100$$

y la cifra de 18.36 % es el promedio de esas tres mejoras por fold, no la mejora sobre el MAE medio agregado —calcularla sobre los MAE medios (103.66 vs 84.99) daría 18.0 %—. Mostrar la

fórmula hace transparente que la cifra verificable no coincide con el 20.7 % del README, y que la diferencia entre promediar mejoras relativas y calcular la mejora del promedio no alcanza para cerrar esa brecha.

Un matiz que el resumen ejecutivo del README omite: el AIC de SARIMAX es **mayor** que el de SARIMA en los tres folds, a pesar de que SARIMAX gana consistentemente en MAE, RMSE y WAPE fuera de muestra. AIC penaliza los parámetros adicionales de las exógenas más de lo que premia la mejora de verosimilitud dentro de la muestra de ajuste; un procedimiento de selección basado solo en AIC —como el usado para elegir el orden ARIMA del Capítulo 3— no habría preferido SARIMAX sobre SARIMA en este caso, ilustrando por qué la Methodology Card exige validar la mejora fuera de muestra, no solo mirar el criterio de información.

Modelo	MAE medio	Desv. estándar entre folds
SARIMA	103.66	10.58
SARIMAX	84.99	14.13

Tabla 21. Robustez entre folds (media y desviación estándar de MAE).

SARIMAX gana en media pero es **menos estable** entre folds que SARIMA (mayor desviación estándar) —una mejora real, pero con más varianza de la que sugiere mirar solo el promedio—. La inestabilidad de coeficientes estandarizados entre folds confirma este patrón: `PRES` (presión) es el coeficiente más inestable (inestabilidad relativa 3.52), seguido de `Iws` (2.73) y las categorías de `cbwd`; `Ir` (horas de lluvia) es el más estable (0.39).

Lectura honesta del resultado. La mejora de SARIMAX es real **dentro del experimento**: reduce el MAE en los tres folds (19.9 %, 12.2 % y 23.0 %), con una mejora media de 18.36 %. Pero es un escenario oracle porque usa el clima observado durante el horizonte de test, no el que habría estado disponible al pronosticar. En producción se necesita un pronóstico meteorológico externo para las seis covariables, y la mejora real dependerá de la calidad de ese pronóstico —su propio error se propagaría al pronóstico de PM2.5—. La cifra de 18.36 % es, por lo tanto, un límite superior optimista, no una expectativa de despliegue.

> **Decisión ejecutiva.** Las variables exógenas solo deben usarse si mejoran la validación, son estables entre folds y estarán disponibles en producción.

4.6 Análisis posterior (post-modeling)

Los diagnósticos se reportan por fold para no ocultar cambios de régimen entre orígenes.

Fold	Test	Estadístico	p-valor	Decisión
1	Ljung-Box (24)	45.56	0.005	rechazar H0
2	Ljung-Box (24)	48.87	0.002	rechazar H0
3	Ljung-Box (24)	98.15	6.2e-11	rechazar H0
1	Jarque-Bera	131,926.59	0.000	rechazar H0
2	Jarque-Bera	20,851.70	0.000	rechazar H0
3	Jarque-Bera	18,731.07	0.000	rechazar H0
1	ARCH LM (24)	867.09	1.35e-167	rechazar H0
2	ARCH LM (24)	111.04	3.72e-13	rechazar H0
3	ARCH LM (24)	190.42	7.37e-28	rechazar H0

Tabla 22. Diagnóstico de residuos de SARIMAX, por fold.

Los tres folds rechazan las tres hipótesis nulas de forma consistente: autocorrelación residual, no normalidad y heterocedasticidad condicional persisten incluso en el modelo con exógenas. La cobertura empírica de intervalos al 95 % nominal—fold 1: 76.8 %, fold 2: 68.5 %, fold 3: 86.3 %—confirma una calibración pobre y, además, inestable entre folds: ningún fold alcanza la cobertura nominal, y la brecha varía por más de 17 puntos porcentuales entre el mejor y el peor fold.

Segmento	MAE medio	Mediana	n
Bajo (Q1)	57.98	37.17	131
Medio	52.96	39.94	250
Alto (Q4)	178.85	163.36	123

Tabla 23. Error absoluto segmentado por nivel de concentración (cuartiles por fold) y sesgo por fold.

Fold	Sesgo medio (real − pronóstico)
1	−4.42 (leve sobre-pronóstico)
2	+84.43 (sub-pronóstico sustancial)
3	+49.68 (sub-pronóstico moderado)

El error se concentra en los episodios de alta concentración, igual que en el Capítulo 3: el segmento alto (178.85) casi triplica al segmento medio (52.96). El signo y la magnitud del sesgo **no son estables** entre folds —de un sesgo prácticamente nulo en el fold 1 a un sub-pronóstico considerable en el fold 2—, lo cual añade otra dimensión de calibración deficiente más allá de la cobertura de intervalos.

4.7 Lecciones metodológicas

1. Una limpieza superficial —como eliminar solo las primeras filas de la serie— no resuelve nulos interiores distribuidos a lo largo de todo el histórico; se requiere una estrategia de imputación explícita y documentada.

2. La imputación debe ajustarse dentro de cada fold, no una sola vez sobre la serie completa, para no filtrar información de test hacia el ajuste.
3. SARIMAX con clima observado es un escenario oracle: la mejora medida (18.36 %) es un límite superior optimista, no una expectativa realista de producción sin forecast meteorológico.
4. Una mejora media positiva debe acompañarse de evidencia de estabilidad entre folds y coeficientes; SARIMAX mejora en media pero es más variable que SARIMA en ambos frentes.
5. Los intervalos gaussianos pierden calibración cuando coexisten autocorrelación residual y efectos ARCH, como ocurre aquí en los tres folds sin excepción.

El escenario oracle de este capítulo es la segunda de cinco instancias verificadas del principio unificado de fugas (Cap. 10 §10.5): random split geométrico (Cap. 2), oracle temporal-conceptual aquí, `casual + registered = cnt` algebraica (Cap. 5), identidad del cliente (Cap. 6), normalización (Cap. 7). Cinco mecanismos, un solo principio. **Conexión con el Capítulo 5.** El Capítulo 5 abandona la forma paramétrica de ARIMA/SARIMAX y la reemplaza por **features** explícitas y modelos de machine learning tabular, con un embargo temporal declarado para controlar la fuga de información que aquí se maneja mediante el recálculo por fold.

Verificación con el sílabo. Las competencias de la Semana 3 quedan materializadas en: la matriz de disponibilidad de covariables (Sección 4.2, Tabla 18), la estrategia rolling-origin con recalibrado por fold (Sección 4.4) y la lectura honesta del resultado oracle (Tabla 21). Trazabilidad consolidada: Tabla 5.

4.8 Deudas abiertas del experimento

- **Fácil.** Comparar límites de `ffill` de 6, 12 y 24 horas y medir el efecto sobre el MAE de validación.
- **Intermedio.** Incorporar estacionalidad semanal mediante términos de Fourier y contrastar contra el `SEASONAL_ORDER` diario actual.
- **Difícil.** Sustituir el clima observado por pronósticos meteorológicos simulados con error realista, para aproximar el desempeño no-oracle esperable en producción.

CAPÍTULO 5

Semana 4: Feature engineering y Machine Learning (ML)

Dataset: Bike Sharing (UCI) · **Notebook:** `04_semana_4_feature_engineering_ml.ipynb` · **Fuente:** UCI Machine Learning Repository, licencia CC BY 4.0.

> **Ficha de caso — Semana 4**
> **Dominio:** micromovilidad.
> **Problema de negocio:** anticipar la demanda horaria de bicicletas.
> **Usuario:** operaciones o rebalanceo de flota.
> **Decisión que habilita:** redistribuir bicicletas, evitar quiebres de servicio y reducir costos.
> **Horizonte:** ventanas horarias.
> **Baseline obligatorio:** modelos lineales con features temporales bien construidos.
> **Riesgo principal:** leakage.
> **Métrica técnica:** MASE.
> **Métrica de negocio:** error en alquileres/hora.

5.1 Fundamentos teóricos aplicados

El diseño de este capítulo descansa en cinco ideas: cómo convertir una serie en una tabla supervisada, qué información capturan los lags y las ventanas móviles, por qué la codificación cíclica es preferible a los identificadores crudos, y las dos disciplinas que organizan todo el diseño —la validación temporal y la prevención de fuga de información—, presentadas en términos accionables para un profesional de industria.

Tabularización: de serie a tabla supervisada. Los capítulos anteriores modelan la serie como un proceso con forma paramétrica (ARIMA, SARIMAX). El machine learning tabular hace una pregunta distinta: en lugar de suponer una forma, convierte la historia en una tabla donde cada fila es un instante t, las columnas son los valores del pasado que se cree informativos (rezagos, promedios móviles, calendario) y la columna objetivo es el valor que se quiere predecir. Una vez construida esa tabla, el problema de pronóstico se vuelve un problema de regresión estándar: aprender una función que mapee features a target. La ganancia es flexibilidad —los árboles pueden capturar relaciones no lineales e interacciones sin que nadie las especifique—; el costo es que la validez de todo el edificio depende de que cada columna contenga únicamente información que existiría en el momento real de predecir.

Lags. Un rezago (lag) es el valor del target h periodos atrás: `lag_1` es lo que ocurrió hace una hora, `lag_24` lo que ocurrió a esta misma hora ayer, `lag_168` lo que ocurrió hace una semana. Los lags son la memoria cruda del modelo: le permiten responder "¿cómo se parecía este momento a momentos anteriores?". En una serie con estacionalidad diaria y semanal como la demanda horaria de bicicletas, un conjunto de lags que cubra ambas escalas —1, 2, 3, 6, 12, 24, 48 y 168 horas— entrega al modelo exactamente las distancias temporales donde la autocorrelación es alta.

En notación, un rezago de orden k es simplemente:

$$\text{lag}_k(t) = y_{t-k}, \quad k \in \{1, 2, 3, 6, 12, 24, 48, 168\}$$

construido con `shift(k)` para garantizar que la fila del instante t nunca contenga el valor contemporáneo y_t.

Elegir los lags no es arbitrario: se apoya en el ACF/PACF del análisis previo, igual que en los capítulos estadísticos.

Rolling windows y ventanas móviles causales. Una ventana móvil resume un bloque del pasado reciente con un estadístico —media, desviación estándar, mínimo, máximo—. Mientras el lag dice "¿cuánto valía la serie hace exactamente 24 horas?", la ventana dice "¿cómo ha venido evolucionando la serie en las últimas 24 horas?". La distinción es útil porque la demanda de bicicletas depende tanto del nivel puntual de hace un día como de la tendencia reciente: una racha de horas altas anticipa un pico, un descenso sostenido anticipa un valle. El detalle que define la validez de una ventana es su **dirección temporal**: la ventana debe calcularse sobre el pasado estrictamente anterior al origen de predicción, nunca centrada alrededor de él.

Para una ventana de media de amplitud w, la versión causal es:

$$\text{rolling}_{\text{mean}}(t) = \frac{1}{w} \sum_{i=1}^{w} y_{t-i}, \quad w \in \{24, 48, 168\}$$

donde el índice arranca en $t-1$ (por el `shift(1)`), de modo que ningún término y_{t+i} futuro entra en el cálculo. Una ventana centrada reemplazaría ese rango por términos que rodean a t —incluyendo horas posteriores—, que es exactamente la fuga que la Tabla 25 cuantifica.

La Sección 5.4 demuestra con datos cuánto se distorsiona la correlación cuando se usa una ventana centrada (0.400 frente a 0.362 de la versión causal): la ventana centrada "ve" el futuro y por eso parece más informativa, pero esa información no existirá en producción.

Codificación cíclica. Las variables de calendario —hora del día, día de la semana, mes— son cíclicas: la hora 23 y la hora 0 están a una hora de distancia, pero codificadas como números enteros están a 23 unidades. La codificación cíclica transforma cada variable en dos componentes, seno y coseno del ángulo que representa su posición en el ciclo, por ejemplo, la hora del día convertida a un ángulo de 0 a 2π.

Para una variable cíclica c con periodo p, la transformación genera dos features:

$$c_{\sin} = \sin\left(2\pi \frac{c}{p}\right), \quad c_{\cos} = \cos\left(2\pi \frac{c}{p}\right)$$

Así, la hora del día usa $p = 24$ (`hour_sin`, `hour_cos`), el día de la semana $p = 7$ (`dow_sin`, `dow_cos`) y el mes $p = 12$ (`month_sin`, `month_cos`). Con esta codificación, la distancia entre la hora 23 y la hora 0 es pequeña en el espacio (seno, coseno), mientras que con el entero crudo sería de 23 unidades.

El modelo ve así la distancia correcta entre horas vecinas y puede aprender que 23h y 0h son contiguas, algo imposible con el identificador crudo. Es la misma idea que explica por qué `hr` y `mnth` se excluyen del conjunto final de features: su versión cruda es redundante con las variables cíclicas ya construidas.

Data leakage. La fuga de información (leakage) ocurre cuando una feature contiene información que no estaría disponible en el momento real de predecir. Puede ser sutil: una ventana centrada usa horas futuras; un escalador ajustado sobre todo el dataset filtra estadísticos de test; una covariable "observada" en test no existe como pronóstico en producción; un componente algebraico del target —`casual` y `registered`, que suman exactamente `cnt`— delata el valor que se intenta predecir. El peligro del leakage es doble: infla las métricas de validación (el modelo parece mejor de lo que es) y, peor, es silencioso —la validación no avisa, simplemente miente—. Un modelo con fuga puede reportar un MASE brillante y fallar estrepitosamente en producción.

Embargo temporal. El embargo (gap) es una franja de tiempo entre el último dato de train y el primero de test que se excluye de ambos. Su función es cortar la dependencia de corto plazo en la frontera: sin embargo, una ventana móvil calculada sobre train puede usar observaciones a minutos de la frontera y el modelo puede "oler" el test por proximidad, sobre todo cuando el horizonte operacional coincide con el ciclo dominante. En este capítulo el embargo es de 24 horas, exactamente el ciclo diario: suficiente para que ninguna ventana de features del lado train comparta información del mismo día que el inicio de test. La regla operativa general es que el embargo debe ser mayor o igual que la dependencia temporal que el modelo explota en el horizonte de predicción.

Walk-forward validation. La validación walk-forward simula el despliegue: entrena con el pasado, pronostica el futuro, y repite desplazando la frontera hacia adelante. A diferencia del split único, mide si la ganancia de un modelo se mantiene cuando cambia el punto de corte —la pregunta que importa en producción, donde el modelo se reentrena periódicamente y el mundo no se detiene en una frontera fija—. Este capítulo usa tres folds de train **expansivo**: cada fold incorpora toda la historia disponible hasta su frontera, en vez de una ventana fija como en el rolling-origin del Capítulo 4. Cada fold reserva además su propio embargo de 24 horas. La métrica por fold se reporta como media ± desviación estándar, para que la lectura operativa distinga entre una mejora consistente y una que depende de un solo origen.

Gradient boosting. El gradient boosting construye un ensamble de árboles de forma secuencial: el primer árbol aprende a predecir el target; el segundo se ajusta sobre los errores que dejó el primero; el tercero, sobre los errores de los dos anteriores; y así sucesivamente. Cada árbol nuevo se especializa en lo que el conjunto aún no sabe hacer, reduciendo el sesgo paso a paso. Para que esa especialización no se convierta en memorizar el ruido de train, dos controles son esenciales: una tasa de aprendizaje baja (cada árbol aporta poco al conjunto) y árboles poco profundos (cada partición captura una pieza pequeña de la relación). El resultado es un modelo

flexible —capaz de representar no linealidades e interacciones como hora × día laborable sin que nadie las construya a mano— que sigue generalizando mejor que un modelo lineal cuando hay señal suficiente, como demuestran los resultados de la Sección 5.5 (MASE 0.457 para LightGBM frente a 0.818 para Ridge).

LightGBM en una frase. LightGBM es una implementación eficiente de gradient boosting: agrupa los datos en histogramas para acelerar el entrenamiento (de ahí el "Light") y crece los árboles por hoja en vez de por nivel, lo que le permite encontrar divisiones informativas con menos iteraciones. Para la operación, las dos propiedades que importan son que escala a datasets grandes —miles de filas y decenas de features como el de este capítulo se ajustan en segundos— y que acepta hiperparámetros de regularización (submuestreo de filas y columnas, profundidad máxima, número de árboles) que controlan el sobreajuste. La diferencia práctica con un modelo lineal es el tipo de relación que puede aprender: donde una regresión necesita que alguien construya la interacción hora × día laborable como feature explícita, un árbol la encuentra solo.

Interpretabilidad y comunicación. Los modelos de árboles entregan dos tipos de importancia de features: la nativa, que resume cuánto contribuyó cada feature a las divisiones del ensamble (gain), y la de permutación, que mide cuánto se degrada el error al desordenar los valores de una feature. Ambas responden preguntas distintas y ninguna demuestra causalidad: una feature puede aparecer como importante porque correlaciona con otra que sí es causal. El capítulo genera ambas visualizaciones en el notebook (Sección 5.5) sin exportar una tabla numérica; la lectura operativa se apoya en el orden de magnitud y en la coherencia entre ambos métodos, no en cifras puntuales.

Resumen operativo. El capítulo entero se apoya en una sola cadena de decisiones: convertir la serie en tabla supervisada con lags y ventanas causales; codificar el calendario de forma cíclica; validar con walk-forward de folds expansivos y embargo; y entrenar árboles con gradient boosting. Cada eslabón tiene una contraparte anti-leakage que lo vuelve seguro: los lags se construyen con `shift()`, las ventanas sobre el pasado, el escalador solo en train, y el test nunca participa en la selección.

5.2 Configuración y contrato de datos

El capítulo usa dos archivos: `data/04_bike/bike_hour.csv` (17,379×17, target `cnt` horario) y `data/04_bike/bike_day.csv` (731×16, `cnt` diario), ambos sin nulos ni duplicados. Dos verificaciones de integridad anteceden cualquier análisis:

- **Identidad algebraica.** `cnt = casual + registered` se cumple exactamente en las 17,379 filas horarias. `casual` y `registered` se excluyen de las features por construcción: son componentes directos del target, no predictores.
- **Consistencia hora–día.** La suma horaria de `cnt` por fecha coincide exactamente (diferencia 0 en las 731 fechas) con el `cnt` del archivo diario, confirmando que ambos archivos describen la misma realidad a distinta granularidad.

El índice temporal `ds` se construye como `dteday + hr`, único y monótono, del 2011-01-01 00:00 al 2012-12-31 23:00. Sobre una rejilla horaria completa de 17,544 horas, se observan 17,379 — 165 horas ausentes (0.94 %)—. El target tiene media 189.46 alquileres/hora, desviación estándar

181.39 (coeficiente de variación 0.96, la dispersión relativa más alta de los capítulos vistos hasta ahora), mediana 142, percentil 99 = 782.22, máximo 977.

5.3 Análisis previo (pre-modeling)

El perfil horario muestra un patrón bimodal en días laborables (picos alrededor de las 8h y las 17-18h, consistentes con desplazamientos casa-trabajo) que desaparece en días no laborables, donde el perfil es unimodal y centrado al mediodía. La demanda también varía por estación del año y muestra una relación positiva, aunque no estrictamente lineal, con la temperatura normalizada.

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Nivel	−6.94 (1.01e-9)	19.41 (0.010)	Ambos rechazan H0
Diferencia diaria (24h)	−22.53 (0.000)	0.014 (0.10)	Estacionaria por ambos tests

Tabla 24. Tests de estacionariedad sobre el nivel y la diferencia diaria (serie de inspección interpolada).

El estadístico KPSS del nivel (19.41) es el más extremo observado hasta ahora en la monografía: la serie combina una tendencia de crecimiento entre 2011 y 2012 —el sistema de bicicletas compartidas gana usuarios con el tiempo— con estacionalidad diaria fuerte, y ninguno de los dos fenómenos es capturado por un modelo constante. La diferencia diaria sí es estacionaria por ambos criterios, y su ACF/PACF motiva directamente el conjunto de lags usado en la Sección 5.4. La detección de outliers (MAD, $|z| > 4.5$) sobre el residuo STL marca 1,887 candidatos, y el CUSUM ubica su mayor desviación en 2012-03-31 09:00.

5.4 Estrategia de validación

Walk-forward con embargo de 24 horas, tres folds de train expansivo. El embargo impide que ventanas de features y targets cercanos a la frontera train/test compartan información del mismo ciclo diario.

Ingeniería de features, construida para no filtrar información:

- **Lags:** `lag_1, lag_2, lag_3, lag_6, lag_12, lag_24, lag_48, lag_168` —ocho rezagos del target, todos vía `shift()`, nunca el valor contemporáneo.
- **Ventanas móviles:** media, desviación, mínimo y máximo sobre ventanas de 24, 48 y 168 horas, calculadas siempre sobre `cnt.shift(1)` —es decir, sobre el pasado estrictamente anterior al origen de predicción, nunca sobre una ventana centrada—.
- **Codificación cíclica:** `hour_sin/cos`, `dow_sin/cos`, `month_sin/cos`.
- **Interacciones:** `temp_x_hour`, `holiday_x_season`.

El conjunto final tiene 37 features (excluyendo `cnt`, `instant`, `yr`, `mnth`, `hr` como identificadores o redundantes con las variables cíclicas). Tras eliminar filas sin historia completa de 168 horas, el dataset modelable tiene 11,840 observaciones, del 2011-04-04 05:00 al 2012-12-24 03:00.

Demostración cuantificada de leakage. El notebook compara explícitamente una media móvil de 24 horas **centrada** (que usa hasta 12 horas futuras) contra la versión seguridad basada en `shift(1)`:

Construcción	Correlación con `cnt`
Rolling centrada de 24h (con fuga)	0.400
Rolling pasada de 24h vía `shift(1)` (segura)	0.362

Tabla 25. Correlación con el target de una feature con fuga versus su versión segura.

La versión con fuga infla la correlación en 0.038 —un efecto modesto en magnitud pero sistemático—, precisamente porque incorpora información del futuro que no estará disponible en el momento real de la predicción. Es una demostración operacional del principio, no solo una advertencia textual.

```
Fold 1: train [0:10208] (expansivo) | embargo 24h [10208:10232] | test [10232:10400] (168h)
Fold 2: train [0:10928] (expansivo) | embargo 24h [10928:10952] | test [10952:11120] (168h)
Fold 3: train [0:11648] (expansivo) | embargo 24h [11648:11672] | test [11672:11840] (168h)
```

Figura 6. Estructura de los tres folds walk-forward con embargo.

A diferencia del rolling-origin de ventana fija del Capítulo 4, aquí el train es **expansivo**: cada fold incorpora toda la historia disponible hasta su frontera, no solo los últimos 180 días.

5.5 Modelado

> **Concepto clave — Tabularización y gradient boosting**
> Convertir una serie en una tabla supervisada (target `cnt`, features = rezagos y estadísticos móviles) reformula el forecasting como un problema de regresión estándar: se asume que la relación entre el pasado reciente y el valor futuro es lo bastante estable como para aprenderse como una función. Esa reformulación es exactamente lo que vuelve crítica la disciplina anti-leakage de este capítulo —si una feature contiene información posterior al origen de predicción, dejó de representar lo que estará disponible en producción, y la "función aprendida" deja de ser válida—.
> **Gradient boosting** (LightGBM, Random Forest en su variante de bagging) construye un ensamble de árboles de forma secuencial: cada árbol nuevo se ajusta sobre el error que dejaron los anteriores, reduciendo el sesgo del conjunto paso a paso, mientras una tasa de aprendizaje baja y árboles poco profundos controlan que ese ajuste no sobreajuste al ruido de train. A diferencia de un modelo lineal, cada partición de un árbol puede representar una relación no lineal o una interacción entre variables (por ejemplo, hora × día laborable) sin que nadie tenga que construir ese término de interacción a mano—de ahí buena parte de la ganancia frente a `Linear`/`Ridge` en la Tabla 26.

Methodology Card — Checklist anti-leakage
1) Lags construidos con `shift()`. 2) Ventanas móviles calculadas sobre el pasado, nunca centradas. 3) Escalador ajustado solo en train. 4) Embargo mayor o igual a la dependencia operacional del ciclo (24h). 5) Componentes algebraicos del target excluidos (`casual`, `registered`). 6) Solo covariables realmente disponibles en el horizonte futuro. 7) El test nunca participa en la selección de modelo o hiperparámetros.
Seed: 42.

$$\text{MAE} = \frac{1}{n}\sum|e_t|, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum e_t^2}, \quad \text{MASE} = \frac{\text{MAE}}{\text{mean}|y_t - y_{t-168}|}, \quad R^2 = 1 - \frac{\sum e_t^2}{\sum (y_t - \overline{y})^2}$$

Cuatro modelos, todos con `random_state=42`: `Linear` y `Ridge(alpha=10.0)` (ambos en pipeline con imputación por mediana y estandarización **ajustadas solo en train**), `RandomForestRegressor(n_estimators=120, max_depth=18, min_samples_leaf=3, max_features=0.7)` y `LGBMRegressor(n_estimators=250, learning_rate=0.05, num_leaves=31, subsample=0.8, colsample_bytree=0.8)`. El ajuste de los cuatro modelos en los tres folds tomó 20.2 segundos en total.

Modelo	MAE	RMSE	MASE	R^2	Tiempo de ajuste (s)
`LightGBM`	27.55 ± 6.01	42.84 ± 11.55	**0.457 ± 0.117**	0.957 ± 0.002	4.06 ± 1.93
`RandomForest`	28.44 ± 4.64	45.66 ± 9.69	0.471 ± 0.095	0.950 ± 0.004	2.51 ± 0.18
`Ridge`	49.54 ± 3.95	72.57 ± 10.23	0.818 ± 0.097	0.869 ± 0.031	0.053 ± 0.021
`Linear`	49.58 ± 3.96	72.59 ± 10.27	0.819 ± 0.097	0.869 ± 0.031	0.092 ± 0.027

Tabla 26. Resumen de métricas por modelo, promediadas sobre los tres folds (media ± desviación estándar).

Este es el primer capítulo de la monografía donde la ganancia de la familia más compleja es grande y consistente: `LightGBM` y `RandomForest` reducen el MASE en cerca de 44 % respecto a los modelos lineales, con R^2 por encima de 0.95 en ambos casos. La diferencia entre `LightGBM` y `RandomForest` es marginal (0.457 vs. 0.471 de MASE) frente a la brecha con los modelos lineales, que es sustancial y estable entre folds. El costo computacional es igualmente informativo: `Ridge` y `Linear` ajustan en un orden de magnitud menos tiempo que los modelos de árboles, un factor a considerar si la ganancia de precisión no justifica la complejidad operacional en un contexto de recursos acotados.

Decisión ejecutiva. El machine learning tabular es rentable cuando el feature engineering es riguroso y la validación temporal evita fugas.

Se calcularon importancia de features nativa de `LightGBM` (por **gain** / número de splits) e importancia por permutación sobre el test del último fold (5 repeticiones); ambas se generaron como visualizaciones en el notebook, sin una tabla numérica exportada de valores exactos, por lo que esta monografía no reproduce cifras específicas de importancia para evitar transcribir valores no verificables directamente. SHAP se intentó como análisis opcional pero la librería no

estaba disponible en el entorno de ejecución (`shap_available=False`); el notebook documenta explícitamente que no es una dependencia obligatoria para su reproducción.

5.6 Análisis posterior (post-modeling)

El diagnóstico se aplica a `LightGBM`, el modelo con menor MASE.

Alcance	Test	Estadístico	p-valor	Decisión (α=0.05)
Fold 1	Ljung-Box (24)	7.88	0.999	**no rechazar H0**
Fold 2	Ljung-Box (24)	104.74	4.62e-12	rechazar H0
Fold 3	Ljung-Box (24)	166.74	2.42e-23	rechazar H0
Agregado	Jarque-Bera	5,079.39	0.000	rechazar H0
Agregado	ARCH LM (24)	25.07	0.402	**no rechazar H0**

Tabla 27. Diagnóstico de residuos de LightGBM, por fold (Ljung-Box) y agregado (Jarque-Bera, ARCH LM).

Dos resultados de esta tabla son atípicos frente al resto de la monografía. Primero, el fold 1 es el único caso, entre los cinco capítulos técnicos vistos hasta ahora, donde Ljung-Box **no** rechaza la hipótesis de ausencia de autocorrelación —aunque los folds 2 y 3 sí la rechazan, de modo que la conclusión agregada del capítulo sigue siendo que persiste estructura residual, solo que de forma menos uniforme que en los capítulos estadísticos—. Segundo, ARCH LM tampoco rechaza homocedasticidad condicional a nivel agregado, la primera vez en la monografía que este test no encuentra evidencia de heterocedasticidad. Jarque-Bera sí rechaza normalidad con fuerza (asimetría −1.55, curtosis 18.24 —cola izquierda pesada, es decir, errores grandes de sub-predicción más frecuentes que de sobre-predicción—).

Segmento	MAE medio	Mediana	n
Pico (7-9h, 16-19h)	43.67	35.46	148
Valle / resto	20.85	11.69	356

Tabla 28. Error absoluto segmentado por franja horaria.

El error en hora punta duplica al del resto del día, un patrón consistente con los Capítulos 3 y 4: el modelo falla más donde la magnitud —y, presumiblemente, la relevancia operacional— es mayor. Dos segmentaciones adicionales previstas en el checklist del curso, por festivo y por estación, no produjeron una comparación válida en este capítulo: los tres bloques de test caen enteramente en periodos **no festivos** (el grupo "festivo" queda vacío) y abarcan solo las estaciones 1 y 4 de las cuatro codificadas —las estaciones 2 y 3 no aparecen en ninguno de los tres folds de test—. Se documenta la limitación en vez de forzar una comparación con datos insuficientes.

Elección de granularidad. El notebook cierra esta sección comparando explícitamente hora contra día: 17,379 filas con CV=0.96 (soporta patrones intradiarios) frente a 731 filas con CV=0.43 (no los soporta). La recomendación operacional es clara: un forecast de disponibilidad de bicicletas en tiempo real requiere granularidad horaria; la planificación agregada de flota puede trabajar con granularidad diaria.

5.7 Lecciones metodológicas

1. `shift(1)` antes de cualquier ventana móvil es obligatorio, no opcional: la Sección 5.4 cuantifica el efecto de omitirlo en 0.038 puntos de correlación inflada.
2. `casual` y `registered` reconstruyen exactamente el target; usarlas como features es fuga directa, no un matiz de diseño.
3. El embargo de 24 horas separa el ciclo diario entre train y test; sin él, ventanas de ambos lados de la frontera compartirían información del mismo día.
4. La importancia de features no implica causalidad, y la importancia por permutación depende de qué conjunto de test se use y de la correlación entre features —dos features redundantes pueden repartirse la importancia real entre ambas—.
5. La granularidad horaria sirve a la operación; la diaria, a la planificación agregada. Ninguna reemplaza a la otra.

Cuenta regresiva de no normalidad: Caps. 2, 3 y 4 ya rechazaron la normalidad de residuos; este capítulo añade el mecanismo ARCH explícito (heterocedasticidad condicional). La cuenta continúa en los Caps. 6, 7 y 8 y se cierra en el §9.6 con el conteo 8/8 de Jarque-Bera rechazado — la regularidad empírica que especifica el sistema probabilístico del Cap. 9.

Conexión con el Capítulo 6. El Capítulo 6 traslada el paradigma tabular de este capítulo a cientos de series simultáneas mediante un modelo global, y contrasta explícitamente el aprendizaje compartido entre series contra el aprendizaje local que domina este capítulo y los anteriores.

Verificación con el sílabo. Las competencias de la Semana 4 quedan materializadas en: la demostración cuantificada de leakage (Sección 5.4, Tabla 25), la validación walk-forward con embargo (Figura 6), el contrato de integridad del dataset (Sección 5.2) y el checklist anti-leakage de siete puntos que distingue este capítulo. Estas competencias se listan de forma consolidada en Tabla 5.

5.8 Deudas abiertas del experimento

- **Fácil.** Retirar `lag_168` del conjunto de features y cuantificar la degradación de MASE, para aislar el valor específico de la memoria semanal.
- **Intermedio.** Comparar pronóstico recursivo (donde cada predicción alimenta el siguiente paso) contra el enfoque directo multi-step usado en este capítulo.
- **Difícil.** Ejecutar SHAP agrupando features correlacionadas (por ejemplo, las cuatro estadísticas de una misma ventana móvil) para obtener una atribución de importancia más estable que la importancia individual por feature.

CAPÍTULO 6

Semana 5: Modelos globales y cross-learning

Dataset: Electricity Load Diagrams · **Notebook:** `05_semana_5_modelos_globales_cross_learning.ipynb` · **Fuente:** UCI Machine Learning Repository, licencia CC BY 4.0.

Ficha de caso — Semana 5
Dominio: energía.
Problema de negocio: predecir el consumo de clientes industriales nuevos.
Usuario: planificación comercial, capacidad o atención de clientes.
Decisión que habilita: dimensionar capacidad, ofertas o recursos.
Horizonte: 168 horas.
Baseline obligatorio: perfil hora×día.
Riesgo principal: creer que un modelo global resuelve cold-start.
Métrica técnica: MASE macro.
Métrica de negocio: capacidad estimada para clientes nuevos.

6.1 Fundamentos teóricos aplicados

Cuatro distinciones ordenan la lectura de este capítulo: qué distingue un modelo local de uno global, qué promete —y qué no— el cross-learning, por qué la heterogeneidad de escala cambia el problema, y las tres variantes del arranque de una serie nueva —cold-start, warm-start y few-shot adaptation—, explicadas en términos accionables para un profesional de industria.

Modelos locales. Un modelo local se ajusta sobre la historia de una sola serie: los ARIMA/SARIMA de los Capítulos 3 y 4 y el LightGBM con lags del Capítulo 5 son locales. Su ventaja es la adaptación total: los parámetros se estiman para capturar la dinámica específica de esa serie, incluida su escala. Su requisito es igualmente total: necesitan historia suficiente de esa misma serie. Si la serie tiene dos años de datos, el modelo local es la opción natural; si la serie no existe todavía, no hay nada que ajustar.

Modelos globales. Un modelo global entrena una sola función sobre muchas series a la vez: cada observación de cada serie contribuye a la misma estimación, y la función resultante mapea features a target para cualquier serie —conocida o nueva—. La ganancia es estadística: al compartir datos entre series, el modelo estima el patrón común con más precisión de la que lograría cualquier serie por separado, y puede pronosticar series que nunca vio. El costo es el inverso de

la ventaja: una sola función no puede representar la idiosincrasia de cada serie individual, y esa limitación se vuelve crítica cuando las series difieren en algo tan fundamental como la escala.

Cross-learning. El cross-learning es el mecanismo por el que un modelo global aprovecha la estructura compartida entre series: las 296 series de entrenamiento de este capítulo enseñan, juntas, cuál es la forma típica del consumo eléctrico —qué hora del día consume más, qué día de la semana, cómo evoluciona el mes—. Lo que se transfiere es la **forma** del patrón; lo que no se transfiere es la **magnitud** individual, porque la magnitud depende de características del cliente que no están en las features. El cross-learning es la razón por la que un modelo global puede hacer un pronóstico razonable de un cliente nuevo: no porque lo conozca, sino porque conoce el patrón de la población a la que pertenece.

Transferencia entre series. La transferencia entre series es la capacidad de aplicar lo aprendido en unas series a otras distintas. Es una idea poderosa —el Capítulo 8 la lleva al extremo con sensores de tráfico nunca vistos—, pero su alcance está condicionado por la **distancia** entre las series de entrenamiento y las de aplicación: si comparten la forma del patrón, la transferencia funciona; si difieren en escala o en régimen, la transferencia transfiere el patrón promedio, no la magnitud correcta. Este capítulo es la demostración más limpia de ese límite: el modelo global transfiere perfectamente el perfil hora×día y falla por completo en recuperar la escala de clientes de consumo extremo.

Sesgo-varianza. El intercambio sesgo-varianza explica por qué agrupar series no es gratis. Un modelo con más parámetros o más datos reduce la varianza de la estimación —el error que viene de la escasez de datos—, pero puede aumentar el sesgo —el error que viene de suponer una forma que no es exacta—. Al entrenar una sola función sobre 296 series heterogéneas, el modelo global reduce la varianza (tiene muchísimos datos) a costa de sesgo (una función no puede representar a la vez a un cliente de consumo medio y a otro 43,000 veces mayor). La lección operativa es que la decisión local vs. global no es una preferencia de arquitectura: es un punto de equilibrio entre ambos errores, y el equilibrio depende de la heterogeneidad del panel.

Heterogeneidad de escala. La escala es el nivel absoluto de la serie: un cliente industrial puede consumir 190,000 unidades por hora y otro 4. La heterogeneidad de escala —la dispersión de esos niveles entre series— es el desafío central de este capítulo: una razón de más de 43,000 entre el cliente más pequeño y el más grande significa que cualquier modelo que no conozca la escala individual de un cliente nuevo está adivinando un número en un rango de cinco órdenes de magnitud. La escala no es un problema de cómputo ni de tuning; es un problema de información: si la escala del cliente nuevo no está en las features, ningún algoritmo puede recuperarla.

Cold-start. El cold-start es el escenario en que una entidad no tiene historia del target: el cliente es nuevo, el sensor acaba de instalarse, el producto no se ha vendido. El modelo debe pronosticar sin haber visto una sola observación de esa serie. La única información disponible es la que no depende de la historia: el calendario y, si existen, metadatos estáticos del cliente —sector, tamaño, tipo de contrato—. Este capítulo opera en cold-start **estricto**: sin metadatos, solo calendario. La métrica resultante es la que cualquier negocio debe esperar si pide un pronóstico para un cliente completamente nuevo: el error de adivinar una escala individual a partir del promedio de la población.

Warm-start. El warm-start es el escenario opuesto: la entidad tiene historia suficiente para ajustar un modelo local o para alimentar rezagos y ventanas móviles. No es que el modelo local sea intrínsecamente mejor que el global; es que, con historia, el modelo local puede capturar la escala y la forma específica de la serie, que es exactamente la información que el cold-start no puede proveer. La comparación del capítulo —MASE mediano 1.34 con historia frente a 38.59 sin ella— cuantifica el valor de esa información: no es una mejora porcentual, es un cambio de régimen.

Few-shot adaptation. Entre el extremo de cero historia y el de historia completa existe un continuo. La few-shot adaptation usa una ventana corta de contexto —en la política de producción de este capítulo, una semana— para adaptar el pronóstico inicial: el modelo global aporta la forma del patrón y la ventana reciente aporta la escala, por ejemplo mediante un factor de ajuste calculado sobre los errores recientes o un reentrenamiento con pesos iniciales del modelo global. No es cold-start (hay algo de historia) ni warm-start pleno (la historia no alcanza para un modelo local estable), pero es el escenario más realista en la operación: pocos clientes llegan sin absolutamente nada de información.

Disponibilidad de historia por entidad. La pregunta que organiza todo el capítulo es una sola: ¿cuánta historia del target tiene cada entidad en el momento de pronosticar? La respuesta clasifica el problema —cold-start, few-shot, warm-start— y determina qué features y qué familia de modelos son siquiera posibles: sin historia, los lags y las ventanas móviles no existen, y el modelo queda reducido al calendario. La lección transversal de la monografía —y de este capítulo en particular— es que la disponibilidad de historia domina sobre la elección de arquitectura: ninguna decisión de modelo compensa la ausencia de la información que el modelo necesitaría.

Resumen operativo. La cadena de decisiones del capítulo es: clasificar cada entidad por su historia disponible; si no hay historia, usar solo calendario y comparar contra un baseline transparente (perfil hora×día); si hay una ventana corta, adaptar (few-shot); si hay historia completa, entrenar local con rezagos o usar el global según el volumen del panel. La métrica macro —promedio simple entre series, no ponderado por volumen— evita que los clientes de mayor escala dominen la evaluación. Y la lectura ejecutiva es directa: el modelo global no es una máquina de resolver cold-start; es una herramienta para explotar estructura compartida cuando la información por entidad lo permite.

6.2 Configuración y contrato de datos

El archivo completo `data/05_electricity/electricity_long.csv` tiene 9,732,480 filas y 3 columnas (`ds`, `unique_id`, `y`) en formato **long**, 403.2 MiB en disco —una estimación conservadora sitúa el uso de RAM en pandas entre 806 y 1,613 MiB, varias veces el tamaño en disco—. Se lee por **chunks** de 500,000 filas: sobre cada chunk se acumula un perfil estadístico por cliente (recorriendo así las 9,732,480 filas completas), pero solo se retiene en memoria una ventana de los últimos 120 días —una decisión temporal explícita para viabilidad en CPU, no un submuestreo aleatorio—. La ventana resultante tiene 1,065,600 filas, del 2014-09-03 al 2014-12-31, con 2,880 timestamps únicos por serie (120 días × 24 h, exactamente, confirmando ausencia de gaps dentro de la ventana) y 370 series sin duplicados de clave (`unique_id`, `ds`).

Heterogeneidad de escala. La media del consumo medio entre las 370 series es 2,348.52, pero el rango individual va de 4.36 a 190,542.4 —una razón de más de 43,000 entre el cliente más pequeño y el más grande—. Esta heterogeneidad de escala, no la estacionalidad ni la tendencia, es el desafío central que organiza todo el capítulo.

6.3 Análisis previo (pre-modeling)

Ordenando los 370 clientes por coeficiente de variación, las series más volátiles (`MT_003`: CV=3.15, `MT_178`: 2.53, `MT_093`: 2.28) contrastan con las más estables (`MT_148`: 0.077, `MT_145`: 0.080, `MT_149`: 0.090). Un **clustering** K-means ($k = 4$) sobre los perfiles horarios normalizados y estandarizados agrupa a los clientes por forma de consumo diario:

Cluster	Clientes
0	213
1	119
2	34
3	4

Tabla 29. Resultado del clustering K-means ($k = 4$) sobre perfiles horarios.

El **silhouette score** obtenido es 0.400 —una separación moderada, no excelente—, y la distribución altamente desigual entre clusters (213 contra 4) anticipa que las categorías de forma de consumo no están uniformemente representadas en la población de clientes.

Sobre la serie agregada del panel (media cross-sectional de las 370 series por timestamp), una descomposición STL semanal (periodo 168) y los tests de estacionariedad producen un resultado distinto al de los capítulos anteriores:

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Agregado (nivel)	−2.73 (0.068)	7.52 (0.010)	**Ambos apuntan a no estacionariedad**
Diferencia semanal	−5.09 (0.000015)	0.221 (0.10)	Estacionaria por ambos tests

Tabla 30. Tests de estacionariedad sobre la serie agregada del panel.

Es la única vez en esta monografía en que ADF **no rechaza** la hipótesis de raíz unitaria: a diferencia de Metro Traffic o de la carga alemana del Capítulo 7, el agregado de 370 clientes heterogéneos no se comporta como una serie estacionaria ni siquiera bajo el criterio más permisivo. La detección de outliers sobre el residuo STL (MAD, $|z| > 4.5$) marca 338 candidatos.

6.4 Estrategia de validación

Cold-start estricto por serie: 296 clientes entrenan, 74 quedan completamente fuera. La partición se hace sobre identificadores de cliente, con semilla 42, sin solapamiento —no es un split temporal clásico, es un split de **transferencia**: mide si lo aprendido en 296 series generaliza a 74 series nunca vistas—.

Contrato de features por escenario. Para los 74 clientes de cold-start, el modelo global solo puede usar **features de calendario**, disponibles para cualquier serie sin importar su historia: `hour_sin/cos`, `dow_sin/cos`, `month`, `trend`. Quedan explícitamente prohibidos, por definición del escenario: rezagos del target, **rolling windows** y **target encoding** por identificador de cliente —ninguno de estos existe para un cliente que el modelo nunca observó—. Como contraste, la Sección 6.5 construye también un experimento **warm-start** local sobre 12 series con historia completa, usando lags (1, 24, 168) y medias móviles, precisamente para exhibir la diferencia entre ambos regímenes.

6.5 Modelado

Concepto clave — Por qué "global" no es automáticamente "mejor"
Un modelo global agrupa muchas series y aprende una sola función, compartida, que mapea features a target. Es un intercambio sesgo-varianza: agrupar reduce la varianza de la estimación —hay más datos para aprender el patrón común—, pero introduce sesgo si las series son heterogéneas, porque esa única función no puede representar perfectamente la idiosincrasia de cada serie individual, como la enorme heterogeneidad de escala entre clientes documentada en la Sección 6.2.
En cold-start estricto, el modelo no tiene ningún mecanismo para diferenciar una serie de otra más allá del calendario: sin identificador de cliente ni rezagos, dos clientes con el mismo día y hora reciben, necesariamente, la misma predicción. No es que el algoritmo esté mal ajustado—es que la información que necesitaría para distinguirlos no está codificada en ninguna de sus features de entrada. Por eso empata con un baseline que hace exactamente eso, de forma explícita: promediar por día y hora.

Methodology Card — Global vs. local
Un modelo global comparte parámetros y escala a muchas series; uno local adapta forma individual. El cold-start estricto prohíbe lags, identificador y target encoding del cliente nuevo. Si existe una ventana de contexto, el problema cambia a **few-shot adaptation**, no a cold-start puro. Comparar siempre contra un baseline global transparente.
Seed: 42.

$$\mathrm{MASE}_i = \frac{\mathrm{MAE}_i}{\mathrm{mean}\left|y_{i,t} - y_{i,t-168}\right|}, \quad \mathrm{WAPE} = 100 \times \frac{\sum_{i,t}|e_{i,t}|}{\sum_{i,t}|y_{i,t}|}$$

La métrica técnica del capítulo agrega ese valor entre series como promedio simple, no ponderado por volumen:

$$\mathrm{MASE}_{\mathrm{macro}} = \frac{1}{N}\sum_{i=1}^{N}\mathrm{MASE}_i$$

donde N es el número de series evaluadas. Esta elección importa: un promedio ponderado por volumen haría que los clientes de mayor escala dominaran la evaluación, ocultando el desempeño sobre el cliente típico.

El baseline obligatorio calcula, para cada combinación de día de semana d y hora h, la media del consumo en train sobre los 296 clientes conocidos:

$$\text{perfil}(d, h) = \frac{1}{|T_{d,h}|} \sum_{t \in T_{d,h}} y_t$$

donde $T_{d,h}$ es el conjunto de timestamps de train que caen en ese día y hora. Es exactamente la función que el LightGBM global termina replicando en la Tabla 31.

Modelo	MAE	WAPE (%)	MASE macro
Perfil hora×día (media de train por día×hora)	3,178.72	115.41	69.44
LightGBM global (solo calendario, n=250 árboles)	3,178.39	115.40	69.44

Tabla 31. Comparación cold-start: baseline transparente vs. LightGBM global, evaluados sobre los 74 clientes nunca vistos.

Este es el resultado más contundente de la monografía hasta este capítulo: el modelo de machine learning entrenado sobre 250 árboles y 57 segundos de cómputo produce una mejora de 0.33 en MAE —un 0.01 %— sobre un baseline que simplemente promedia el consumo de los 296 clientes conocidos por día de la semana y hora. El modelo global no falla por estar mal ajustado; falla porque el problema tal como está planteado —predecir la escala de un cliente sin ninguna información sobre ese cliente— no es resoluble con las features disponibles, y ningún algoritmo de aprendizaje puede recuperar información que no está en los datos de entrada.

> **Decisión ejecutiva.** No se recomienda usar un modelo global para clientes completamente nuevos sin historia. El perfil hora×día es la política más robusta en cold-start estricto.

Contraste con warm-start local. Sobre 12 series representativas con historia completa, evaluando las últimas 168 horas como holdout, un LightGBM local con lags y rolling windows obtiene una mediana de MASE de 1.34 (rango intercuartílico 1.00–1.52).

Escenario	Series evaluadas	Historia del target disponible	MASE mediano
Local warm-start	12	Sí	1.34
Global cold-start estricto	74	No	38.59

Tabla 32. Cross-learning interpretado: comparación de régimen de disponibilidad de historia.

La brecha entre ambos escenarios —de 1.34 a 38.59 en MASE mediano, casi 29 veces— es la lección central del capítulo: la disponibilidad de historia del target domina por completo la elección entre modelo local y modelo global, muy por encima de cualquier decisión de arquitectura o hiperparámetros.

Methodology Card — Política de pronóstico para clientes nuevos
Sin historia: perfil hora×día (cold-start estricto).
Con una semana de historia: adaptación few-shot sobre la forma del modelo global.
Con cuatro o más semanas: modelo local/global con rezagos y ventanas móviles.
Clientes de alta escala: exigir metadatos o contexto adicional; sin ellos, el error estará dominado por la escala.

6.6 Análisis posterior (post-modeling)

El diagnóstico se aplica al residuo agregado (error medio por timestamp) del modelo global cold-start.

Test	H0	Estadístico	p-valor	Decisión
Ljung-Box (24)	sin autocorrelación	12,569.49	0.000	rechazar H0
Ljung-Box (168)	sin autocorrelación	66,404.05	0.000	rechazar H0
Jarque-Bera	normalidad	4.78e7	0.000	rechazar H0
ARCH LM (24)	homocedasticidad	2,434.53	0.000	rechazar H0

Tabla 33. Diagnóstico de residuos del modelo global cold-start.

Los cuatro tests rechazan; el residuo agregado conserva estructura diaria y semanal completa, coherente con que el modelo simplemente no está resolviendo el problema de escala individual.

Cluster	MASE medio	MASE mediano	n (clientes de test)
0	52.37	26.58	42
1	112.60	66.78	25
2	14.25	6.30	6
3	38.29	38.29	1

Tabla 34. Error por cluster de forma de consumo (aplicado a los 74 clientes de test).

El error varía casi 8 veces entre el cluster más favorable (2) y el más adverso (1), lo que sugiere que la forma de consumo diaria sí influye en cuánto puede "transferir" el modelo global, incluso cuando la escala absoluta no se recupera en ningún cluster. El cluster 3 tiene un solo cliente de test, por lo que su cifra no debe leerse como representativa.

El sesgo por serie individual expone la magnitud del problema de escala: el mayor sub-pronóstico corresponde a `MT_196`, con un sesgo medio de +71,399.59 —el modelo subestima sistemáticamente a un cliente de consumo muy alto que nunca vio durante el entrenamiento—, mientras que el mayor sobre-pronóstico (`MT_007`, −2,310.43) es un orden de magnitud menor.

6.7 Lecciones metodológicas

1. El **chunking** permite auditar 9.7 millones de filas sin cargar el archivo completo en memoria de una vez.

2. El cold-start estricto excluye, por definición y no por elección de diseño, lags, identificador de cliente y **target encoding** del cliente nuevo.
3. Un modelo global puede transferir la **forma** del calendario —el perfil hora×día—, pero no la **escala** individual sin metadatos o una ventana de contexto del cliente nuevo.
4. Los modelos locales son fuertes cuando hay historia disponible, pero simplemente no existen como opción para clientes sin historia.
5. Las métricas macro (promedio simple entre series, no ponderado por volumen) evitan que los clientes de mayor escala dominen artificialmente la evaluación agregada.

Conexión con el Capítulo 7. El Capítulo 7 abandona el problema de panel multi-cliente y vuelve a una sola serie, pero introduce tensores secuenciales y una red LSTM, contrastada explícitamente contra los mismos baselines tabulares que dominaron este capítulo.

Verificación con el sílabo. Las competencias de la Semana 5 quedan materializadas en: el experimento de cold-start estricto (Sección 6.5, Tabla 31), el contraste warm-start vs. cold-start (Tabla 32), la estrategia de validación por serie (Sección 6.4) y el diagnóstico de residuos del modelo global (Tabla 33). Referencia consolidada en Tabla 5.

6.8 Deudas abiertas del experimento

- **Fácil.** Añadir una semana de contexto histórico al cliente nuevo (few-shot en vez de cold-start estricto) y medir cuánto se recupera del MASE macro.
- **Intermedio.** Comparar $k = 3$ a $k = 8$ clusters mediante silhouette y evaluar si un k distinto mejora la interpretación del error por cluster de la Tabla 34.
- **Difícil.** Convertir el CSV a formato Parquet particionado por cliente y medir el impacto en tiempo de lectura frente a la estrategia de chunking usada aquí.

CAPÍTULO 7

Semana 6: Deep Learning con LSTM

Dataset: OPSD Germany (Open Power System Data) · **Notebook:** `06_semana_6_deep_learning_lstm.ipynb` · **Fuente:** Open Power System Data, términos de atribución OPSD (no UCI).

> **Ficha de caso — Semana 6**
> **Dominio:** energía.
> **Problema de negocio:** predecir carga eléctrica nacional.
> **Usuario:** despacho energético, planificación o mesa de operaciones.
> **Decisión que habilita:** despacho, capacidad o gestión de renovables.
> **Horizonte:** 168 horas.
> **Baseline obligatorio:** Seasonal Naive 168.
> **Riesgo principal:** usar LSTM por defecto.
> **Métrica técnica:** MASE.
> **Métrica de negocio:** error en MW.

7.1 Fundamentos teóricos aplicados

Antes de entrar al modelado conviene aclarar qué es una red recurrente, por qué el gradiente se desvanece, qué agregan las compuertas de una LSTM y qué costo operativo implica todo ello — siempre pensando en un profesional de industria que decide si adopta deep learning.

Redes recurrentes (RNN). Una red neuronal recurrente procesa una secuencia paso a paso y mantiene una memoria interna —el **estado oculto**— que se actualiza con cada observación nueva. A diferencia de un modelo tabular, que recibe una fila de features ya construidas (rezagos, ventanas móviles, calendario), la RNN recibe los valores en orden y decide internamente qué retener de cada paso: la ingeniería de features se traslada, en parte, al interior del modelo. En forecasting, la salida de la red se construye a partir del último estado oculto mediante una **cabeza lineal** —una capa final sin recurrencia—, que es exactamente la arquitectura compacta que este capítulo entrena. Esa propiedad es atractiva en teoría, pero también es una hipótesis fuerte: supone que el patrón relevante es aprendible a partir de la secuencia cruda con los datos disponibles, sin que nadie se lo indique explícitamente. Si el problema ya está bien resuelto con rezagos explícitos, la recurrencia no tiene nada adicional que explotar.

El problema del gradiente que se desvanece. Entrenar una red es ajustar sus pesos para reducir el error, y ese ajuste se propaga hacia atrás por la red mediante el gradiente. En una

red recurrente, el gradiente de un paso depende del gradiente del paso siguiente a lo largo de toda la secuencia: el error de una predicción hecha en la hora actual debe viajar hacia atrás por cada una de las horas de contexto para actualizar los pesos de los pasos lejanos. Al multiplicarse repetidamente, ese gradiente tiende a hacerse infinitesimal —se **desvanece**— o a crecer sin control —**explota**—. Cuando se desvanece, la red deja de aprender de los pasos lejanos en la práctica: la arquitectura existe, pero el entrenamiento no logra actualizar los pesos relevantes. Es la razón teórica por la que una RNN simple no es competitiva con secuencias largas como una semana completa de carga eléctrica horaria (168 pasos).

LSTM y sus compuertas. La Long Short-Term Memory (LSTM) fue diseñada precisamente para mitigar el desvanecimiento del gradiente. Su idea central es separar la memoria en dos carriles: el **estado de celda**, que transporta información a lo largo de la secuencia con un flujo aditivo —sumar y restar, no solo multiplicar—, y el estado oculto, que se expone a la salida de cada paso. Tres mecanismos, llamados **compuertas**, regulan ese flujo: la **compuerta de olvido** decide qué parte de la memoria pasada se descarta; la **compuerta de entrada** decide qué información nueva del paso actual se incorpora a la memoria; y la **compuerta de salida** decide qué parte de la memoria se proyecta hacia la salida y hacia el siguiente paso. Cada compuerta es, en el fondo, una pequeña capa con activación sigmoide que emite valores entre 0 y 1 —"olvidar todo" o "conservar todo"—. El resultado es una red que, en principio, puede retener información durante cientos de pasos. Ese "en principio" importa: la arquitectura **permite** memorias largas, pero solo las **aprovecha** si el problema las necesita y si hay datos suficientes para aprenderlas. La combinación de compuertas no es magia: sigue siendo un modelo entrenado con gradientes, y sigue siendo vulnerable al sobreajuste si los datos son escasos.

GRU. La Gated Recurrent Unit (GRU) es una simplificación de la LSTM: fusiona el estado de celda y el estado oculto en uno solo y reduce las compuertas a dos (actualización y reinicio). Es más barata de entrenar —menos parámetros— y en muchos problemas empíricos alcanza resultados comparables a la LSTM con menos recursos; por eso suele ser la primera alternativa a probar cuando una LSTM no justifica su costo. La deuda abierta de comparar GRU y LSTM a igual número de parámetros se formaliza en la Sección 7.8.

Tensores 3D. La diferencia operativa más concreta entre el deep learning secuencial y los modelos tabulares de los capítulos anteriores es el formato de los datos. Un modelo tabular recibe una matriz de dos dimensiones: muestras × features, donde cada feature ya es un número resumido. Una LSTM recibe un **tensor tridimensional**: muestras × pasos de tiempo × features —en este capítulo, (muestras, 168, 6)—. Construir correctamente esa forma —cortar la serie en ventanas de 168 horas con paso 3, preservando el orden temporal y sin mezclar folds— es buena parte del trabajo de ingeniería de este capítulo, y es un costo que no existe en LightGBM ni en el Seasonal Naive.

Normalización. Las redes neuronales son sensibles a la escala de sus entradas: si una feature está en miles de MW y otra en décimas, el gradiente queda dominado por la de mayor magnitud. La normalización —en este capítulo, un `StandardScaler`— lleva cada variable a una escala comparable. El detalle metodológico crítico es que el escalador se ajusta **solo con los datos de train de cada fold**, nunca con el fold completo ni con el test: usar estadísticas del test para

escalar filtraría información futura y sobreestimaría el desempeño real. Es el mismo principio de embargo temporal de los Capítulos 4 y 5, ahora aplicado dentro del propio pipeline de la red.

Early stopping y regularización. Una red con suficientes parámetros puede memorizar el train en vez de aprender el patrón general —el sobreajuste en su forma más costosa—. Dos herramientas lo controlan en este capítulo. El **early stopping** detiene el entrenamiento cuando el error de validación interna deja de mejorar (aquí, tras una paciencia de épocas sin mejora), en lugar de fijar a ciegas el número de épocas. El **scheduler** de tasa de aprendizaje (`ReduceLROnPlateau`) reduce la tasa cuando la mejora se estanca, para afinar el descenso sin saltarse el mínimo. Ambas reducen el sobreajuste, pero no deben confundirse con una garantía de buen desempeño: controlan **cómo** se entrena la red, no **si** la red era necesaria.

Costo computacional. Entrenar una LSTM es cualitativamente distinto de ajustar un modelo tabular: requiere construir ventanas secuenciales, definir épocas y paciencias, mantener un split interno de validación, probar hiperparámetros (tamaño de capa, tasa de aprendizaje, lookback, stride) y, en general, un ciclo de experimentación más largo. Cada época recorre la secuencia completa hacia adelante y hacia atrás (**backpropagation through time**), por lo que el costo crece con la longitud del contexto. En este capítulo ese costo es medible y modesto en términos absolutos —23.3 segundos en total para los cuatro modelos en los tres folds, con la mayor parte consumida por la LSTM—, pero la operación real añade costos que un benchmark de laboratorio no captura: más hiperparámetros que vigilar, más inestabilidad entre corridas y más esfuerzo de mantenimiento cuando los datos cambian.

Cuándo usar y cuándo no usar una LSTM. La regla operativa que este capítulo ilustra es directa. **Sí considerar** una LSTM cuando hay mucha historia (años de datos horarios), múltiples covariables, dependencias no lineales de largo alcance que un modelo tabular no captura con rezagos y ventanas, y presupuesto para entrenar y mantener una infraestructura más compleja. **No usarla por defecto** cuando el seasonal naive o un modelo tabular con buen feature engineering ya resuelven el problema: aquí LightGBM logra un MASE de 0.234 frente a 0.576 de la LSTM, y la propia LSTM queda por debajo del Seasonal Naive (0.495). La complejidad debe justificarse con mejora medible bajo el mismo protocolo de validación, no con el prestigio de la arquitectura. En términos de volumen, las redes neuronales son exigentes: con menos de unos miles de muestras, su ventaja teórica rara vez se materializa, y la comparación contra un baseline se vuelve un requisito, no una formalidad.

7.2 Configuración y contrato de datos

El archivo `data/06_opsd/germany_energy.csv` tiene 50,401 filas y 43 columnas horarias, con rango completo del 2014-12-31 23:00 al 2020-09-30 23:00 UTC sobre una rejilla sin gaps. El target es `DE_load_actual_entsoe_transparency` (carga eléctrica nacional alemana, en MW). De las 43 columnas se seleccionan 6 —el target más cinco covariables de cobertura nacional alta: `DE_load_forecast_entsoe_transparency`, `DE_solar_generation_actual`, `DE_wind_generation_actual`, `DE_wind_offshore_generation_actual`, `DE_wind_onshore_generation_actual`—; el precio día-adelanto (`DE_LU_price_day_ahead`) se descarta explícitamente por cobertura corta (17,540 de 50,401 valores no nulos, apenas 35 %).

El arco oracle recorre tres capítulos: la Tabla 11 formalizó la pregunta de disponibilidad futura, el Cap. 4 cuantificó el precio del oracle (18.36 % bajo clima observado), y en este capítulo el caso es afortunado: `DE_load_forecast` está legítimamente disponible ex-ante. La pregunta de disponibilidad no es un filtro previo — es una dimensión de la auditoría permanente. **Periodo de laboratorio.** Se retienen los últimos 730 días (17,520 horas) con cobertura estable en las 6 columnas seleccionadas —un recorte temporal deliberado, del 2018-10-02 al 2020-09-30, no un muestreo aleatorio—, dentro del cual no hay valores nulos en ninguna de las 6 variables. El target tiene media 55,143.54 MW, desviación estándar 9,987.79, rango de 31,923 a 76,925 MW.

7.3 Análisis previo (pre-modeling)

El perfil horario muestra el patrón esperado de demanda eléctrica nacional: picos en horario laboral, valle nocturno pronunciado. El perfil semanal separa claramente días laborables de fin de semana. Una descomposición STL semanal (periodo 168) sobre la ventana de laboratorio antecede a los tests de estacionariedad:

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Nivel	−14.06 (3.11e-26)	4.59 (0.010)	Ambos rechazan H0
Diferencia semanal	−11.07 (4.68e-20)	0.041 (0.10)	Estacionaria por ambos tests

Tabla 35. Tests de estacionariedad sobre el nivel y la diferencia semanal.

Como en Metro Traffic (Capítulo 2), el nivel rechaza ambas hipótesis nulas simultáneamente —señal de estacionalidad fuerte no capturada por ninguno de los dos tests por separado—, y la diferencia semanal resuelve la ambigüedad. La detección de outliers (MAD, $|z| > 4.5$) marca 1,272 candidatos sobre el residuo STL.

7.4 Estrategia de validación

Ventanas móviles: tres folds, cada uno entrena con los 365 días previos (8,760 horas) y valida los 7 días siguientes (168 horas), con orígenes separados 7 días entre sí —simulando un ciclo de reentrenamiento semanal incremental—.

Preparación causal y forma tensorial. La imputación (`ffill` con límite 24 y mediana de train) y el `StandardScaler` se ajustan exclusivamente sobre el bloque de train de cada fold, incluyendo el propio target —necesario porque irá dentro del tensor de entrada de la LSTM—. Con `LOOKBACK=168` y `stride=3` (para acotar el volumen de secuencias de entrenamiento), cada muestra tiene la forma (168 pasos de tiempo, 6 features).

7.5 Modelado

Concepto clave — Qué resuelve, y qué no, una LSTM
Una red recurrente simple propaga el gradiente hacia atrás en el tiempo multiplicándolo en cada paso; con secuencias largas, ese producto repetido tiende a desvanecerse o a explotar, y la red pierde la capacidad de aprender dependencias lejanas. Una LSTM introduce un estado de celda con compuertas —de olvido, entrada y salida— que regulan qué información se conserva, se actualiza o se descarta de forma aditiva, no solo multiplicativa: eso es, en teoría, lo que le permite retener información a lo largo de secuencias más largas que una RNN simple.
Esa ventaja estructural solo se traduce en mejor desempeño cuando existe una dependencia no lineal de largo alcance que un modelo más simple no puede representar. Si esa dependencia ya está capturada explícitamente por rezagos y ventanas móviles bien elegidos —como los que usa `LightGBM` en este mismo capítulo—, la LSTM no tiene nada adicional que explotar, y paga el costo de una optimización más difícil (más datos, más hiperparámetros, más inestabilidad de entrenamiento) sin ninguna compensación a cambio. Es la razón teórica detrás de por qué "más capacidad de modelado" no es sinónimo de "mejor modelo" en este capítulo.

Methodology Card — Cuándo usar Deep Learning
Use LSTM si hay suficiente historia, múltiples covariables y patrones no lineales. No la use por defecto con pocos datos, baja latencia extrema, o cuando seasonal naive o gradient boosting ya resuelven el problema. Compare siempre precisión, estabilidad **y** costo.
Seed: 42. **Dispositivo:** CPU.

Arquitectura. Una capa `LSTM` (`hidden=32`) seguida de una cabeza lineal, 5,153 parámetros en total. Entrenamiento con Adam ($\text{lr} = 10^{-3}$), hasta 12 épocas, `ReduceLROnPlateau` (factor 0.5, paciencia 2) y **early stopping** con paciencia 4, sobre un split interno 85/15 del propio bloque de train de cada fold.

Modelos de contraste. `LightGBM` usa las 5 covariables contemporáneas más lags (1, 24, 168) y codificación cíclica, entrenado sobre el año completo de train tabular de cada fold (`n_estimators=180`). `ARIMA(2,1,2)`, sin componente estacional, se ajusta solo sobre los últimos 90 días de train —una especificación deliberadamente simple, que **no** sigue el proceso riguroso de selección de orden del Capítulo 3—; su función aquí es la de un contraste rápido, no la de un ARIMA competitivamente ajustado. `SeasonalNaive` usa el rezago de 168 horas.

Modelo	MAE (MW)	RMSE (MW)	MASE	WAPE (%)
`LightGBM`	565.47 ± 33.62	755.40 ± 48.20	**0.234 ± 0.015**	1.07 ± 0.04
`SeasonalNaive`	1,195.72 ± 522.79	1,417.02 ± 535.44	0.495 ± 0.218	2.25 ± 0.93
`LSTM`	1,393.40 ± 106.70	1,891.36 ± 146.89	0.576 ± 0.046	2.64 ± 0.14
`ARIMA(2,1,2)`	7,896.26 ± 172.47	8,982.15 ± 174.38	3.263 ± 0.066	14.98 ± 0.60

Tabla 36. Comparación de modelos, promediada sobre los tres folds (media ± desviación estándar).

`LightGBM` domina con claridad; `LSTM` queda por detrás incluso del `SeasonalNaive` —el mismo patrón del Capítulo 2, ahora repetido con una arquitectura de deep learning en vez de ETS—. El ajuste de los cuatro modelos en los tres folds tomó 23.3 segundos en total, la mayor parte consumida por el entrenamiento de la LSTM. La Sección 7.6 discute por qué esta comparación no debe leerse como una condena general de las redes recurrentes, pero sí es un recordatorio empírico consistente con la filosofía de "cultura del baseline" del repositorio.

> **Decisión ejecutiva.** No se recomienda usar LSTM por defecto en este caso. LightGBM ofrece mejor precisión, menor costo y mayor simplicidad operativa.

7.6 Análisis posterior (post-modeling)

El diagnóstico se aplica a `LightGBM`, el modelo con menor MASE.

Alcance	Test	Estadístico	p-valor	Decisión
Fold 1	Ljung-Box (24)	65.03	1.21e-5	rechazar H0
Fold 2	Ljung-Box (24)	130.93	1.05e-16	rechazar H0
Fold 3	Ljung-Box (24)	78.73	9.68e-8	rechazar H0
Agregado	Jarque-Bera	91.80	1.16e-20	rechazar H0
Agregado	ARCH LM (24)	161.48	2.36e-22	rechazar H0

Tabla 37. Diagnóstico de residuos de LightGBM, por fold y agregado.

Segmento	MAE medio	Mediana	n
Normal (Q1-Q3)	594.49	448.38	378
Pico (Q4)	478.42	452.63	126

Tabla 38. Error absoluto segmentado por nivel de carga (cuartil superior por fold vs. resto).

A diferencia de todos los capítulos anteriores —donde el error se concentraba en los picos—, aquí el error en el segmento de mayor carga es **menor** que en el resto. Una lectura razonable es que los picos de demanda eléctrica nacional siguen un patrón laboral rutinario y predecible, mientras que los valles —determinados por comportamiento residencial nocturno más heterogéneo— son relativamente más difíciles de anticipar. El sesgo por fold es positivo y de magnitud similar en los tres (fold 1: +314.82, fold 2: +299.93, fold 3: +252.65 MW), es decir, `LightGBM` sub-pronostica de forma sistemática y consistente a lo largo del periodo evaluado.

¿Vale la pena el deep learning aquí? La Tabla 39 resume el argumento completo del capítulo:

Modelo	MASE medio	¿Requiere tensor secuencial?	Costo computacional	Recomendación
`LightGBM`	0.234	No	Bajo — no desglosado (resto del total de 23.3 s)	**Recomendado**: mejor precisión y menor complejidad operativa
`SeasonalNaive`	0.495	No	Trivial — sin entrenamiento (rezago fijo de 168 h)	Baseline de referencia obligatorio
`LSTM`	0.576	**Sí**	Alto — consume la mayor parte de los 23.3 s totales	No recomendada por defecto en este caso
`ARIMA(2,1,2)`	3.263	No	Bajo — ajuste corto deliberado (90 días)	Contraste rápido; no competitivo aquí

Tabla 39. Precisión frente a costo: los cuatro modelos comparados.

Nota de costo. La columna de costo se apoya únicamente en lo medido y documentado en este capítulo: el ajuste de los cuatro modelos en los tres folds tomó 23.3 segundos en total, con la mayor parte consumida por la LSTM. El tiempo individual de cada modelo no está desglosado en el notebook; las etiquetas de los demás modelos son cualitativas y se derivan de la especificación documentada (Seasonal Naive no entrena; ARIMA se ajusta deliberadamente solo sobre 90 días).

La LSTM no solo queda en tercer lugar en precisión: es también, de los cuatro, el modelo que exige la infraestructura más compleja —tensores 3D, entrenamiento por épocas, **scheduler** de tasa de aprendizaje— sin que esa complejidad adicional se traduzca en ninguna ventaja medible en este protocolo de evaluación.

7.7 Lecciones metodológicas

1. El escalador —y cualquier normalización, en general— se ajusta exclusivamente con train, nunca con el fold completo, incluso cuando el target forma parte del tensor de entrada.
2. Un tensor para LSTM tiene la forma muestras × 168 pasos de tiempo × features; construir correctamente esa forma es buena parte del trabajo de ingeniería de una red secuencial.
3. Una ventana de validación de una semana (168 horas) cubre el ciclo operativo completo de esta serie.
4. **Early stopping** y el **scheduler** de tasa de aprendizaje reducen el sobreajuste, pero no garantizan que el modelo resultante supere a alternativas más simples.
5. El deep learning debe justificar su costo de infraestructura frente a LightGBM y frente al seasonal naive; no debe asumirse superior por defecto.

6. La evaluación "one-step actualizada" —que usa observaciones reales del test conforme el horizonte avanza— no es equivalente a un forecast recursivo puro de 168 horas; son protocolos de evaluación distintos, y confundirlos sobreestima el desempeño real en producción.

Conexión con el Capítulo 8. El Capítulo 8 introduce arquitecturas diseñadas específicamente para forecasting (N-BEATS, un Transformer compacto inspirado en TFT) y traslada la pregunta de este capítulo —¿vale la pena la complejidad?— al terreno de la transferencia entre series nunca vistas.

Verificación con el sílabo. Las competencias de la Semana 6 quedan materializadas en: el experimento comparativo de modelos (Sección 7.5, Tabla 36), el diagnóstico de residuos de LightGBM (Tabla 37), el análisis de costo-precisión (Tabla 39) y la decisión ejecutiva de no usar LSTM por defecto. Ver Tabla 5.

7.8 Deudas abiertas del experimento

- **Fácil.** Cambiar `LOOKBACK` a 24, 336 y 672 pasos y medir la sensibilidad del MASE de la LSTM a esta elección.
- **Intermedio.** Evaluar un forecast verdaderamente recursivo —sin observaciones reales del test disponibles conforme avanza el horizonte— y comparar contra el protocolo "one-step actualizado" usado en este capítulo.
- **Difícil.** Comparar GRU y LSTM con igual número de parámetros para aislar el efecto de la arquitectura recurrente del efecto del tamaño del modelo.

CAPÍTULO 8

Semana 7: Arquitecturas modernas y transferencia entre series

Dataset: PeMS-SF (California PEMS traffic) · **Notebook:** `07_semana_7_transformers_transfer_sensores.ipynb` · **Fuente:** UCI Machine Learning Repository, licencia CC BY 4.0.

> **Ficha de caso — Semana 7**
> **Dominio:** tráfico/logística.
> **Problema de negocio:** predecir ocupación vial en sensores nuevos.
> **Usuario:** autoridad de transporte u operaciones viales.
> **Decisión que habilita:** desplegar sensores nuevos, estimar comportamiento o monitorear red.
> **Horizonte:** 6 pasos de 10 minutos.
> **Baseline obligatorio:** LastValue y Lag6.
> **Riesgo principal:** confundir transferencia con forecasting temporal clásico.
> **Métrica técnica:** MAE.
> **Métrica de negocio:** error en ocupación normalizada.

8.1 Fundamentos teóricos aplicados

Cuatro piezas de vocabulario sostienen el resto del capítulo: qué son N-BEATS y los Transformers, qué resuelve la atención temporal, qué distingue un TFT-like de un TFT completo, y —sobre todo— por qué un split por sensor cambia la pregunta que se está respondiendo, en términos accionables para un profesional de industria que decide si adoptar una arquitectura moderna.

Brecha metodológica: el Cap. 6 midió explícitamente el régimen warm/cold-start en clientes; este capítulo no midió la condición análoga en sensores nunca vistos. La diferencia se formaliza como limitación en el Cap. 10 §10.7. **Forecasting temporal vs. transferencia entre series.** El forecasting temporal clásico pronostica el futuro de una serie a partir de su propio pasado: el train y el test comparten la misma serie y difieren en el tiempo. La transferencia entre series responde una pregunta distinta: lo aprendido en unas series se aplica a otras que el modelo nunca vio durante el entrenamiento. En este capítulo el tiempo se comparte entre train y test —los mismos 440 días— y lo que cambia es el conjunto de sensores: 770 entrenan y 193 quedan completamente fuera. Es la misma distinción que el Capítulo 6 introdujo con clientes eléctricos

(296 entrenan, 74 nunca vistos), pero llevada a un extremo estructural: aquí el valor del modelo no está en predecir el día siguiente de un sensor conocido, sino en estimar el comportamiento de un sensor que acaba de instalarse, sin una sola observación propia. Confundir ambas preguntas invalida la lectura de las métricas: un MAE obtenido con un split temporal no dice nada sobre la capacidad de transferir a sensores nuevos, y viceversa.

Por qué el horizonte corto cambia las reglas. Con un horizonte de 6 pasos de 10 minutos (una hora), la persistencia de corto plazo domina el problema: la ocupación vial de los próximos 10 minutos se parece mucho a la de los últimos 10 minutos. Ese es el motivo por el que el baseline `LastValue` —repetir el último valor observado— es tan fuerte en este capítulo (MAE 0.013671): no porque sea un modelo sofisticado, sino porque la señal que predice es altamente persistente en el corto plazo. Cualquier arquitectura que aspire a justificar su costo debe superar ese piso bajo el mismo protocolo, y es exactamente el contraste que organiza la evaluación.

N-BEATS. N-BEATS (**Neural Basis Expansion Analysis for Time Series**) es una arquitectura de forecasting diseñada específicamente para series temporales, sin recurrencia ni atención: está compuesta por bloques apilados de redes **feed-forward** (MLP) que procesan la ventana de entrada. Su nombre alude a que cada bloque puede entenderse como una expansión de la serie en "bases" aprendidas —componentes que el bloque es capaz de representar—, en lugar de usar bases fijas predefinidas como las de una descomposición clásica. En este capítulo se usa una versión compacta de dos bloques, cada uno con un MLP de dos capas ocultas.

Backcast y forecast. Cada bloque de N-BEATS produce dos salidas: un **backcast**, que reconstruye (explica) la parte de la ventana de entrada que el bloque entiende, y un **forecast**, que predice el horizonte futuro a partir de esa comprensión. La idea es que el bloque primero "mira hacia atrás" para modelar lo que ya pasó, y usa ese modelo para "mirar hacia adelante". El backcast no se usa directamente como predicción: se descuenta de la ventana original para dejar expuesto lo que el bloque no supo explicar.

Apilamiento residual. El apilamiento residual es el mecanismo que conecta los bloques: el segundo bloque no recibe la ventana original, sino el **residuo** de la reconstrucción del primero —la ventana menos su backcast—. Cada bloque intenta explicar la porción que los anteriores no pudieron, de forma análoga a cómo el boosting ajusta árboles sobre el error del ensamble previo. Al final, el forecast total es la suma de los forecasts de todos los bloques. Esta descomposición secuencial hace que la arquitectura sea interpretable por construcción: se puede inspeccionar cuánto explica cada bloque, una propiedad que las redes recurrentes no ofrecen con la misma transparencia.

Transformers. Un Transformer es una arquitectura que procesa la secuencia completa en paralelo —no paso a paso como una RNN— y decide qué partes de la entrada son relevantes mediante **autoatención**. En lugar de una memoria interna que se actualiza secuencialmente, cada posición de la secuencia puede "mirar" directamente a cualquier otra posición y ponderar su influencia. Para forecasting, esto elimina la restricción de memoria de largo plazo de las RNN: en principio, un paso de tiempo puede atender a cualquier otro paso de la ventana, sin importar la distancia. Esa flexibilidad tiene un costo: los Transformers son más exigentes en datos y cómputo

que los modelos tabulares, y su ventaja solo se materializa cuando existe una dependencia de largo alcance que la arquitectura puede explotar.

Atención temporal. La atención temporal es el mecanismo concreto que implementa esa capacidad de "mirar": para cada posición, el modelo calcula un peso de relevancia hacia las demás posiciones de la ventana —qué tan importante es cada paso pasado para predecir el futuro— y combina la información de forma ponderada. En este capítulo, el experimento de sensibilidad de la Sección 8.6 aproxima por permutación qué bloques de la ventana pesan más en la predicción final: el bloque más cercano al horizonte (la última hora antes de pronosticar) domina por completo, un resultado coherente con que la ocupación vial es altamente persistente en el corto plazo. Es importante no sobreleer el resultado: la sensibilidad por permutación mide el impacto de remover información, no la atención espacial entre sensores ni causalidad.

TFT-like. El Temporal Fusion Transformer (TFT) es una arquitectura diseñada para forecasting con covariables: combina autoatención temporal con **gating** —mecanismos que aprenden a filtrar qué señales pasan— y soporta variables estáticas (que no cambian en el tiempo) y variables conocidas de antemano. Un **TFT-like** es una implementación compacta inspirada en esa arquitectura: en este capítulo, una proyección lineal de la ventana, una capa `TransformerEncoderLayer` de 4 cabezas de atención y un **gate** aprendido (sigmoide) antes de la cabeza de salida, con 9,697 parámetros. La diferencia con un TFT completo es sustancial: faltan las redes de selección de variables, las covariables estáticas y futuras explícitas, y el decodificador multi-horizonte de la arquitectura original.

Etiquetado honesto. La distinción entre un TFT-like y un TFT completo no es un matiz académico: afecta lo que las métricas significan. Un TFT completo requiere un framework dedicado y covariables estáticas/futuras explícitas —ninguna de las cuales existe en PeMS-SF, donde no hay fechas calendario reales ni metadatos geográficos—. Por eso el capítulo etiqueta la implementación como **inspirada** en TFT, y ninguna afirmación de esta monografía debe leerse como una evaluación del TFT original. El mismo principio aplica al N-BEATS compacto: es una versión reducida de la arquitectura publicada, no la implementación de referencia.

Complejidad vs. beneficio. La pregunta operativa que cierra el capítulo es cuánto compra cada unidad de complejidad. El `TFT-like` gana con un MAE de 0.013122 frente al baseline `LastValue` (0.013671), una mejora de apenas 4.0 % tras 398.45 segundos de entrenamiento y 9,697 parámetros; `N-BEATS` alcanza casi el mismo resultado (0.013360, a 1.7 % del `TFT-like`) en 95.41 segundos —una cuarta parte del tiempo—; y la LSTM, la arquitectura recurrente "clásica", pierde contra el baseline (0.013817) tras 360.58 segundos. El costo total de las tres arquitecturas es de 854.4 segundos, el más alto de la monografía. La lección no es que las arquitecturas modernas no sirvan: es que su beneficio en precisión puntual es marginal sobre este dataset, y su valor real hay que buscarlo en la capacidad de transferir a sensores nunca vistos.

Leer el MAE en unidades de negocio. La métrica técnica (MAE) y la métrica de negocio (error en ocupación normalizada) son la misma cifra vista desde dos ángulos: la ocupación se normalizó a [0,1], así que un MAE de 0.013122 significa un error promedio de 1.3 puntos porcentuales de ocupación —poco, pero no despreciable en un sensor crítico donde una hora de congestión mal anticipada tiene costo operativo—. El **costo de propiedad** suma además al tiempo de entrena-

miento: las arquitecturas modernas exigen selección de hiperparámetros, control de versiones de pesos y reentrenamiento periódico, tareas que un baseline sin entrenamiento simplemente no tiene.

¿Qué hace que un modelo transfiera? La transferencia a sensores nuevos no es una propiedad automática de la arquitectura: depende de que el modelo aprenda patrones **universales** —el ciclo diario de congestión, la persistencia de corto plazo— en lugar de memorizar el identificador de cada sensor. Tres condiciones la favorecen: entradas normalizadas de forma que no dependan de estadísticas globales del train (el escalado por ventana de la Sección 8.4 es un ejemplo deliberado), regularización que evite sobreajustar los 48 sensores de entrenamiento, y una arquitectura cuya inducción temporal —residual, recurrente o atencional— sea lo bastante general para extrapolar a series nuevas. Ninguna de estas condiciones garantiza transferencia: la única evidencia válida es la medición sobre sensores nunca vistos, que es exactamente lo que el split por sensor de este capítulo produce.

Resumen operativo. La cadena de decisiones del capítulo es: definir la pregunta correcta (transferencia a sensores nuevos, no forecast temporal); construir baselines simples de persistencia (`LastValue` y `Lag6`) que fijen el piso; evaluar arquitecturas modernas bajo el mismo protocolo de split por sensor y cohorte declarada; y decidir por complejidad vs. beneficio, no por prestigio de arquitectura. La transferencia es el valor principal que estas arquitecturas pueden aportar; si solo importa la precisión puntual, un baseline simple puede ser suficiente.

8.2 Configuración y contrato de datos

El archivo `data/07_traffic_pems/traffic_pems.csv` tiene 63,360 filas y 967 columnas (408.22 MiB): 4 columnas de metadatos (`split`, `day_id`, `slot_10min`, `weekday_label`) más 963 columnas `station_*`, una por sensor de ocupación vial. A diferencia de los siete datasets anteriores, **no existen fechas calendario reales**: `day_id` y `slot_10min` son identificadores posicionales, y se construye un índice temporal `time_index = day_id × 144 + slot_10min`, único, de 0 a 63,359 —440 días de 144 intervalos de 10 minutos cada uno—.

Split obligatorio: por sensor, no por tiempo. 770 sensores entrenan y 193 quedan completamente fuera, con una partición determinista (semilla 42) sin solapamiento. Este split mide **transferencia a sensores nunca vistos**, no forecast temporal clásico: el tiempo se comparte entre train y test, lo que cambia es el conjunto de sensores.

Cohorte de cómputo. Para viabilidad en CPU, este capítulo carga y entrena únicamente sobre una cohorte determinista de 48 sensores de entrenamiento y 24 de test —un subconjunto explícito del split completo, no datos simulados—, resultando en una matriz de trabajo de (63,360, 76) y 22.58 MiB en memoria. **El split completo de 770/193 queda definido pero no ejecutado en este notebook**; todas las cifras de esta sección corresponden a la cohorte 48/24, y así se citan en toda la monografía.

Nótese además una columna `split` heredada de la fuente original (train: 38,448 filas, test: 24,912 filas) que opera a **nivel de fila**, no de sensor: es distinta del split por sensor que estructura este capítulo y no debe confundirse con él. El rango de ocupación normalizada (sobre los 72 sensores

cargados) tiene media 0.054, mediana 0.047, percentil 99 = 0.258, máximo 1.0 —una distribución con cola derecha pronunciada, típica de una medida de ocupación acotada en [0,1]—.

8.3 Análisis previo (pre-modeling)

El perfil intradiario (media por `slot_10min` entre los sensores de entrenamiento) y el perfil por `weekday_label` muestran los ciclos diario y semanal esperables en tráfico vial. Una matriz de correlación entre 24 sensores de entrenamiento permite calcular una medida de **conectividad estadística** —correlación absoluta media con el resto de sensores, no topología física de la red vial—: el sensor más "conectado" es `station_401186` (0.616), el más "aislado" `station_400560` (0.341).

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Nivel	−19.09 (0.000)	0.278 (0.10)	Estacionaria por ambos tests
Diferencia diaria (144)	−14.65 (3.48e-27)	0.006 (0.10)	Estacionaria por ambos tests

Tabla 40. Tests de estacionariedad sobre un sensor de ejemplo (`station_400137`, primeros 120 días).

La detección de outliers (MAD, $|z| > 4.5$) sobre el residuo STL diario del sensor de ejemplo marca 1,454 candidatos.

8.4 Estrategia de validación

```
Split por sensor (obligatorio):    770 sensores train  |  193 sensores test (nunca vistos)
Cohorte CPU (ejecutada aquí):       48 sensores train   |   24 sensores test
```

Figura 7. Partición por sensor (transferencia) frente a partición temporal.

Construcción de ventanas. `LOOKBACK=72` pasos (12 horas) y `HORIZON=6` pasos (1 hora). Las ventanas de train se extraen de los días 0–300 con paso (`stride`) 24; las de test, de los días 300–440 con paso 12. El resultado: 86,256 ventanas de train y 40,176 de test, cada una de forma (72, 1). El escalado es **por ventana**, dividiendo por la media absoluta del propio contexto (`lookback`) —una elección deliberada: esta forma de escalar está disponible incluso para un sensor genuinamente nuevo, porque no depende de estadísticas globales del conjunto de entrenamiento—.

8.5 Modelado

Concepto clave — Apilamiento residual y atención
`N-BEATS` apila bloques donde cada uno intenta "explicar" (**backcast**) la parte de la ventana de entrada que entiende, y produce su propia porción de pronóstico (**forecast**) a partir de esa comprensión; lo que el bloque no logra explicar se pasa como residuo al siguiente. Es, en espíritu, una descomposición secuencial parecida a boosting, pero implementada con bloques de red neuronal en cascada en vez de árboles.
La autoatención de un Transformer reemplaza la recurrencia estricta de una LSTM por un mecanismo donde cada paso de tiempo de la ventana de contexto puede "mirar" directamente a cualquier otro paso y ponderar su relevancia para la predicción, sin depender de propagar esa información paso a paso. Es justamente lo que el experimento de sensibilidad de la Sección 8.6 aproxima por permutación: qué bloques de la ventana pesan más en la predicción final. Que la ganancia de estas arquitecturas sobre un baseline trivial sea de apenas 4 % no contradice la teoría: sugiere que la señal de ocupación vial de este dataset ya es, en gran medida, capturable sin modelar dependencias complejas de largo alcance; donde estas arquitecturas sí aportan es en la capacidad de transferir a sensores nunca vistos, no necesariamente en la precisión punto a punto.

Methodology Card — Selección de arquitectura
`N-BEATS`: univariado y sin covariables. `LSTM`: recurrencia compacta. Transformer/TFT-like: atención temporal y **gating**. Un TFT completo requiere un framework dedicado y covariables estáticas/futuras explícitas. Aquí se implementa un Transformer compacto **inspirado** en TFT, etiquetado honestamente como tal —no se afirma equivalencia con la arquitectura original del paper—.
Seed: 42. **Dispositivo:** CPU.

Arquitecturas evaluadas (además de dos baselines, `LastValue` y `Lag6`, el valor una hora atrás dentro del propio lookback):

- **`N-BEATS` compacto:** dos bloques residuales; cada bloque es un MLP (`Linear(72→64)`, ReLU, `Linear(64→64)`, ReLU) con cabezas separadas de **backcast** y **forecast**; el segundo bloque opera sobre el residuo (`x - backcast_1`) del primero, siguiendo el apilamiento residual característico de N-BEATS. 27,154 parámetros.
- **`LSTM` pequeña:** una capa `LSTM(1,32)` con cabeza lineal. 4,513 parámetros.
- **`TFT-like`:** proyección lineal a $d_{\text{model}} = 32$, una capa `TransformerEncoderLayer` (4 cabezas de atención, `dim_feedforward=64`, `dropout=0.1`) y un **gate** aprendido (sigmoide) antes de la cabeza de salida. 9,697 parámetros.

Las tres arquitecturas se entrenan con Adam (lr $= 10^{-3}$), hasta 8 épocas, **early stopping** con paciencia 3 sobre el 15 % final de las ventanas de train.

Modelo	MAE	RMSE	WAPE (%)	Tiempo de ajuste (s)
`TFT-like`	**0.013122**	0.028147	25.31	398.45
`N-BEATS`	0.013360	0.028224	25.77	95.41
`LastValue` (baseline)	0.013671	0.029532	26.37	—
`LSTM`	0.013817	0.029438	26.65	360.58
`Lag6` (baseline)	0.019749	0.038775	38.09	—

Tabla 41. Comparación final, ordenada por MAE, sobre las 40,176 ventanas de test de la cohorte 48/24.

El entrenamiento de las tres arquitecturas tomó 854.4 segundos en total (14.2 minutos) —el capítulo computacionalmente más costoso de la monografía—. El resultado central, sin embargo, es de magnitud modesta: `TFT-like` gana con un MAE de 0.013122 frente al baseline trivial `LastValue` (0.013671), una mejora de **apenas 4.0 %** tras casi 400 segundos de entrenamiento y 9,697 parámetros. `N-BEATS` alcanza casi lo mismo (0.013360, a 1.7 % de `TFT-like`) en una cuarta parte del tiempo de ajuste. `LSTM` —la arquitectura recurrente "clásica"— **pierde** contra el baseline `LastValue`, repitiendo el patrón ya visto en el Capítulo 7.

> **Decisión ejecutiva.** Las arquitecturas modernas pueden justificar su uso cuando la transferencia a sensores nuevos es un requisito de negocio. Si solo importa precisión puntual, un baseline simple puede ser suficiente.

Esta cifra de 0.013122 corresponde a la cohorte CPU 48/24 efectivamente ejecutada, no al split completo de 770/193 sensores. El README del repositorio resume el resultado como "MAE 0.01314 con 770 sensores entrenados, prediciendo 193 nuevos"; esta monografía usa la cifra de la cohorte 48/24 —0.013122, muy cercana mas no idéntica— porque es la que corresponde a la ejecución verificable en el notebook citado, no al split completo que queda definido pero no corrido.

Sensibilidad temporal aproximada. Permutando bloques de 6 pasos del `lookback` del `TFT-like` y midiendo el aumento de MAE resultante, el bloque más cercano al horizonte de predicción (posiciones 67–72, es decir, la última hora antes de pronosticar) domina por completo la sensibilidad (

$$\Delta\text{MAE} = 0.0373$$

), mientras que bloques más lejanos (posiciones 1–61) tienen impacto casi nulo (entre −0.00004 y 0.00016). Esto es una medida de **sensibilidad**, no de atención espacial ni de causalidad entre sensores —el propio notebook advierte contra esa lectura—.

8.6 Análisis posterior (post-modeling)

Modelo	MAE (media ± std)	WAPE (%, media ± std)
`TFT-like`	0.01312 ± 0.00338	25.22 ± 4.26
`N-BEATS`	0.01336 ± 0.00381	25.63 ± 4.92
`LastValue`	0.01367 ± 0.00335	26.32 ± 4.06
`LSTM`	0.01382 ± 0.00383	26.53 ± 4.89
`Lag6`	0.01975 ± 0.00501	37.89 ± 5.64

Tabla 42. Métricas macro por sensor (media ± desviación estándar sobre los 24 sensores de test).

Según el mejor modelo (`TFT-like`), el sensor más fácil de predecir es `station_401185` y el más difícil `station_400715` —identificados por MAE individual, sin que el notebook explique la causa de esa dificultad diferencial, dado que no hay metadatos geográficos disponibles—.

Test	H0	Estadístico	p-valor	Decisión
Ljung-Box (10)	sin autocorrelación	26.05	0.0037	rechazar H0
Ljung-Box (20)	sin autocorrelación	50.25	0.0002	rechazar H0
Jarque-Bera	normalidad	3.83e6	0.000	rechazar H0
ARCH LM (12)	homocedasticidad	83.84	7.60e-13	rechazar H0

Tabla 43. Diagnóstico de residuos de `TFT-like` sobre un sensor de test ordenado temporalmente.

Los tres tests rechazan, patrón ya familiar en la monografía. El error segmentado por `weekday_label` varía de 0.007 (etiquetas 1 y 7, consistentes con fin de semana dado su menor tráfico y error) a 0.0165 (etiqueta 3), con tamaños de segmento entre 4,032 y 7,200 ventanas. Ningún modelo de este capítulo se ejecutó en GPU; el notebook documenta explícitamente esa limitación sin cuantificar el costo relativo de una ejecución acelerada, que queda fuera de su alcance.

Recomendación ejecutiva de arquitectura. La Tabla 44 resume la decisión de arquitectura sobre la evidencia de la cohorte 48/24:

Modelo	MAE	Tiempo (s)	Complejidad	Recomendación
`TFT-like`	**0.013122**	398.45	Alta — atención y **gating** (9,697 parámetros)	Ganador en MAE; justificar el costo si la transferencia es requisito
`N-BEATS`	0.013360	95.41	Media — apilamiento residual (27,154 parámetros)	Costo-eficiente: casi igual, en la cuarta parte del tiempo
`LastValue` (baseline)	0.013671	—	Nula — sin entrenamiento	Baseline de referencia; suficiente si solo importa precisión puntual
`LSTM`	0.013817	360.58	Media — recurrencia (4,513 parámetros)	No recomendada: pierde contra el baseline
`Lag6` (baseline)	0.019749	—	Nula — sin entrenamiento	Baseline de referencia; no competitivo

Tabla 44. Recomendación ejecutiva de arquitectura, cohorte 48/24.

8.7 Lecciones metodológicas

1. PeMS-SF no incluye fechas calendario reales; no deben fabricarse fechas ficticias para completar el análisis.
2. Un split por sensor mide transferencia a series nunca vistas, no forecast temporal estándar; son preguntas distintas y requieren lecturas distintas de las métricas resultantes.
3. Las cohortes de cómputo reducido deben declararse explícitamente como tales; presentarlas como si fueran el benchmark del split completo sería una afirmación no respaldada por la ejecución real.
4. Un `TFT-like` no es un TFT completo: faltan variables estáticas, redes de selección de variables y un decodificador multi-horizonte propios de la arquitectura original.
5. La sensibilidad temporal medida por permutación de bloques no implica causalidad espacial entre sensores, aun cuando la visualización resultante pueda sugerirla.

La misma disciplina —registrar lo no ejecutado en vez de disfrazarlo de resultado— rige para DeepAR en el Cap. 9 y se formaliza en la tabla “especificado vs. ejecutado” del Cap. 10 §10.7.

Veredicto agregado del deep learning en el documento: Cap. 6 (no fabrica información sin historia), Cap. 7 (pierde por mucho cobrando caro), este capítulo (gana por poco, no paga la factura). El Cap. 10 §10.4 sintetiza: ‘capacidad comprobada, beneficio no pagado’.

Conexión con el Capítulo 9. El Capítulo 9 abandona el pronóstico puntual que ha organizado los Capítulos 2 a 8 y pasa a pronósticos probabilísticos —distribuciones y cuantiles—, cerrando con las prácticas de monitoreo y gobierno de modelos en producción.

Verificación con el sílabo. Las competencias de la Semana 7 quedan materializadas en: el análisis de transferencia entre sensores (Sección 8.5, Tabla 41), la recomendación ejecutiva de arquitectura (Tabla 44) y el diagnóstico de residuos del TFT-like (Tabla 43). Detalle consolidado por competencia: Tabla 5.

8.8 Deudas abiertas del experimento

- **Fácil.** Aumentar la cohorte hasta el split completo de 770/193 sensores y medir el costo computacional adicional frente a la Tabla 41.
- **Intermedio.** Implementar un forecast verdaderamente multi-horizonte (6, 12 y 18 pasos simultáneos) en vez del horizonte único de 6 pasos usado en este capítulo.
- **Difícil.** Ejecutar el mismo experimento en GPU y reportar energía y latencia frente a la ejecución CPU documentada aquí.

CAPÍTULO 9

Semana 8: Forecasting probabilístico y MLOps

Dataset: M3 Monthly (M Competition) · **Notebook:** `08_semana_8_forecasting_probabilistico_mlops.ipynb` · **Fuente:** Zenodo / M Competition, términos de la competencia M3.

Ficha de caso — Semana 8
Dominio: planeamiento mensual, finanzas u operaciones.
Problema de negocio: gestionar riesgo bajo incertidumbre.
Usuario: planificación, finanzas, inventarios o riesgo.
Decisión que habilita: stock de seguridad, presupuesto o capacidad.
Horizonte: 18 meses.
Baseline obligatorio: EmpiricalNaive.
Riesgo principal: intervalos gaussianos sin verificar cobertura.
Métrica técnica: CRPS aproximado.
Métrica de negocio: cobertura del intervalo P10-P90.

9.1 Fundamentos teóricos aplicados

Esta sección entrega el vocabulario mínimo que el resto del capítulo usa: qué significa pasar de un pronóstico puntual a uno probabilístico, cómo se leen cuantiles e intervalos, por qué el pinball loss y el CRPS son reglas de puntuación propias, qué miden la cobertura y la **sharpness**, y qué herramientas de monitoreo —drift, PSI, Model Card— sostienen un motor probabilístico en producción. Todo en términos accionables para un profesional de industria que decide qué entregar, cómo vigilarlo y cuándo reentrenar.

Predicción puntual vs. probabilística. Un pronóstico puntual entrega un único número —el P50 o la media— y con él una sola respuesta a la pregunta de negocio. Un pronóstico probabilístico entrega una distribución o un conjunto de cuantiles sobre los posibles valores futuros, y con ella una respuesta con rango: el intervalo donde es razonable esperar que caiga el valor y la probabilidad asociada a cada escenario. Para planeamiento mensual —stock de seguridad, presupuesto o capacidad— la decisión no depende solo del número central sino de la cola: cuánto stock extra cubre un mal mes, qué presupuesto es prudente reservar. Esa información solo la aporta la distribución, no el punto. El punto sigue siendo útil como resumen —por eso este capítulo reporta el P50—, pero no es el producto principal cuando la decisión es de riesgo.

Cuantiles. El cuantil q_τ es el valor que deja una fracción τ de la distribución por debajo: el P10 deja el 10 % de los escenarios por debajo y el P90 deja el 90 %. Cinco cuantiles definen el entregable de este capítulo: P10, P25, P50, P75 y P90. El P50 (mediana) es el punto central robusto; los cuantiles extremos (P10 y P90) delimitan el intervalo de incertidumbre; P25 y P75 describen la masa central. Leerlos en conjunto responde tres preguntas: ¿cuál es el valor esperado?, ¿qué tan dispersos están los escenarios?, ¿hacia dónde está sesgada la incertidumbre? Un modelo probabilístico bien construido debe producir cuantiles que no se crucen —$P_{10} < P_{25} < P_{50} < P_{75} < P_{90}$ en cada instante— y que, agregados, cubran la realidad observada.

Intervalos de predicción. Un intervalo de predicción entre dos cuantiles delimita el rango donde se espera que caiga la observación con una probabilidad dada. El intervalo P10–P90 de este capítulo tiene cobertura nominal de 80 %: si el modelo está bien calibrado, el 80 % de las observaciones futuras debería caer dentro del intervalo. La palabra clave es **nominal**: es lo que el intervalo promete, no lo que cumple. La cobertura empírica —la fracción realmente observada dentro del intervalo— puede diferir, y esa diferencia es el diagnóstico central del capítulo. A diferencia de un intervalo de confianza, que describe la incertidumbre de un parámetro estimado, el intervalo de predicción describe la incertidumbre de la observación futura misma —que es la que importa para decidir—.

Pinball loss. La función pinball cuantifica el error de un cuantil de forma asimétrica: penaliza distinto subestimar que sobrestimar, según el cuantil evaluado. Para el P90, por ejemplo, caer por debajo del valor real (subestimar un riesgo) cuesta más que caer por encima. Su propiedad teórica decisiva es que el valor que minimiza su valor esperado es exactamente el cuantil verdadero de la distribución condicional —por eso entrenar un modelo por cuantil con `objective="quantile"` produce cuantiles calibrados y no bandas arbitrarias alrededor de un punto central—. Este capítulo la usa en la Sección 9.5 para entrenar `LightGBMQuantile` y en la fórmula del CRPS aproximado.

Proper scoring rules. Una **proper scoring rule** es una función de pérdida donde la mejor estrategia posible del modelo es reportar su creencia real sobre la distribución —no una versión deliberadamente más angosta o más ancha para mejorar una métrica aislada—. El pinball loss, agregado sobre varios cuantiles, y el CRPS son reglas propias; el error absoluto sobre un punto no lo es. Esta propiedad es la razón teórica por la que este capítulo no construye bandas gaussianas simétricas como los Capítulos 2 a 4: la normalidad se rechazó en el 100 % de los diagnósticos de residuos de la monografía, y una regla de puntuación propia sobre cuantiles empíricos no depende de ese supuesto para seguir siendo válida.

CRPS aproximado. El CRPS (**Continuous Ranked Probability Score**) mide la calidad global de una distribución de pronóstico: castiga tanto la desviación del punto central como la falta de calibración de los extremos, con lo que resume calibración y **sharpness** en una sola cifra, en las unidades del target. El CRPS exacto es una integral sobre todos los cuantiles; este capítulo lo **aproxima** evaluando el pinball en los cinco cuantiles del entregable y promediando —una aproximación de menor resolución que la integral completa, como advierte la Sección 9.7—. Menor CRPS es mejor: 627.38 para `EmpiricalNaive` frente a 356.27 para `LightGBMQuantile` significa que el segundo produce distribuciones más cercanas a la realidad, en las unidades de la serie.

Cobertura. La cobertura empírica es la fracción de observaciones que cae dentro de un intervalo dado. Un intervalo P10–P90 nominal del 80 % debe cubrir, en promedio, el 80 % de las observaciones: `LightGBMQuantile` alcanza 78.9 % (casi nominal), `EmpiricalNaive` solo 69.9 % (intervalos demasiado angostos o mal ubicados) y `LocalTrend200` 44.9 % (severamente descalibrado). La cobertura se verifica, no se asume: es la métrica de negocio del capítulo porque un intervalo que promete 80 % y cubre 70 % produce decisiones de stock y presupuesto sistemáticamente subdimensionadas. Monitorear la cobertura en producción (Tabla 48) es la garantía de que la promesa nominal sigue cumpliéndose cuando el mundo cambia.

Sharpness. La **sharpness** mide qué tan angosto es el intervalo: a igual cobertura, un intervalo más angosto es más informativo. El ancho medio P10–P90 de este capítulo —2,152.56 para `EmpiricalNaive`, 1,547.86 para `LightGBMQuantile`— es su medida operativa. La relación entre cobertura y sharpness es la que da sentido al diagnóstico: un intervalo angosto con cobertura baja no es mejor que uno ancho y calibrado —es simplemente optimista—, y uno ancho con cobertura alta no es útil para decidir. Las dos métricas se leen juntas, y ninguna sustituye a la otra.

Data drift. El **data drift** (deriva de datos) es un cambio en la distribución de las entradas que el modelo consume —aquí, la escala y el patrón estacional de las series, o la distribución de los errores—, sin que necesariamente cambie la relación entre entrada y salida. Puede deberse a cambios de mercado, de cliente o de medición. Detectar data drift no significa que el modelo esté mal: significa que el contexto en que fue entrenado dejó de ser el contexto en que opera, y que las métricas históricas ya no describen el desempeño actual.

Concept drift. El **concept drift** es un cambio en la relación entre las entradas y el target: la función que el modelo aprendió deja de ser válida, aunque las entradas se vean iguales. En el caso de este capítulo, un cambio de régimen que altere la dinámica mensual de las series —una política, una crisis, un cambio estructural— cambia la relación entre historia y futuro. La distinción importa porque la respuesta difiere: ante data drift puede bastar reentrenar con datos nuevos; ante concept drift hay que revisar el modelo y su especificación, no solo actualizar pesos.

PSI. El **Population Stability Index** (PSI) cuantifica cuánto se desplaza la distribución de una variable entre dos ventanas —referencia y actual—, dividiendo el rango en bandas y comparando proporciones. Es un estándar de la industria financiera para monitorear cambios de distribución. Este capítulo lo usa sobre la distribución del error absoluto: la simulación de la Sección 9.6 produce $\text{PSI} = 0.349$, por encima del umbral de 0.25 de la Tabla 48, lo que activaría la regla de investigar segmentos. PSI no dice por qué cambió la distribución; dice que cambió lo suficiente para merecer atención.

Model Card. Un Model Card es un documento estructurado que declara qué hace el modelo, con qué datos fue entrenado, qué horizonte cubre, qué salidas produce, cuál es su uso previsto, cuáles sus limitaciones, quién es su propietario y cuándo se reentrena. Es el artefacto de gobierno que convierte un modelo de laboratorio en un activo operativo auditable. La Tabla 50 documenta el Model Card de este capítulo: nueve campos, del nombre a la política de reentrenamiento, para el modelo `Empirical Seasonal-Error Quantiles` (el baseline `EmpiricalNaive`), con la salvedad explícita de la Sección 9.6.

Monitoreo y reentrenamiento. Un motor probabilístico en producción se vigila con umbrales explícitos y acciones concretas, no con un dashboard pasivo. La Tabla 48 define cuatro reglas: drift de MAE superior a 25 % (investigar o reentrenar), cobertura P10–P90 fuera de 0.75–0.85 (recalibrar), PSI de errores mayor que 0.25 (investigar segmentos) y antigüedad de datos mayor a un periodo esperado (detener el scoring). Cada regla tiene un umbral y una acción; sin esos tres elementos —umbral, propietario, acción— el monitoreo no es gobierno de modelos. El reentrenamiento se decide por reglas, no por calendario: la antigüedad de datos y el drift de MAE lo activan; la cobertura y el PSI activan recalibración o investigación antes de reentrenar a ciegas.

9.2 Configuración y contrato de datos

El archivo `data/08_m3/m3_monthly.csv` contiene 167,562 filas y 3 columnas (`unique_id`, `ds`, `y`): 1,428 series mensuales del benchmark M3, sin nulos ni duplicados de clave. A diferencia de los siete capítulos anteriores, **no hay covariables** —cada serie aporta únicamente su propia historia—.

Heterogeneidad estructural. La longitud de las series varía de 66 a 144 observaciones mensuales (media 117.3, mediana 133): no todas las series tienen la misma cantidad de historia disponible, una condición que el diseño de validación debe respetar. Las escalas también son heterogéneas —media de medias 4,971.28, con series individuales de media tan baja como 1,268.24 y valores mínimos que llegan a ser negativos en algunas series—, y el coeficiente de variación entre series va de 0.012 a 1.710.

9.3 Análisis previo (pre-modeling)

Sobre la serie más larga del panel (`M1058`, 144 observaciones), una descomposición STL mensual (periodo 12) antecede a los tests de estacionariedad:

Serie	ADF (p-valor)	KPSS (p-valor)	Decisión conjunta
Nivel	0.470 (0.984)	0.918 (0.010)	Ambos apuntan a no estacionariedad
Diferencia estacional (12)	−1.006 (0.751)	1.347 (0.010)	**Ambos siguen apuntando a no estacionariedad**

Tabla 45. Tests de estacionariedad sobre la serie diagnóstica `M1058`.

Este es el único caso en toda la monografía donde la diferenciación estacional **no** logra estabilizar la serie según ninguno de los dos criterios: ni ADF rechaza la raíz unitaria ni KPSS deja de rechazar estacionariedad, incluso después de diferenciar. Es consistente con series financieras o de negocio de baja frecuencia (mensual, con apenas 66 a 144 puntos), donde la evidencia estadística para diagnosticar estacionariedad es intrínsecamente más débil que en las series horarias de alta frecuencia de los capítulos anteriores. La detección de outliers (MAD, $|z| > 4.5$) sobre el residuo STL marca solo 15 candidatos, la tasa más baja de la monografía en términos relativos —esperable en una serie con apenas 144 observaciones—.

9.4 Estrategia de validación

Holdout oficial: los últimos 18 meses de cada una de las 1,428 series —el horizonte estándar de la competencia M3 Monthly, reconstruido contando hacia atrás desde el final de cada serie—. El resultado: 141,858 observaciones de train, 25,704 de test, cubriendo la totalidad de las 1,428 series. Usar el horizonte oficial de la competencia, en vez de un horizonte elegido ad-hoc, es lo que permite comparar resultados de este capítulo contra la literatura de M3 sin ambigüedad de protocolo.

9.5 Modelado

> **Concepto clave — Por qué el pinball loss produce cuantiles calibrados**
> El pinball loss es asimétrico: penaliza distinto un error por debajo que uno por encima del valor predicho, según el cuantil τ que se esté evaluando. Su propiedad clave, la que justifica todo el capítulo, es que el valor que minimiza su valor **esperado** es exactamente el cuantil τ verdadero de la distribución condicional de y—por eso entrenar un modelo separado por cuantil, con `objective="quantile"`, produce cuantiles calibrados y no solo bandas arbitrarias alrededor de un punto central.
> Esta propiedad convierte al pinball loss —y, agregado sobre varios cuantiles, al CRPS— en lo que se llama una **proper scoring rule**: una función de pérdida donde la mejor estrategia posible del modelo es reportar su creencia real sobre la distribución, no una versión deliberadamente más angosta o más ancha para "verse mejor" en una sola métrica aislada. Es la razón teórica por la que este capítulo no construye bandas gaussianas simétricas, como los Capítulos 2 a 4: dado que la normalidad se rechazó en el 100 % de los diagnósticos de residuos de esta monografía (Capítulo 10), una regla de puntuación propia sobre cuantiles empíricos es la que no depende de ese supuesto para seguir siendo válida.

> **Methodology Card — Producción probabilística**
> Entregar cuantiles, comprobar ausencia de cruces entre ellos, y evaluar calibración y cobertura por segmento. Un intervalo estrecho pero descalibrado no es mejor que uno ancho y calibrado. El modelo final necesita un propietario (**owner**), fuente de datos, límites de operación, monitoreo activo y un criterio explícito de reentrenamiento.
> **Seed:** 42. **Cuantiles:** P10, P25, P50, P75, P90.

$$L_\tau(y, q) = (\tau - \mathbb{1}[y < q])(y - q)$$

Donde L_τ es el pinball loss para el cuantil τ, y el valor observado y q el cuantil predicho. La cobertura al 80 % es la fracción de observaciones y que cae dentro del intervalo $[q_{0.1}, q_{0.9}]$; el CRPS se aproxima promediando L_τ sobre el conjunto de cuantiles evaluados y multiplicando por dos.

Se comparan tres enfoques:

- **`EmpiricalNaive` (baseline):** último valor observado más los cuantiles empíricos de la diferencia estacional (12 meses) del propio train de cada serie. Evaluado sobre las 1,428 series completas (n=25,704).
- **`LightGBMQuantile`:** cinco modelos `LGBMRegressor` independientes (`objective="quantile"`, un `alpha` distinto por cuantil), con features de lags (1, 2, 3, 6, 12, 18, 24), media y desviación móviles (6 y 12 meses, sobre el pasado) y codificación cíclica del mes. Los cuantiles se ordenan por fila después de predecir, para impedir cruces —una corrección post hoc, no una garantía estructural del modelo—. Evaluado también sobre las 1,428 series completas.
- **`LocalTrend200` ("ETS empírico"):** tendencia lineal local ajustada sobre las últimas 36 observaciones de cada serie, con cuantiles derivados de los residuos in-sample de esa tendencia. **Evaluado sobre una cohorte de solo 200 series** (n=3,600) por razones de costo en CPU, no sobre el panel completo.

Modelo	n (obs.)	CRPS aprox.	Cobertura 80 % (nominal 0.80)	Ancho medio P10–P90
`EmpiricalNaive`	25,704	627.38	0.699	2,152.56
`LightGBMQuantile`	25,704	**356.27**	**0.789**	**1,547.86**
`LocalTrend200`	**3,600**	1,121.37	0.449	3,081.47

Tabla 46. Comparación de los tres enfoques probabilísticos.

`LightGBMQuantile` gana en todas las métricas frente a `EmpiricalNaive`: reduce el CRPS aproximado en 43.2 %, con cobertura empírica (78.9 %) mucho más cercana al 80 % nominal que el baseline (69.9 %), y con intervalos más angostos. La comparación con `LocalTrend200` requiere una advertencia explícita: al evaluarse sobre solo 200 de las 1,428 series (n=3,600 frente a n=25,704), su cifra de CRPS **no es directamente comparable** en la misma base muestral que las otras dos filas —su desempeño notablemente peor podría reflejar, en parte, un subconjunto de series distinto, no solo una diferencia real de calidad del método—. Esta monografía reporta la cifra tal como aparece en el notebook, con esta salvedad explícita, en vez de homologar artificialmente el tamaño de muestra.

Sobre `DeepAR`. El notebook especifica —pero **no ejecuta**— un modelo `DeepAR`, documentando explícitamente que la dependencia (`NeuralForecast` o `GluonTS`) está ausente del entorno congelado y que entrenarlo en CPU excede el alcance de esta entrega. Queda registrada la especificación (horizonte 18, verosimilitud Student-T o binomial negativa según soporte, criterio de aceptación: pinball y cobertura deben superar al baseline empírico) como artefacto de diseño, no como resultado: en ningún punto de esta monografía se reporta una métrica de `DeepAR`, porque ninguna fue producida por una ejecución real.

> **Decisión ejecutiva.** En producción no basta con entregar una predicción puntual. Se recomienda entregar cuantiles calibrados y monitorear cobertura, drift y antigüedad de datos.

9.6 Análisis posterior (post-modeling)

Incertidumbre por serie. Las cinco series con mayor ancho de intervalo P10–P90 son `M704` (31,098), `M12` (20,486), `M689` (18,776.8), `M1349` (16,210) y `M585` (16,004.4). Una correlación de Spearman entre longitud de la serie y ancho del intervalo resulta significativa y negativa (

$$\rho = -0.392$$

, $p = 1.54 \times 10^{-53}$, rechazar $H_0 : \rho = 0$): series con más historia tienden a producir intervalos más angostos, un resultado intuitivo —más datos permiten estimar cuantiles de error con más precisión— y estadísticamente sólido dado el tamaño de la muestra.

Test	H0	Estadístico	p-valor	Decisión
Ljung-Box (12)	sin autocorrelación	14,490.66	0.000	rechazar H0
Jarque-Bera	normalidad	1.71e7	0.000	rechazar H0
ARCH LM (12)	homocedasticidad	1,793.63	0.000	rechazar H0

Tabla 47. Diagnóstico de residuos del punto central (P50) del baseline `EmpiricalNaive`.

El patrón de rechazo sistemático, ya familiar en toda la monografía, refuerza la elección metodológica del capítulo: en vez de asumir normalidad para construir bandas gaussianas —como se hizo, con matices, en los Capítulos 2, 3 y 4—, este capítulo estima cuantiles empíricos directamente, sin depender de ese supuesto.

Simulación de drift y detección. Se simula un escenario de cambio de régimen aplicando un incremento del 20 % al último tercio del target observado —una simulación explícita, claramente separada de los datos originales—: el MAE del P50 pasa de 835.66 a 1,067.90 (aumento de 27.8 %). Un detector PSI (**Population Stability Index**) sobre la distribución del error absoluto, comparando la primera mitad del periodo contra la segunda con drift simulado, produce $\text{PSI} = 0.349$.

Métrica	Umbral	Acción
Drift de MAE	Más de 25 % vs. referencia	Investigar / reentrenar
Cobertura P10–P90	Fuera de 0.75–0.85	Recalibrar
PSI de errores	> 0.25	Investigar segmentos
Antigüedad de datos	> 1 periodo esperado	Detener el scoring

Tabla 48. Reglas de monitoreo y reentrenamiento propuestas.

Del concepto a la herramienta. El capítulo define el **qué** del gobierno —qué vigilar, con qué umbral y con qué acción—; la Tabla 49 mapea cada concepto a la categoría de herramienta estándar que lo implementa. El mapeo es de categoría, no de marca: dentro de cada fila, las herramientas son alternativas entre sí.

Concepto	Herramienta estándar sugerida	Por qué esa categoría
Cobertura empírica y PSI	Evidently AI, NannyML, Alibi Detect	Monitorean distribuciones, drift y cobertura sobre predicciones en producción, sin exigir reetiquetado inmediato.
Model Card y registro de modelos	MLflow Model Registry, Weights & Biases (W&B)	Versionan el artefacto, registran linaje y controlan la promoción entre entornos; alojan el Model Card de la Tabla 50.
Prevención de leakage (embargo y **point-in-time**)	Feast, Tecton (feature stores)	Garantizan **point-in-time correctness**: sirven cada feature con el mismo corte temporal del entrenamiento, cortando la fuga por proximidad.
Orquestación del reentrenamiento (reglas de la Tabla 48)	Airflow, Prefect, Metaflow	Programan el pipeline, evalúan los umbrales y disparan reentrenamiento o recalibración según las reglas del capítulo.

Tabla 49. Del concepto de gobierno a la herramienta estándar: stack MLOps de referencia.

Con $\text{PSI} = 0.349$ superando el umbral de 0.25 de la Tabla 48, el escenario de drift simulado activaría, según las propias reglas del capítulo, una investigación de segmentos —una coherencia interna que vale la pena señalar: el ejercicio de simulación y el sistema de monitoreo propuesto están diseñados para calzar entre sí—.

Reentrenamiento ante concept drift: técnicas de olvido. Cuando el PSI supera el umbral y el diagnóstico confirma **concept drift** —no solo cambió la distribución de las entradas, cambió la relación entre entradas y target—, reentrenar sobre todo el histórico es un error: las observaciones anteriores al cambio enseñan una relación que ya no rige. El gobierno necesita una política de **olvido** explícita, con dos estrategias operativas estándar:

- **Ventana deslizante (sliding window).** Descartar el histórico anterior a un horizonte fijo y reentrenar solo con los últimos N periodos. Simple de auditar; exige elegir N y asumir que dentro de la ventana no hay otro cambio de régimen.
- **Ponderación temporal (decaimiento exponencial).** Conservar todo el histórico pero asignar a cada observación un peso que decae con su antigüedad. Retiene señal de regímenes previos sin dejar que dominen el ajuste actual; añade un hiperparámetro de decaimiento que debe validarse con el mismo protocolo temporal.

La elección no es neutral: la ventana deslizante reacciona más rápido al cambio, pero olvida los patrones estacionales largos (por ejemplo, el ciclo anual) si N es corto; el decaimiento exponencial los conserva, pero reacciona más lento. En ambos casos, el reentrenamiento se evalúa con rolling-origin y se verifica que la cobertura P10–P90 haya vuelto al rango 0.75–0.85 de la Tabla 48.

Una nota sobre el Model Card. El **Model Card** que cierra el capítulo documenta formalmente el modelo `"Empirical Seasonal-Error Quantiles"` —es decir, el baseline `EmpiricalNaive`— y no `LightGBMQuantile`, pese a que este último gana en las tres métricas de la Tabla 46. El propio notebook titula esta sección como "resumen del baseline seleccionado por robustez y alcance completo": una razón plausible es que el baseline se evalúa sobre el panel completo de 1,428 series con un método sin supuestos distribucionales ni dependencias de librerías adicionales,

mientras que `LightGBMQuantile`, aunque más preciso, introduce una capa adicional de complejidad operacional. Esta monografía documenta la elección tal como aparece en el notebook, sin inferir una justificación que el propio notebook no declara explícitamente.

Campo	Valor
Nombre	Empirical Seasonal-Error Quantiles
Versión	1.0
Datos	M3 Monthly, 1,428 series
Horizonte	18 meses
Salidas	P10 / P25 / P50 / P75 / P90
Uso previsto	Benchmark y enseñanza
Limitaciones	Sin variables exógenas; colas empíricas; no causal
Propietario	Equipo docente del curso
Reentrenamiento	En cada ciclo mensual completo, tras revisión de drift

Tabla 50. Contenido del Model Card del modelo documentado (`EmpiricalNaive`).

9.7 Lecciones metodológicas

1. Un horizonte de evaluación estándar —los 18 meses oficiales de M3— permite comparabilidad con la literatura, algo que un horizonte elegido libremente no garantiza.
2. El pinball loss mide la calidad de cada cuantil individual; cobertura y ancho, en conjunto, evalúan calibración y **sharpness** de forma que ninguna métrica sola captura por completo.
3. El CRPS reportado aquí es una aproximación por cinco cuantiles, no la integral exacta sobre la distribución completa; debe leerse como tal.
4. Un modelo de cuantiles independientes (`LightGBMQuantile`) puede producir cuantiles cruzados; este capítulo aplica un ordenamiento post hoc, que corrige el síntoma sin eliminar la causa estructural del problema.
5. MLOps exige umbrales explícitos, un propietario nombrado y acciones concretas ante cada alerta —un dashboard sin esos tres elementos no constituye gobierno de modelos, aunque lo parezca—.
6. No se reportan métricas de `DeepAR` porque no se ejecutó; documentar una especificación no ejecutada como si fuera un resultado sería, según los propios criterios de rigor de esta monografía, un error grave.

Conexión con el Capítulo 10. El Capítulo 10 cierra la monografía respondiendo a las preguntas de investigación planteadas en el Capítulo 1, con una tabla sinóptica que reúne los ocho experimentos —incluidas todas las salvedades de comparabilidad documentadas en cada capítulo— y una jerarquía de complejidad fundamentada en la evidencia acumulada, no en la intuición.

Verificación con el sílabo. Las competencias de la Semana 8 quedan materializadas en: el Model Card del baseline (Tabla 50), las reglas de monitoreo con umbrales y acciones (Tabla 48), la comparación de tres enfoques probabilísticos (Tabla 46) y la simulación de drift (Sección 9.6). Con

esto se cierra el recorrido por las ocho competencias; la síntesis consolidada queda en Tabla 5 y en Tabla 51.

9.8 Deudas abiertas del experimento

- **Fácil.** Calibrar los cuantiles de `LightGBMQuantile` mediante conformalización, como alternativa al ordenamiento post hoc usado en este capítulo.
- **Intermedio.** Ampliar el conjunto de cuantiles para obtener una aproximación de CRPS de mayor resolución que la actual (cinco niveles).
- **Difícil.** Implementar `DeepAR` como dependencia opcional y contrastar sus métricas contra las de la Tabla 46, cerrando la brecha que este capítulo deja documentada pero sin resolver.

CAPÍTULO 10

10

Síntesis, lecciones transversales y conclusiones

10.1 Tabla sinóptica de los ocho experimentos

Cap.	Pregunta de negocio	Familia ganadora	Veredicto (con condición)	Número-tótem	Palanca que lo explicó
2	Pronosticar tráfico horario con estacionalidad semanal fuerte	SeasonalNaive168 (baseline)	Gana el baseline bien alineado por timestamp; cualquier modelo más complejo paga caro sin descontar	MASE 0.916	Protocolo: split temporal 80/20. El random split produjo error menor — la lección fundacional
3	Pronosticar consumo energético residencial a 10 min	ETS (no ARIMA)	Procedimiento Box–Jenkins formalmente correcto no garantiza ganar; ETS supera a ARIMA(1,0,2)	MASE 0.681 vs. 0.901	Selección por AIC sobre cinco candidatos indistinguibles en validation (302 MAE); teoría sin sabor en test
4	Pronosticar PM2.5 con covariables meteorológicas	SARIMAX	Las exógenas ayudan — pero bajo clima observado, escenario oracle explícito	MAE 84.99 vs. 103.66 (−18.36 %) *	**techo oracle**: en producción exige pronóstico meteorológico real, no valor observado
5	Pronosticar uso de bicicletas con feature engineering tabular	LightGBM	Victoria real y estable — pero la atribución es al protocolo, no al modelo	MASE 0.457 vs. 0.818 (lineal) *	Features: 37 features con embargo 24h y checklist anti-leakage. Casual + registered = cnt es la trampa encarnada
6	Predecir consumo de clientes nuevos (cold-start estricto)	Empate 0.01 %	LightGBM global empata contra perfil hora×día en 0.01 %; cross-learning no fabrica información	MASE macro: 1.34 vs. 38.59 (29×)	Historia: la disponibilidad por entidad domina sobre la arquitectura. Sin historia, ningún modelo recupera la escala individual
7	Pronosticar carga eléctrica con covariables legítimas	LightGBM	Gana por mucho, pero el veredicto tiene asterisco de atribución abierta	MASE 0.234 vs. 0.576 (LSTM) *	Covariable: el pronóstico oficial de carga (`DE_load_forecast`) está legítimamente disponible ex-ante — pero condiciona la atribución de la victoria
8	Transferir a sensores nunca vistos	TFT-like	Gana por 4 % — pero N-BEATS es el ganador práctico al cuarto del costo	MAE 0.013122 vs. 0.013671 (+4 %) *	Arquitectura: capacidad comprobada, beneficio no pagado. Cohorte 48/24 ejecutada, no la 770/193 especificada
9	Emitir intervalos calibrados y monitorear drift	LightGBM Quantile	Único doble win del documento: calibración Y sharpness simultáneas	CRPS 356.27 vs. 627.38 (−43.2 %)	Calibración: cuantiles empíricos cobran la deuda acumulada de siete capítulos de no normalidad (8/8 Jarque-Bera rechazado)

Tabla 51. Síntesis por palanca, no por métrica. Cada fila condensa el hallazgo, su número-tótem con su asterisco visible (la condición que lo califica) y la palanca que lo explicó. La métrica cambia de escala entre capítulos: MASE puntual (2, 3, 5, 7), MAE absoluto (4, 8), MASE macro mediano (6), CRPS aproximado (9). Compárense las palancas, no los números.

Leyenda de asteriscos: (*) el número-tótem lleva una condición de validez que el veredicto hace explícita. **Incomparabilidad**: las métricas no son comparables entre capítulos (datasets, protocolos y estadísticos distintos); la tabla no invita a comparar 0.234 con 0.457 con 0.013 — comparaciones sin sentido que el documento pasó cuatro capítulos advirtiendo.

10.2 Respuesta a las preguntas de investigación

Las tres preguntas formuladas en §1.8 organizan la síntesis. Cada respuesta carga su evidencia específica y marca sus esquinas sin responder.

Pregunta 1: ¿Cuándo un modelo complejo aporta valor real sobre un baseline estacional simple, y bajo qué condiciones ese valor se diluye? El valor se concentra en dos circunstancias verificadas: cuando hay **features que explotar** — Capítulo 5 reduce el MASE a 0.457 con 37 features construidas bajo embargo temporal; Capítulo 9 reduce el CRPS en 43.2 % al pasar de punto a probabilístico — y cuando una **covariable legítimamente disponible ex-ante** aporta información genuinamente nueva — Capítulo 7 domina con `DE_load_forecast` como feature, no por la arquitectura. El valor se diluye en tres circunstancias igualmente verificadas: cuando la estacionalidad es tan fuerte y regular que un baseline bien alineado por timestamp deja poco margen (Capítulo 2, MASE 0.916); cuando la información faltante simplemente no existe en los datos disponibles, sin importar la sofisticación del modelo (Capítulo 6, MASE mediano 38.59 sin historia); y cuando el protocolo de evaluación "one-step actualizado" y un presupuesto acotado impiden que una LSTM compacta despliegue su capacidad teórica (Capítulo 7). **Esquina sin responder:** el few-shot intermedio (1 semana de historia del cliente nuevo) nunca se ejecutó en el Capítulo 6; la frontera exacta entre "demasiada poca historia para cross-learning" y "suficiente para Few-Shot" permanece sin mapear.

Pregunta 2: ¿Qué papel cumplen las exógenas y los modelos globales cuando la disponibilidad de contexto es limitada, oracle o inexistente? Las exógenas añaden valor solo cuando la pregunta de disponibilidad futura tiene respuesta genuinamente positiva — distinción introducida en la Tabla 11 y aplicada sistemáticamente después. En el Capítulo 4, el clima observado mejora el pronóstico en 18.36 %, pero bajo escenario oracle explícito: la mejora es un **techo**, no un rendimiento de producción. En el Capítulo 7, las covariables de generación eólica/solar y el propio pronóstico oficial de carga sí están legítimamente disponibles ex-ante, lo que ayuda a explicar por qué LightGBM domina con MASE 0.234. En el Capítulo 6, cuando no existe ningún sustituto de la información faltante, ningún modelo puede recuperarla: el techo lo fija la disponibilidad de datos (1.34 vs. 38.59, 29×), no el algoritmo. **Esquina sin responder:** la transferencia en sensores (Capítulo 8) nunca se descompuso en sus dos componentes — generalización temporal (ventanas futuras) y de identidad (sensores nunca vistos); el error reportado mezcla ambas sin separarlas. Documentarlo como deuda, no como omisión menor.

Pregunta 3: ¿Cómo se traduce la no normalidad y heterocedasticidad casi universal de los residuos en decisiones operativas concretas? De los ocho capítulos con diagnóstico de residuos, Jarque-Bera rechaza normalidad en la totalidad de los casos (8/8) y ARCH LM rechaza homocedasticidad en siete de ocho — la única excepción es LightGBM en el Capítulo 5. La cuenta regresiva comenzó en el Capítulo 2 y se cerró en el §9.6: no es una preferencia estilística, es una regularidad empírica que justifica la progresión del curso. La respuesta operativa tiene dos partes: primero, dejar de construir intervalos gaussianos (las coberturas 76.8 %/68.5 %/86.3 %

contra 95 % nominal del Capítulo 4 son la evidencia más dura de que los intervalos simétricos mienten, y mienten de forma inestable entre folds — rango de 17.8 puntos porcentuales); segundo, estimar cuantiles empíricamente, monitorear su calibración de forma continua y definir umbrales explícitos de cobertura (0.75–0.85), PSI (> 0.25) y antigüedad de datos (Tabla 48). El Capítulo 9 entrega la primera victoria documentada sobre la incertidumbre: cobertura empírica 78.9 % contra 80 % nominal, con intervalos 28 % más angostos que el baseline empírico (1547.86 vs. 2152.56). **Esquina sin responder:** la calibración probabilística solo se validó en M3 Monthly (benchmark sin covariables); su comportamiento en los otros siete datasets queda como deuda explícita.

10.3 Análisis transversal: la asimetría de palancas

Lo que el documento descubrió, capítulo a capítulo, no fue una jerarquía de modelos sino una asimetría de palancas. La palanca más fuerte verificada por arquitectura — LightGBM con feature engineering riguroso, Capítulo 5 — rinde −44 % sobre el lineal. La palanca más fuerte verificada por régimen de datos — historia disponible por entidad, Capítulo 6 — rinde 29× en MASE mediano. La palanca del pronóstico oficial como covariable (Capítulo 7) explica en gran parte el mejor resultado absoluto del documento. La palanca del cambio de pregunta (puntual ⟶ probabilístico, Capítulo 9) rinde −43.2 % en CRPS. Entre el 44 % máximo que una familia de modelos puede aportar y el 2900 % que rinde un cambio de régimen de datos, hay un orden de magnitud de diferencia. La frase que solo puede vivir aquí: **la elección de arquitectura rara vez fue la decisión que más movió la aguja**.

Tres momentos concentran la densidad de hallazgo del documento y justifican que reciban aquí más espacio que el resto: el **Capítulo 2** como lección fundacional (el random split produjo un error menor, no mayor — lo que invalida la intuición naïve de que "más complejo = mejor" y enseñó que el protocolo es la condición de posibilidad de toda métrica); la **dupla 5–6** como palanca dominante (juntas muestran que el tabular bien alimentado es la arquitectura más rentable del curso, pero también que su victoria se desvanece sin historia — el −44 % del 5 y el empate del 6 son la misma lección vista desde los dos extremos de la palanca de features); y el **Capítulo 9** como única victoria sobre la incertidumbre (donde siete capítulos de no normalidad acumulada se cobran con un doble win estructuralmente difícil: calibración Y sharpness simultáneas).

El contrafactual afila la lectura: si el Capítulo 3 hubiera mostrado a ARIMA ganando (en vez de ETS con MASE 0.681 vs. 0.901), el documento habría perdido su evidencia más temprana de que la selección por AIC no garantiza ganar en test. Si el Capítulo 8 hubiera rendido 20 % en vez de 4 % (TFT-like vs. LastValue), las arquitecturas modernas habrían cobrado una recomendación operativa que la evidencia actual no sostiene. La contingencia de los hallazgos — no su generalidad — es lo que les da credibilidad.

10.4 Jerarquía de palancas, no de técnicas

Palanca	Decisión	Evidencia en el documento	Magnitud verificada	Cap. base
1. Protocolo	Split temporal correcto + embargo + test consultado una sola vez	Random split en Cap. 2 produjo error **menor**; el protocolo correcto no mejora la métrica, la vuelve significativa	condición necesaria	2, 5, 9
2. Features	Construcción tabular rigurosa con checklist anti-leakage + embargo 24h	LightGBM con 37 features en Cap. 5; casual + registered = cnt como trampa encarnada en Tabla 25	MASE 0.457 vs. 0.818	5
3. Historia	Disponibilidad por entidad (warm vs. cold-start)	MASE mediano 1.34 (warm) vs. 38.59 (cold) en Cap. 6; cross-learning no fabrica información	29×	6
4. Covariables	Disponibilidad futura real de la covariable (no oracle)	Pronóstico oficial de carga `DE_load_forecast` legítimamente disponible ex-ante en Cap. 7; clima observado como oracle en Cap. 4	0.234 (atribución abierta)	4, 7
5. Arquitectura	Cambio de familia (lineal → tabular → recurrente → attention)	LightGBM vs. LSTM en Cap. 7 (0.234 vs. 0.576); TFT-like vs. LastValue en Cap. 8 (+4 %)	máx. 44 %	5, 7, 8
6. Calibración	Paso de puntual a probabilístico con cuantiles empíricos	Cobertura empírica 78.9 % vs. 80 % nominal; intervalos 28 % más angostos en Cap. 9	−43.2 % CRPS	9

Tabla 52. Jerarquía de palancas construida sobre los resultados de los ocho capítulos. Cada nivel corresponde a una decisión metodológica cuya magnitud verificada supera la de la arquitectura elegida. Reemplaza los "Niveles 0–5" del sílabo (organizados por complejidad de técnica) por una ordenación por impacto medido.

La jerarquía no es lineal: las palancas 1 (protocolo) y 6 (calibración) son **transversales** — aplican a cualquier familia de modelos y son condiciones de validez, no optimizaciones. Las palancas 2, 3 y 4 son **dependientes del problema** — solo son activables si el dataset y el contexto de negocio lo permiten. La palanca 5 (arquitectura) es la de menor magnitud verificada en este documento, lo que invierte la intuición de que escalar complejidad es escalar valor. La asignación de esfuerzo editorial del documento es proporcional a la magnitud verificada de cada palanca, no a la sofisticación de la técnica que la activa.

10.5 Cuatro lecciones transversales (de longitud desigual)

1. **El principio unificado de fugas.** No filtrar información del futuro ni del evaluado, por ninguna vía — ni geométrica (random split, Cap. 2), ni temporal-conceptual (clima observado en test, Cap. 4), ni algebraica (`casual + registered = cnt`, Cap. 5), ni de identidad (lags del cliente nuevo, Cap. 6), ni de normalización (escalador ajustado a todo el dataset, Cap.

7). Cinco instancias verificadas, cinco mecanismos distintos, un solo principio. El oficio que este documento enseña es, antes que nada, la disciplina de auditar sistemáticamente estas cinco vías antes de atribuir una mejora al modelo. El checklist anti-leakage de siete puntos del Capítulo 5 (Tabla 25) es la materialización más concreta del principio; la Tabla 11 (contrato de disponibilidad futura) es su formalización más temprana.

2. **La jerarquía de palancas importa más que la jerarquía de técnicas.** La elección de arquitectura rindió como máximo un 44 % (Cap. 5); la disponibilidad de historia rindió 29× (Cap. 6). Esa asimetría reordena las prioridades: invertir primero en auditoría de features, segundo en covariables con disponibilidad futura real, tercero en régimen de datos por entidad — y recién después en arquitectura. Documentar el cambio de historia como variable explicativa principal (Cap. 6) y el asterisco de atribución del 0.234 (Cap. 7) es parte de esta lección.

3. **La condicionalidad como método.** La pregunta útil nunca fue "¿sirve X?" sino "¿sirvió aquí, con este presupuesto, contra estos competidores?". Los veredictos son condicionales y con número: la LSTM (Cap. 7) que compitió fue una LSTM compacta de `hidden=32` con presupuesto acotado, y perdió contra LightGBM y contra seasonal naive en ese dataset; el ARIMA del Cap. 3 fue un procedimiento Box–Jenkins formalmente correcto con selección por AIC sobre cinco candidatos indistinguibles (302 MAE en validation), y produjo MASE 0.901 — perdió contra ETS. La condicionalidad no es cautela retórica: es la forma de veredicto que la evidencia soporta.

4. **La honestidad como política documental.** Las dos honestidades estructurales del documento — la cohorte 48/24 ejecutada en vez de la 770/193 especificada (Cap. 8) y DeepAR especificado y no ejecutado en M3 (Cap. 9) — no son admisiones tardías sino decisiones administradas desde el diseño. La tabla "especificado vs. ejecutado" de la Sección 10.7 las formaliza. La costumbre adquirida: cuando una especificación no se ejecuta, se documenta como tal, no se reescribe como resultado. **Trazabilidad con el sílabo:** las cinco competencias del Capítulo 1 quedan cubiertas con esta evidencia — traducir negocio a forecasting (apertura de cada capítulo técnico), diseñar validación temporal (Caps. 2, 4, 5, 6), seleccionar modelos por decisión (Caps. 3, 7, 8), generar pronósticos probabilísticos (Cap. 9: Tabla 46 y motor `LightGBMQuantile`), desplegar y monitorear (Caps. 9 y 10: Tabla 48 y §10.10). La Tabla 5 queda remitida aquí; las dos representaciones ya no conviven.

10.6 Matriz de decisión (sustituye a las decisiones por capítulo)

La Tabla 51 agrupó hallazgos por capítulo. Esta matriz los agrupa por **condición de uso**: si su problema tiene [condición], entonces [familia recomendada], con esta evidencia [capítulo].

Si su problema tiene…	Entonces…	Con esta evidencia	Cap.
Estacionalidad estable y regular (semana/día/hora)	SeasonalNaive alineado por timestamp como piso difícil de superar; calcularlo siempre primero	MASE 0.916, modelo complejo no descuenta	2

Features ricas y embargo temporal posible	LightGBM con checklist anti-leakage y walk-forward	MASE 0.457 vs. 0.818 (lineal)	5
Series nuevas sin historia (cold-start estricto)	Perfil hora×día como baseline transparente; el modelo global solo iguala, no supera	MASE macro 1.34 vs. 38.59 (29×)	6
Covariables con disponibilidad futura real (no oracle)	LightGBM con covariables legítimas; auditar atribución	MASE 0.234 con asterisco de atribución abierta	7
Necesidad de transferir a entidades nuevas	TFT-like si el costo es asumible; N-BEATS si se prioriza costo/beneficio	MAE 0.013122 vs. 0.013671 (+4 %)	8
Necesidad de intervalos calibrados (riesgo operativo)	Cuantiles empíricos + monitoreo continuo de cobertura y PSI	CRPS −43.2 %, cobertura 78.9 % vs. 80 % nominal	9
Cualquier escenario con residuos no gaussianos (casi siempre)	Pasar de intervalos gaussianos a cuantiles empíricos	8/8 Jarque-Bera rechazado	2–9

La matriz está pensada para usarse en una decisión concreta: identifica la condición, lee la familia recomendada y el capítulo que la respalda. No es un menú, es un árbol de decisión.

10.7 Limitaciones con dientes

Las limitaciones siguientes están ordenadas por **impacto en un lector que quisiera aplicar este documento el lunes**, no por orden alfabético ni por severidad retórica.

1. **Atribución abierta del 0.234 (Capítulo 7).** El mejor resultado absoluto del documento no está plenamente cobrado: el pronóstico oficial de carga (`DE_load_forecast`) como feature legítimamente disponible condiciona la victoria. Un ablation — entrenar LightGBM sin esa covariable — no se ejecutó. Es la deuda que más condiciona la credibilidad del número estrella del documento; el lector debería aplicarlo con precaución hasta que ese ablation se ejecute.
2. **Cohorte 48/24 ejecutada, no la 770/193 especificada (Capítulo 8).** El experimento completo de PeMS-SF quedó sin correr. El veredicto "TFT-like gana por 4 %" se sostiene sobre 48 sensores de entrenamiento y 24 de test, no sobre los 770/193 del diseño original. La diferencia puede cambiar el ranking práctico entre N-BEATS y TFT-like.
3. **Validación del probabilístico en benchmark sin covariables (Capítulo 9).** El doble win calibración Y sharpness solo se verificó en M3 Monthly. Su comportamiento en los otros siete datasets — particularmente en series heterogéneas como Electricity Load Diagrams (Cap. 6) — queda como deuda.
4. **Incomparabilidad métrica transversal.** La Tabla 51 advirtió explícitamente: las métricas no son comparables entre capítulos. Esta limitación es metodológica, no accidental — el documento eligió protocolos distintos para responder preguntas distintas. Cualquier intento de "rankear" los ocho capítulos por su número-tótem comete el pecado de incomparabilidad que la propia tabla denuncia.

Especificado	Ejecutado	Cap.	Estatus
Split completo PeMS-SF 770/193 sensores	Cohorte 48/24 sensores	8	Recortado por costo de cómputo
`DeepAR` sobre M3	No entrenado	9	Especificado y no ejecutado — caso emblema de la política de honestidad
Few-shot con 1 semana de historia del cliente nuevo	No ejecutado	6	Punto ciego entre warm-start y cold-start
Ablation de LightGBM sin `DE_load_forecast` (pronóstico oficial OPSD)	No ejecutado	7	Deuda prioritaria: condiciona la atribución del mejor resultado del documento (MASE 0.234)
GRU como comparador de LSTM a igual número de parámetros	No entrenada	7	Omitida por presupuesto; relevante para aislar el efecto de arquitectura

Tabla 53. Especificado vs. ejecutado. La tabla más creíble que el documento puede ofrecer: las limitaciones no se descubren aquí, se administran desde el diseño.

10.8 Deudas prioritarias

El trabajo futuro honesto de este documento ya está escrito — son las deudas que las secciones X.8 documentaron capítulo a capítulo. La priorización no es estética: la primera deuda de la lista condiciona la credibilidad del mejor resultado del documento.

1. **Ablation del pronóstico oficial OPSD (Cap. 7).** Entrenar LightGBM sin `DE_load_forecast` y comparar con el modelo actual. Si solo pudiera ejecutar un experimento más, sería este: condiciona la atribución del 0.234 y, con ella, el veredicto sobre si la victoria de LightGBM en OPSD es del modelo o de la covariable.

```
Ablation de atribución — OPSD (Cap. 7)
1. Partir del pipeline de LightGBM del Cap. 7:
   5 covariables contemporáneas + lags (1, 24, 168) + codificación cíclica.
2. Eliminar la columna DE_load_forecast de la tabla de entrada.
3. Reentrenar con los mismos 3 folds, el mismo embargo y SEED = 42.
4. Comparar el MASE resultante contra el 0.234 del modelo completo.

Regla de decisión:
  MASE > 0.40   -> la victoria era de la covariable
  MASE < 0.30   -> la victoria es de la arquitectura tabular
  0.30 - 0.40   -> atribución mixta: reportar ambas y no reclamar el 0.234
```

2. **Split completo PeMS-SF 770/193 (Cap. 8).** Re-ejecutar la cohorte completa para verificar si el ranking práctico entre N-BEATS y TFT-like se mantiene.
3. **Few-shot intermedio (Cap. 6).** Mapear la curva MASE-vs-historia para identificar la frontera entre "demasiada poca historia para cross-learning" y "suficiente para Few-Shot".

```
Curva MASE-vs-historia — Electricity Load Diagrams (Cap. 6)
1. Tomar los 74 clientes de test del cold-start estricto.
2. Para cada ventana w en {0, 1, 2, 4, 8, 12} semanas de contexto:
   a. w = 0  -> reproducir el perfil hora x dia (cold-start estricto).
   b. w > 0  -> adaptar el modelo global con la ventana (few-shot):
                factor de escala sobre los errores recientes, o reentrenamiento
                con pesos iniciales del modelo global.
3. Reportar MASE macro(w) con dispersión entre clientes (no solo la media).
4. Localizar w*: el menor w cuyo MASE macro caiga por debajo del doble del
   warm-start local (2 x 1.34).
```

4. **`DeepAR` sobre M3 (Cap. 9).** El único modelo especificado que no se entrenó — cerrar la brecha o eliminar la especificación del scope.
5. **Calibración probabilística en los otros siete datasets (Cap. 9).** Extender el doble win calibración Y sharpness más allá de M3 Monthly.
6. **Ciclo de reentrenamiento incremental real (Cap. 9).** No simulado — siguiendo las reglas de la Tabla 48 sobre al menos un pipeline productivo.

10.9 Traducción a costo total de propiedad (TCO)

La evidencia de esta monografía está medida en precisión y en segundos de cómputo. Para un director de ingeniería o un responsable financiero, ninguna de las dos es una cifra de decisión: lo que decide es el **costo total de propiedad** (TCO) frente al **costo del error evitado**. La conversión tiene un resultado incómodo y una consecuencia práctica.

Los segundos no son el costo. A una tarifa ilustrativa de USD 0.20 por vCPU-hora en la nube, las corridas de entrenamiento documentadas cuestan centavos (la Tabla 54 lo cuantifica). El cómputo no fue la restricción de estos ocho experimentos —ni lo es de la mayoría de los pipelines de forecasting—. El TCO real lo dominan otros dos términos: el **costo recurrente de ingeniería y mantenimiento** del modelo (selección de hiperparámetros, control de versiones de pesos, monitoreo, reentrenamiento) y el **costo del error evitado**, que es el único que justifica —o descarta— el modelo.

Experimento	Tiempo de ajuste	Costo de cómputo / corrida	Mejora sobre el baseline	Costo dominante
Cap. 5 — LightGBM tabular	20.2 s	< USD 0.01	−44 % MASE vs. modelo lineal	Ingeniería de features (una vez)
Cap. 6 — LightGBM global	57.25 s	< USD 0.01	0.01 % (empate)	Historia ausente: no comprable
Cap. 7 — LightGBM + covariables	23.3 s	< USD 0.01	MASE 0.234 vs. 0.495 (SN)	Covariable ex-ante legítima
Cap. 8 — TFT-like	398.45 s	≈ USD 0.02	+4 % MAE vs. `LastValue`	Mantenimiento de infraestructura DL
Cap. 8 — N-BEATS	95.41 s	< USD 0.01	+2.3 % MAE vs. `LastValue`	Costo-eficiente: casi igual, en 1/4 del tiempo

Tabla 54. Traducción a costo total de propiedad. El costo de cómputo por corrida es despreciable frente al término que decide: el error evitado y el mantenimiento. Tarifa ilustrativa de USD 0.20 por vCPU-hora.

La regla de decisión. Considérese el caso estrella del documento: el `TFT-like` del Cap. 8 reduce el MAE de ocupación vial en 4 % sobre el baseline trivial `LastValue`, a cambio de 400 segundos de entrenamiento y una infraestructura de deep learning que exige mantenimiento continuo. Si el costo operativo de actuar sobre un error de pronóstico es Y por unidad y el modelo genera D decisiones por entidad y periodo, el ahorro esperado es $0.04 \times Y \times D$ por entidad. La mejora justifica el `TFT-like` solo cuando ese ahorro agregado sobre Z entidades supera el TCO (cómputo más ingeniería más monitoreo). Para el resto de la red, `LastValue` es la decisión financiera correcta: no porque sea más preciso, sino porque la precisión que compra el modelo no cubre su propio costo de operación.

La lección financiera del documento. El cómputo nunca fue la restricción; lo que se paga no son los segundos, sino (a) el costo recurrente de mantener un modelo complejo y (b) el costo del error que la mejora evita. La primera cifra se subestima con facilidad; la segunda, con la misma facilidad, no se mide nunca.

10.10 Recomendación operativa + cierre

La recomendación operativa que sintetiza los ocho experimentos, en orden de aplicación:

1. **Calcular siempre un baseline alineado por timestamp antes de cualquier modelo complejo.** En la mitad de los capítulos ese baseline resultó difícil o imposible de superar de forma significativa.
2. **Auditar features y covariables antes de elegir arquitectura.** La palanca de features (−44 %) y la de historia (29×) superan por un orden de magnitud la palanca de arquitectura (máx. 44 %).

3. **Validación temporal obligatoria con test consultado una sola vez.** Ninguna métrica significa nada sin el protocolo correcto: el contraste del Capítulo 2 muestra que el split inválido produjo un error **menor**, no mayor.
4. **Pasar a cuantiles empíricos como salida estándar** dado que Jarque-Bera rechazó normalidad en el 100 % de los diagnósticos. Monitorear cobertura (0.75–0.85), PSI (> 0.25) y antigüedad de datos con propietario nombrado y acciones concretas — la plantilla de la Tabla 50 (Model Card) es directamente reutilizable.

El cierre por número, no por epígrama: cuatro de ocho experimentos (Capítulos 2, 3, 7 y 8) mostraron al modelo formalmente más sofisticado igualando o apenas superando a una alternativa simple; las dos victorias grandes del documento (Capítulos 5 y 9) se explican por palancas que no son la arquitectura (features y calibración, respectivamente); la única victoria sobre la incertidumbre es la del Capítulo 9. Lo que no se ejecutó — DeepAR, split 770/193, ablation del pronóstico oficial, few-shot, GRU, calibración probabilística fuera de M3 — queda listado en la Tabla 53 como deuda administrada, no como omisión descubierta al cierre.

APÉNDICE A

Tabla de datasets

Cap.	Dataset	Fuente	Dimensión	Target	Nulos	Peculiaridades
2	Metro Interstate Traffic	UCI ML Repository, CC BY 4.0	48,204×9	`y` (tráfico)	`holiday` 99.87 %; resto 0 %	17 filas duplicadas; 7,629 timestamps duplicados; 11,976 h faltantes en rejilla
3	Appliances Energy	UCI ML Repository, CC BY 4.0	19,735×29 (10 min)	`y` (energía)	0 %	`rv1`/`rv2` controles negativos sintéticos; rejilla perfectamente regular
4	Beijing PM2.5	UCI ML Repository, CC BY 4.0	43,800×14	`pm2.5`	4.66 % en target (2,043 valores, 213 gaps)	Sin gaps de timestamp, solo nulos de target; `cbwd` categórica
5	Bike Sharing	UCI ML Repository, CC BY 4.0	17,379×17 (h) / 731×16 (d)	`cnt`	0 %	`cnt = casual + registered`; 165 h faltantes (0.94 %)
6	Electricity Load Diagrams	UCI ML Repository, CC BY 4.0	9,732,480×3 (370 series)	`y` (consumo)	0 %	Escala 43,000× entre cliente mínimo y máximo; formato **long**
7	OPSD Germany	Open Power System Data (OPSD)	50,401×43 (6 usadas)	`DE_load_actual_...`	Variable por columna; 0 % en ventana de 730 días usada	43 columnas con cobertura desigual; precio excluido (35 % cobertura)
8	PeMS-SF (Traffic)	UCI ML Repository, CC BY 4.0	63,360×967 (963 sensores)	Ocupación vial	0 %	Sin fechas calendario reales; split completo 770/193 definido, no ejecutado
9	M3 Monthly	Zenodo / M Competition	167,562×3 (1,428 series)	`y`	0 %	Longitud heterogénea (66–144 meses); sin covariables; algunos mínimos negativos

Tabla 55. Los ocho datasets utilizados: fuente, dimensiones, frecuencia, target, nulos y peculiaridades documentadas.

APÉNDICE B

Tabla de modelos

Modelo	Tipo	Hiperparámetros clave	Librería	Tiempo
Naive	Regla simple	Último valor de train	pandas	No medido
Seasonal Naive	Regla simple	Lag 24, 168 o 144 según capítulo	pandas/ numpy	No medido
Media móvil (MA)	Regla simple	Ventana 7 / 30 / 90	pandas	No medido
ETS	Suavizado exponencial	`trend="add"`, `damped_trend=True`, `seasonal="add"`	statsmodels	No medido
Theta	Estadístico	`season_length=24`	statsforecast	No medido
ARIMA(1,0,2)	Box–Jenkins	Orden por grid AIC + validación	statsmodels	7.4 s (grid, Cap. 3)
SARIMA / SARIMAX	Box–Jenkins estacional	`order=(1,0,1)`, `seasonal_order=(1,0,0,24)`	statsmodels	107.8 s (3 folds, Cap. 4)
Linear / Ridge	Lineal regularizado	`alpha=10.0` (Ridge)	scikit-learn	0.05–0.09 s / fold (Cap. 5)
Random Forest	Ensamble de árboles	`n_estimators=120`, `max_depth=18`	scikit-learn	2.51 s / fold (Cap. 5)
LightGBM (regresión)	Gradient boosting	`n_estimators=180–250`, `learning_rate=0.05`, `num_leaves=31`	lightgbm	4.06 s/fold (Cap. 5); 57.25 s (Cap. 6)
LightGBM (cuantílico)	Gradient boosting	`objective="quantile"`, 5 modelos (α por cuantil)	lightgbm	Incluido en tiempo total (Cap. 9)
Perfil hora×día	Baseline no paramétrico	Media de train agrupada por (día, hora)	pandas	No medido
LocalTrend200	Tendencia local	Ajuste lineal sobre últimas 36 obs.	numpy	No medido
LSTM	Red recurrente	`hidden=32`, 1 capa	PyTorch	Cap. 7: sin desglose (23.3 s totales); 360.58 s (Cap. 8), incl. en 854.4 s
N-BEATS compacto	Red residual MLP	2 bloques, `hidden=64`	PyTorch	95.41 s (Cap. 8)
TFT-like	Transformer compacto	$d_{\text{model}} = 32$, 4 cabezas, 1 capa encoder	PyTorch	398.45 s (Cap. 8)

Tabla 56. Todos los modelos evaluados en la monografía, con hiperparámetros clave, librería y tiempo de ejecución cuando fue medido en el notebook de origen.

APÉNDICE C

Entorno computacional

Semilla	`SEED = 42`, fijada al inicio de cada uno de los ocho notebooks, antes de cualquier operación aleatoria
Dispositivo	CPU en los ocho notebooks; ninguno usó GPU (documentado explícitamente en los Capítulos 8 y 9)
PyTorch	`2.13.0+cu130`, verificada directamente contra el entorno virtual del repositorio (`.venv`); el sufijo `cu130` indica un build con soporte CUDA disponible, pero `torch.cuda.is_available()` devuelve `False` en esa instalación y la ejecución documentada corrió sobre `torch.device("cpu")`
Sistema operativo	Linux (confirmado por `uname` sobre el entorno de ejecución y consistente con las rutas de archivo observadas en los outputs de los notebooks)
Otras librerías usadas	pandas 2.3.3, numpy 2.4.6, matplotlib 3.11.1, seaborn 0.13.2, scipy 1.17.1, statsmodels 0.14.6, statsforecast 2.1.1, scikit-learn 1.9.0, lightgbm 4.7.0 — versiones verificadas contra el entorno virtual del repositorio, no contra los outputs de texto de los notebooks

Tabla 57. Información de entorno recuperable directamente de los ocho notebooks.

Tiempos de ejecución documentados por capítulo (secciones específicas cronometradas dentro de cada notebook, no necesariamente el tiempo total del notebook completo):

Capítulo	Sección cronometrada	Tiempo
3	Grid search de 18 órdenes ARIMA sobre train	7.4 s
4	Backtesting SARIMA + SARIMAX, 3 folds	107.8 s
5	Walk-forward, 4 modelos, 3 folds	20.2 s
6	Ajuste de LightGBM global cold-start	57.25 s
7	Entrenamiento y evaluación, 4 modelos, 3 folds	23.3 s
8	Entrenamiento de N-BEATS, LSTM y TFT-like	854.4 s

Nota sobre discrepancias de tiempo total. El README del repositorio estima el tiempo de ejecución del notebook de la Semana 7 (Capítulo 8) en “ 10 minutos”. La sección de entrenamiento de arquitecturas documentada en este mismo capítulo, por sí sola, toma 854.4 segundos —14.2 minutos—, ya por encima de esa estimación total. Esta monografía no puede reconciliar la

discrepancia sin acceso al tiempo de pared completo del notebook (incluyendo carga de datos, EDA y todas las celdas de visualización), y la reporta en vez de ocultarla.

Artefactos de reproducibilidad verificados directamente contra el repositorio. A diferencia de una versión previa de este apéndice, los siguientes extractos se verificaron contra los archivos reales del repositorio `course-timeseries-benchmarks-pro`, no solo contra los outputs de texto de los notebooks.

`requirements.txt` (rangos compatibles, no versiones exactas — ver Apéndice C, tabla de librerías, para las versiones instaladas):

```
pandas>=2.1,<3
numpy>=1.26,<3
requests>=2.31,<3
datasetsforecast>=1.0,<2
py7zr>=0.22,<1
orjson>=3.10,<4
PyYAML>=6,<7
pytest>=8,<9
matplotlib>=3.8,<4
seaborn>=0.13,<1
scipy>=1.11,<2
statsmodels>=0.14,<1
statsforecast>=1.7,<3
scikit-learn>=1.4,<2
jupyter>=1.0,<2
nbformat>=5.9,<6
nbclient>=0.10,<1
lightgbm>=4,<5
torch>=2.2,<3
```

Extracto de `registry.yaml` (entrada del dataset del Capítulo 6, OPSD Germany, como muestra representativa del catálogo de ocho entradas):

```
- id: 06_opsd
  week: 6
  title: Open Power System Data Germany
  files:
  - data/06_opsd/germany_energy.csv
  rows: 50401
  columns: 43
  frequency: hourly
  target: DE_load_actual_entsoe_transparency
  covariates: load forecasts, prices, solar/wind generation, profiles and capacities
```

Extracto del receipt JSON del mismo dataset (`data/06_opsd/germany_energy.csv.receipt.json`), que documenta procedencia y transformación:

```
{
  "created_at_utc": "2026-08-05T21:56:13.062404+00:00",
```

```
  "source": {
    "url": "https://data.open-power-system-data.org/time_series/...",
    "downloaded_bytes": 130339665,
    "source_sha256": "6a7f2bc571314cbf9c321cc03437691cd4be95..."
  },
  "output": {
    "path": "germany_energy.csv",
    "bytes": 14880014,
    "sha256": "b0aa46353d19a7c0d398f737b22ea4bf7a3041666..."
  },
  "details": {
    "frequency": "hourly",
    "cleaning": "selected timestamp and all DE_ columns from European single-index file",
    "rows": 50401
  }
}
```

Encabezado de `MANIFEST.sha256` (hash SHA-256 de cada archivo del release; el archivo completo cubre todo el repositorio, incluyendo código, datos y entorno):

```
b98fd67f95a913a6fef5f5366fce15975234292b038f82faa109e8abcf84b5ec  .github/workflows/ci.yml
2fd42e8f3e50e405928c84048e682961ac798e100cab8a89ee3d45aa2b195a01  .gitignore
49a506dd32096b010d75205acf3430c9ae6c40351888129499e5a5e487126c93  .python-version
```

Instrucciones mínimas de reproducción (documentadas en `docs/REPRODUCIBILITY.md` del repositorio):

```
sha256sum -c MANIFEST.sha256
python scripts/validate_repository.py
python -m pytest -q
python scripts/run_notebooks.py --notebook 03
python scripts/run_notebooks.py --all --timeout 1800
```

Reproducibilidad byte-level: contenedor. El `requirements.txt` acota las versiones de Python, pero no las dependencias de sistema —compilador, BLAS, bibliotecas C++ de LightGBM — que también determinan el resultado numérico. El siguiente `Dockerfile` fija ambas capas, de modo que un auditor o un equipo de plataformas levante el mismo entorno en CI/CD sin fricción (un `.devcontainer.json` puede reutilizar la misma imagen como base):

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
RUN apt-get update \
 && apt-get install -y --no-install-recommends build-essential git \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "scripts/run_notebooks.py", "--all", "--timeout", "1800"]
```

El propio `docs/REPRODUCIBILITY.md` advierte que la ejecución completa puede generar tiempos y optimización numérica ligeramente distintos entre plataformas —las semillas reducen, pero no eliminan, diferencias de BLAS y versión de librería— y fija una política de resultados explícita: no copiar métricas fuera de su contexto, y registrar commit, hash de dataset, entorno, split, horizonte, features, modelo y métricas junto a cualquier número reportado.

APÉNDICE D

Methodology Cards unificadas

Cap.	Foco	Criterio de uso	Advertencia central
2	Baselines	Comparar toda complejidad adicional contra reglas simples antes de avanzar	Los gaps horarios hacen que “168 filas antes” no siempre equivalga a “168 horas antes”
3	Cuándo usar ARIMA	Serie única, frecuencia regular, dependencia lineal, residuos diagnosticables	Evitar con miles de series, covariables futuras desconocidas o cambios de régimen frecuentes
4	Exógenas	Usar una covariable solo si mejora la validación, es estable entre folds y existe en el horizonte	Clima observado en test es un escenario oracle; producción exige pronóstico meteorológico real
5	Checklist anti-leakage	Siete puntos: lags con `shift`, rolling sobre pasado, escalador solo en train, embargo, exclusión de componentes del target, disponibilidad futura real, test fuera de selección	Ninguno de los siete puntos es opcional ni sustituible por los demás
6	Global vs. local	El cold-start estricto prohíbe lags, identificador y **target encoding** del cliente nuevo, por definición	Comparar siempre el modelo global contra un baseline transparente antes de atribuirle valor
7	Cuándo usar deep learning	Historia suficiente, múltiples covariables, patrones no lineales	No usar por defecto con pocos datos o cuando un baseline / gradient boosting ya resuelve el problema
8	Selección de arquitectura	N-BEATS para univariado sin covariables; LSTM para recurrencia compacta; Transformer/TFT-like para atención temporal	Un TFT-like no es un TFT completo; debe etiquetarse honestamente, sin afirmar equivalencia
9	Producción probabilística	Entregar cuantiles, verificar ausencia de cruces, calibrar por segmento	Un intervalo estrecho pero descalibrado no es mejor que uno ancho y bien calibrado

Tabla 58. Las ocho Methodology Cards de la monografía, normalizadas a un formato comparable de cuatro campos.

APÉNDICE E

Glosario

ACF (función de autocorrelación) Mide la correlación de una serie con sus propios rezagos en distintos lags; se usa para identificar estructura de dependencia y órdenes candidatos de modelos ARMA.

ADF (Augmented Dickey-Fuller) Test de estacionariedad cuya hipótesis nula es la presencia de raíz unitaria (no estacionariedad).

ARCH LM Test cuya hipótesis nula es la homocedasticidad condicional; su rechazo indica heterocedasticidad, es decir, varianza del error que cambia en el tiempo.

Cobertura empírica Fracción de observaciones reales que caen dentro de un intervalo de predicción declarado, comparada contra el nivel nominal de ese intervalo.

Cold-start Escenario de evaluación donde el modelo debe predecir sobre una entidad —serie, cliente, sensor— sin ninguna historia previa disponible.

CRPS (Continuous Ranked Probability Score) Medida de distancia entre una distribución de pronóstico completa y el valor observado; en esta monografía se aproxima integrando **pinball loss** sobre un conjunto discreto de cuantiles.

Cross-learning Capacidad de un modelo global de transferir patrones aprendidos de un conjunto de series hacia otras series, incluyendo potencialmente series nunca vistas en entrenamiento.

Embargo temporal Brecha deliberada entre el final del bloque de entrenamiento y el inicio del bloque de test, para evitar que features con ventanas móviles compartan información entre ambos bloques.

Escenario oracle Evaluación que usa el valor real (observado) de una covariable que en producción solo estaría disponible como pronóstico, sobreestimando el desempeño alcanzable en despliegue real.

Horizonte de pronóstico Número de pasos hacia el futuro que un modelo debe predecir en una evaluación dada.

Jarque-Bera Test cuya hipótesis nula es la normalidad de una distribución, basado en su asimetría y curtosis muestral.

KPSS Test de estacionariedad cuya hipótesis nula es, al contrario del ADF, que la serie **sí** es estacionaria; se usa junto al ADF porque ambos responden hipótesis complementarias.

Leakage (fuga de información) Uso, directo o indirecto, de información no disponible en el momento real de la predicción durante el entrenamiento o la validación de un modelo.

Ljung-Box Test cuya hipótesis nula es la ausencia de autocorrelación conjunta hasta un lag determinado en una serie de residuos.

Lookback (ventana de contexto) Número de observaciones pasadas que un modelo secuencial usa como entrada para producir cada predicción.

MAE (Mean Absolute Error) Promedio del valor absoluto de los errores de pronóstico.

MASE (Mean Absolute Scaled Error) MAE escalado respecto al de un baseline estacional ingenuo; un valor menor a 1 indica que el modelo supera a ese baseline.

Methodology Card Ficha que documenta de forma estandarizada la pregunta, el split, el horizonte, las features, los modelos, las métricas, la semilla y las limitaciones de un experimento de forecasting.

Modelo global vs. modelo local Un modelo global comparte parámetros y aprende de muchas series simultáneamente; un modelo local se ajusta de forma independiente para cada serie individual.

Model Card Documento que resume un modelo ya seleccionado para producción, incluyendo su propósito, límites de uso, métricas, propietario y criterio de reentrenamiento.

Pinball loss Función de pérdida asimétrica usada para evaluar la calidad de un cuantil específico de una predicción probabilística.

PSI (Population Stability Index) Medida de cuánto ha cambiado la distribución de una variable —por ejemplo, el error de un modelo— entre un periodo de referencia y un periodo posterior; se usa para detectar **drift**.

RMSE (Root Mean Squared Error) Raíz del error cuadrático medio; penaliza más los errores grandes que el MAE.

Rolling-origin Estrategia de validación temporal con múltiples orígenes de pronóstico sucesivos, cada uno con su propio bloque de entrenamiento y de test.

STL (Seasonal-Trend decomposition using Loess) Descomposición que separa una serie en tendencia, componente estacional y residuo.

Walk-forward validation Estrategia de validación temporal donde el conjunto de entrenamiento se expande o se desliza hacia adelante en el tiempo, evaluando siempre sobre el bloque inmediatamente posterior.

WAPE (Weighted Absolute Percentage Error) Error absoluto total dividido entre la suma de los valores observados, expresado como porcentaje.

APÉNDICE F

Bibliografía

Referencias completas en formato APA. La primera parte reúne los autores citados a lo largo de la monografía (Sección 1.7 y siguientes); la segunda documenta la fuente primaria de cada uno de los ocho datasets, con su DOI o URL, tal como constan en las fichas `README.md` de `data/` y en `docs/SOURCE_NOTES.md` del repositorio.

Referencias metodológicas y de herramientas

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time series analysis: Forecasting and control* (5th ed.). Wiley.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.

Gneiting, T., & Katzfuss, M. (2014). Probabilistic forecasting. *Annual Review of Statistics and Its Application*, *1*, 125–151. https://doi.org/10.1146/annurev-statistics-062713-085831

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). O'Reilly Media. https://otexts.com/fpp3

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, *22*(4), 679–688. https://doi.org/10.1016/j.ijforecast.2006.03.001

Lazzeri, F. (2020). *Machine learning for time series forecasting with Python*. Wiley.

Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, *37*(4), 1748–1764. https://doi.org/10.1016/j.ijforecast.2021.03.012

Nielsen, A. (2019). *Practical time series analysis: Prediction with statistics and machine learning*. O'Reilly Media.

Oreshkin, B. N., Carpov, D., Chapados, N., & Bengio, Y. (2020). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*.

Shumway, R. H., & Stoffer, D. S. (2017). *Time series analysis and its applications: With R examples* (4th ed.). Springer.

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps*. O'Reilly Media.

Fuentes de los datasets

Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, *140*, 81–97. https://doi.org/10.1016/j.enbuild.2017.01.083

Chen, C., Petty, K., Skabardonis, A., Varaiya, P., & Jia, Z. (2001). Freeway Performance Measurement System: Mining loop detector data. *Transportation Research Record*, *1748*, 96–102. https://doi.org/10.3141/1748-12

Fanaee-T, H., & Gama, J. (2014). Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, *2*, 113–127. https://doi.org/10.1007/s13748-013--0040-3 (dataset: https://doi.org/10.24432/C5W894)

Liang, X., Zou, T., Guo, B., Li, S., Zhang, H., Zhang, S., Huang, H., & Chen, S. X. (2015). Assessing Beijing's PM2.5 pollution: Severity, weather impact, APEC and winter heating. *Proceedings of the Royal Society A*, *471*, 20150257. https://doi.org/10.1098/rspa.2015.0257 (dataset: https://doi.org/10.24432/C5JS49)

Makridakis, S., & Hibon, M. (2000). The M3-Competition: Results, conclusions and implications. *International Journal of Forecasting*, *16*(4), 451–476. https://doi.org/10.1016/S0169-2070(00)00057-1 (dataset: https://doi.org/10.5281/zenodo.4656298)

UCI Machine Learning Repository. (n.d.). *Metro Interstate Traffic Volume*. https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

UCI Machine Learning Repository. (n.d.). *ElectricityLoadDiagrams20112014*. https://archive.ics.uci.edu/dataset/321/electricityloaddiagrams20112014

Wiese, F., Schlecht, I., Bunke, W.-D., Gerbaulet, C., Hirth, L., Jahn, M., Kunz, F., Lorenz, C., Mühlenpfordt, J., Reimann, J., & Schill, W.-P. (2019). Open Power System Data – Frictionless data for electricity system modelling. *Applied Energy*, *236*, 401–409. https://doi.org/10.1016/j.apenergy.2018.11.097